VOLUME 02: TRACK 1 - HOST ENGINE & VIRTUAL MACHINE MANUAL

Chapter 9: Track 1 - Diagnostics & 4-Block Linter Subsystem

Preflight Boundary Audits, 4-Block Slicing, SVO Syntax, Scaffolding & QuadState Gates

> ***Witness:*** *Colossians 2:19 (KJV) - The whole body fitly joined together*

SYSTEM TRACK
<TRACK 1> : {Host Engine & Virtual Machine}

CHAPTER IDENTIFIER
CH 09

NOTEBOOKLM WORD COUNT
24,744 words

CATALOGED CHAPTER FILES
38 files (5,699 lines)

RECOMMENDED NOTEBOOKLM COGNITIVE QUERY

"Why is sensory telemetry logging inside the linter essential for the autonomic DAR self-healing loop?"

CHAPTER MANIFEST & INDEX (38 FILES)

#	FILE PATH	TYPE	LINES	WORDS	SIZE (KB)
1	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/README.md**	MD	176	1,229	12.3
2	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/WHY_LINTER_LOGGING_IS_ESSENTIAL.md**	MD	131	890	8.7
3	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/audit.rs**	RS	180	731	6.8
4	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/mod.rs**	RS	472	1,675	19.5
5	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/structure/README.md**	MD	89	485	4.5
6	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/structure/detector.rs**	RS	283	1,149	10.5
7	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/structure/mod.rs**	RS	110	436	4.1
8	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/structure/omni.rs**	RS	240	939	10.0
9	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/structure/tape.rs**	RS	293	1,159	12.5
10	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/opening/README.md**	MD	71	359	2.9
11	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/opening/mod.rs**	RS	134	479	5.5
12	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/opening/reel.rs**	RS	106	508	4.6
13	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/opening/score.rs**	RS	71	267	2.5
14	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/opening/tape_setup.rs**	RS	112	456	4.4
15	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/body/README.md**	MD	70	446	3.4
16	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/body/mod.rs**	RS	135	489	5.5
17	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/body/reel.rs**	RS	115	508	4.8
18	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/body/score.rs**	RS	71	266	2.5
19	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/body/tape_rolls.rs**	RS	116	512	4.6
20	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/preflight/README.md**	MD	97	541	4.6
21	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/preflight/assembly.rs**	RS	283	1,055	12.9
22	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/preflight/mod.rs**	RS	71	265	2.7
23	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/closing/README.md**	MD	69	413	3.1
24	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/closing/mod.rs**	RS	134	493	5.6
25	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/closing/reel.rs**	RS	113	551	5.1
26	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/closing/score.rs**	RS	71	271	2.6
27	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/closing/tape_commit.rs**	RS	112	499	4.6
28	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/syntax/README.md**	MD	91	555	4.5
29	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/syntax/grammar.rs**	RS	223	904	9.3
30	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/syntax/mod.rs**	RS	70	283	2.8
31	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/syntax/spatial.rs**	RS	191	706	7.9
32	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/metadata/README.md**	MD	72	446	3.1
33	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/metadata/head.rs**	RS	247	953	10.3
34	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/metadata/mod.rs**	RS	168	617	7.6
35	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/metadata/reel.rs**	RS	250	995	10.6

#	FILE PATH	TYPE	LINES	WORDS	SIZE (KB)
36	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/metadata/score.rs**	RS	104	452	4.2
37	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/scaffold/README.md**	MD	92	427	3.6
38	**<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/scaffold/mod.rs**	RS	266	1,335	10.3

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/README.md | MARKDOWN / DOC | 176 LINES | 1229 WORDS | 12.3 KB

OmniCode Diagnostic Linter Subsystem

> ***"A false balance is abomination to the LORD: but a just weight is his delight."*** *— Proverbs 11:1 (KJV)*
>
> ***"Let all things be done decently and in order."*** *— 1 Corinthians 14:40 (KJV)*

1. Overview & Architectural Purpose

The **OmniCode Linter Subsystem** (`src/diagnostics/linter`) is the foundational static analysis, syntax verification, structural gatekeeper, and automated remediation engine of the OmniCode Host Engine (Track 1).

Unlike legacy compiler linters that merely dump text warnings to `stderr`, the OmniCode Linter operates as an active **sensory and immune organ**. It verifies source files (`.omni` and `.tape`) against the canonical language specifications demonstrated in `character.omni`, `character.tape`, `number.omni`, and `number.tape`. It checks strict conformance to the **4-Block Architecture**, validates multi-tier syntactic boundaries (including asymmetric emergences `[ ... }` and impositions `{ ... ]`), computes a deterministic [0–100] health score, and transmits rich telemetry directly into the `DiagnosticLogger`.

Core Responsibilities

1. **Multi-Pass Diagnostic Pipeline (Passes 1–6)**:
2. **Pass 1: Preflight Boundary & Line Length Audit** (`preflight`): Ensures 120-column spatial ceilings, checks UTF-8 validity, scans delimiter stacks, and generates automated `FixSuggestion` patches.
3. **Pass 2: 4-Block Structural Slicing** (`structure`): Detects whether a file is an `.omni` specification or `.tape` execution stream, extracting the 4 required block regions:
   - `BLOCK 1: METADATA` (`metadata`)
   - `BLOCK 2: OPENING` (`opening`)
   - `BLOCK 3: BODY` (`body`)
   - `BLOCK 4: CLOSING` (`closing`)
4. **Pass 3: Grammatical & Spatial Syntax Verification** (`syntax`): Checks Subject-Verb-Object (SVO) alignment, bracket naming rules, whitespace discipline, bit rail identifier rules (`Bit_0` .. `Bit_7`), and binary literal formatting (`0b`).
5. **Pass 4: Multi-Block Quality Scoring & Weight Balancing**: Computes an aggregate score from individual block metrics.
6. **Pass 5: Scaffold Generation & Alignment** (`scaffold`): Generates canonical templates for newly scaffolded `.omni` and `.tape` files.
7. **Pass 6: QuadState Gate Decision & Flight Recording** (`audit`): Issues a formal `QuadStateGate` verdict (`Proceed`, `Await`, `Rollback`, or `Halt`) and logs telemetry to the in-memory flight recorder.

2. Directory Architecture

```
src/diagnostics/linter/
├── README.md                          <-- You are here (Subsystem Master Reference)
├── WHY_LINTER_LOGGING_IS_ESSENTIAL.md  <-- Deep treatise on telemetry, self-healing, & AI
├── mod.rs                             <-- OmniLinter facade & coordinator
├── audit.rs                           <-- LintAudit, QuadStateGate, and score balancing
├── structure/                         <-- Pass 2: 4-Block boundary detector & partitioner
│   ├── README.md                      <-- Structure guide (.omni and .tape slicing)
│   ├── mod.rs                         <-- Structure coordinator
│   ├── detector.rs                    <-- Slicing regex & header discovery
│   ├── omni.rs                        <-- Omni-specific block validator
│   └── tape.rs                        <-- Tape-specific block validator
├── metadata/                          <-- Block 1 linter (Authorship, Epoch, Scripture)
│   ├── README.md                      <-- Block 1 documentation
│   └── mod.rs                         <-- Metadata header, tag, & docstring auditor
├── opening/                           <-- Block 2 linter (Dependencies, Imports, Seeds)
│   ├── README.md                      <-- Block 2 documentation
```

```
│   └── mod.rs                      <-- Opening imports & environment setup auditor
├── body/                           <-- Block 3 linter (Types, State, SVO Grammars)
│   ├── README.md                   <-- Block 3 documentation
│   └── mod.rs                      <-- Core body structural & logic auditor
├── closing/                        <-- Block 4 linter (Teardown, Proofs, Witnesses)
│   ├── README.md                   <-- Block 4 documentation
│   └── mod.rs                      <-- Verification proofs & cleanup auditor
├── syntax/                         <-- Pass 3: Detailed grammatical & spatial syntax
│   ├── README.md                   <-- Syntax rules & boundary constraints
│   ├── mod.rs                      <-- Syntax coordinator
│   ├── grammar.rs                  <-- SVO verb rules, keyword validation
│   └── spatial.rs                  <-- 120-col limits, bracket identifiers, tabs vs spaces
├── preflight/                      <-- Pass 1: Raw line length & delimiter stack analysis
│   ├── README.md                   <-- Preflight assembly & delimiter repair guide
│   ├── mod.rs                      <-- Preflight coordinator
│   └── assembly.rs                 <-- Delimiter stack validator & FixSuggestion generator
└── scaffold/                       <-- Pass 5: Canonical 4-block boilerplate generation
    ├── README.md                   <-- Scaffold generator guide
    └── mod.rs                      <-- Scaffold generator for .omni and .tape files
```

3. Why Linter Logging is Essential: The Cognitive & Autonomic Feedback Loop

See the full treatise at `WHY_LINTER_LOGGING_IS_ESSENTIAL.md` .

Now that the `DiagnosticLogger` is fully operational with zero-allocation ring buffers, persistent inscribers, failpoint remediations, and episodic neural-symbolic memory, logging within the linter transforms it into a dynamic feedback control system:

```
graph TD
    A[Source Code: .omni / .tape] -->|lint_source| B[OmniLinter]
    B -->|Preflight Delimiter Anomaly| C[DiagnosticLogger Failpoint]
    C -->|FixSuggestion| D[Restorator DAR Loop]
    D -->|Apply Auto-Patch| A
    B -->|Pass 6 RollFrame| E[FlightRecorder RingBuffer]
    B -->|Incident CognitiveNote| F[Episodic Memory]
    F -->|Query Past Anomalies| G[AICortex Generation]
    B -->|QuadStateGate Verdict| H{Compiler Gate}
    H -->|Proceed +1| I[Track 1 VM Bytecode Pipeline]
    H -->|Await 0 / Rollback -1| J[Halt & Remediation]
```

1. **Autonomic DAR Loop (Detect, Assess, Remediate)**: The linter emits structured `FixSuggestion` instances with exact line, column, target token, replacement token, and mathematical rationale. The `Restorator` can consume these logs and heal broken boundaries automatically.
2. **AI Reflection via Episodic Memory**: `AICortex` inspects prior linter `CognitiveNote` entries before code generation, eliminating repetitive hallucinations and ensuring new code strictly follows the 4-block layout and boundary grammar.
3. **Flight Recorder Post-Mortem Diagnostics**: Every lint pass appends a lightweight `RollFrame` into the ring buffer. If a compilation step halts unexpectedly downstream, the last anomalous frame can be queried instantly.

4. QuadState Audit Gate Mechanics

The linter scores files out of 100 points and assigns a `QuadStateGate` (`audit.rs`):

QuadState Verdict	Score Threshold	System Meaning	Compilation Gate Behavior
`QuadStateGate::Proceed`	`90.0 ..= 100.0`	Pristine code, zero structural errors.	Passes assembly gate; bytecode generation permitted.
`QuadStateGate::Await`	`75.0 .. < 90.0`	Minor formatting or style warnings.	Permitted with warnings; flagged for developer review.
`QuadStateGate::Rollback`	`50.0 .. < 75.0`	Missing required blocks or boundary mismatch.	Assembly blocked; trigger auto-remediation DAR loop.

QuadState Verdict	Score Threshold	System Meaning	Compilation Gate Behavior
`QuadStateGate::Halt`	`< 50.0`	Catastrophic structural corruption or fatal parse error.	Immediate abort; sound warning alarm in EventBus.

5. Parity with Canonical Specifications (`character` & `number`)

The linter is calibrated against the gold standard files:

- `lexicon/01_nouns/ideas/types/character.omni`
- `lexicon/01_nouns/ideas/types/character.tape`
- `lexicon/01_nouns/ideas/types/number.omni`
- `lexicon/01_nouns/ideas/types/number.tape`

Key Syntactic & Structural Rules Enforced

1. **Four Explicit Blocks**: Must define Block 1 (Metadata), Block 2 (Opening), Block 3 (Body), and Block 4 (Closing).
2. **KJV Scripture Witness**: Metadata block must include an explicit Scripture witness citation (e.g. `Witness: Proverbs 11:1 (KJV)`).
3. **Asymmetric Boundary Support**:
4. `[ ... }` : `BottomUp` emergence boundary (Rank 4).
5. `{ ... ]` : `TopDown` imposition boundary (Rank 5).
6. **Identifier Exemptions**: Bit rail identifiers (`Bit_0` .. `Bit_7`) and binary numeric literals (`0b00000000`) are recognized and exempted from snake_case warnings.
7. **Canonical SVO Verbs**: Full support for verbs including `DEFINE` , `SET` , `EMIT` , `VERIFY` , `APPLY` , `PARTITION` , `CALCULATE` , and `CLOSE` .

6. Public API & Usage

```
use omnicode_host::diagnostics::linter::OmniLinter;
use omnicode_host::diagnostics::logger::DiagnosticLogger;

// 1. Create a logger with in-memory or terminal sink
let logger = DiagnosticLogger::new();

// 2. Instantiate OmniLinter with attached logger
let linter = OmniLinter::with_logger(logger.clone());

// 3. Lint an OmniCode source file
let report = linter.lint_source(source_text, "lexicon/01_nouns/ideas/types/number.omni");

// 4. Inspect report & QuadState audit gate
println!("Score: {}/100", report.score);
println!("Gate: {:?}", report.audit.gate);
if report.is_clean() {
    println!("Source verified and ready for bytecode assembly!");
} else {
    for err in &report.errors {
        eprintln!("Error: {}", err);
    }
}
```

7. Verification & Test Suite

The entire `linter` module contains 49 integrated unit tests verifying all passes:

```
cargo test --lib diagnostics::linter
```

All 49 unit tests pass with zero warnings, verifying full compatibility with both `.omni` and `.tape` specifications.

Why Linter Logging is Essential: Sensory Telemetry, Autonomic Self-Healing, and AI Reflection

> ***"A false balance is abomination to the LORD: but a just weight is his delight."*** *— Proverbs 11:1 (KJV)*
>
> ***"And he gave unto Moses, when he had made an end of communing with him upon mount Sinai, two tables of testimony, tables of stone, written with the finger of God."*** *— Exodus 31:18 (KJV)*

1. The Core Architectural Question: Why Feed the Logger?

In conventional software toolchains, a linter is treated as a detached, static verification script that runs in isolation, prints a few plain-text error messages or warnings to `stderr`, and exits with a non-zero exit code.

In OmniCode, this traditional paradigm is completely rejected.

Now that the `DiagnosticLogger` subsystem is modularized into its five specialized organs (`universal`, `failpoint`, `telemetry`, `inscriber`, and `query`), the `OmniLinter` functions as the primary **sensory organ** of the Host Engine (Track 1).

Instead of letting errors disappear into terminal scrollback, the linter actively streams:

1. **Structured Failpoints with Machine Remediation** (`FixSuggestion`),
2. **Circular Execution Flight Frames** (`RollFrame`),
3. **Neural-Symbolic Episodic Cognitive Notes** (`CognitiveNote`), and
4. **QuadState Compilation Gate Invariants** (`QuadStateGate`).

2. What This Enables the System to Do More Effectively

By coupling the linter directly with the diagnostic logger, the system achieves four fundamental capabilities that are impossible in legacy compilers:

A. The Autonomic DAR Loop (Detect, Assess, Remediate)

When the linter detects a syntax anomaly (such as an illegal boundary pair `< 1 ]` or an unclosed delimiter `{ Cell [ 1 ]`), it does not just complain; it generates a concrete [`FixSuggestion`] containing:

- The exact source `line` and `col`,
- The erroneous `target_token` (e.g. `"]"`),
- The suggested mathematical patch (e.g. `">"`), and
- The architectural `rationale`.

```
sequenceDiagram
    participant Source as Source Code (.omni / .tape)
    participant Linter as OmniLinter
    participant Logger as DiagnosticLogger
    participant Restorator as Restorator Immune System
    participant Debugger as OmniDebugger
    participant Oracle as Test Oracle

    Source->>Linter: lint_source()
    Linter->>Linter: Pass 2: Delimiter Stack Audit
    Note over Linter: Detected Illegal Crossing `< 99 ]`
    Linter->>Logger: failpoint_with_remediation(BoundaryMismatch, FixSuggestion)
    Logger-->>Restorator: Broadcast Failpoint Event
    Restorator->>Debugger: execute_restoration_from_log()
    Debugger->>Source: Apply Substitution (']' -> '>')
    Debugger->>Oracle: Validate Healed Code
```

```
    Oracle-->>Restorator: Certification: OK (+1:PROCEED)
    Restorator->>Logger: record_ai_observation("REPAIR_VERIFIED")
```

The `Restorator` and `OmniDebugger` consume this structured telemetry directly from `logger.failpoints()` and `logger.remediations()`. They apply the patch in isolated scratch memory, verify it against an automated test oracle, and write the healed code back to disk without human intervention!

B. Non-Hallucinatory AI Cognitive Reflection

In Track 1, the `AICortex` is responsible for generating, understanding, and modifying code. A common failure mode of AI assistants in other ecosystems is **repetitive hallucination**: an AI agent generates code with the same invalid delimiter or missing block header repeatedly because it has no memory of past lint failures.

Because the linter logs [`CognitiveNote`] records into episodic memory:

```
self.logger.record_ai_observation(
    "LINTER_INCIDENT",
    &format!("Audit veto for [{file_path}]: {}", audit.gate.name()),
    &format!("Errors: {}", report.errors.join("; ")),
    &["linter", "audit", "syntax_anomaly", "veto"],
    Some(file_path),
);
```

The AI Cortex can query its own episodic memory via `EpisodicQueryEngine` before attempting code generation:

1. "What syntax anomalies were recorded in previous sessions for `lexicon/01_nouns/ideas/types/` ?"
2. "Which delimiters frequently mismatched, and what were the verified remediations?"
3. "Did the last assembly run result in `QuadState::Proceed` or `QuadState::Rollback` ?"

This grounds AI generation in deterministic historical fact rather than statistical guessing.

C. Ring Buffer Flight Recording for Crash Diagnostics

The `FlightRecorder` maintains a zero-allocation circular buffer of execution roll frames.

When `OmniLinter::lint_source` runs, it records a [`RollFrame`] representing the final Pass 6 audit gate:

- Roll Index: 6/6
- Verb: `"LINT_AUDIT_GATE"`
- QuadState: `Proceed` , `Await` , `Rollback` , or `Halt`
- Failpoint Mask: `0` (Clean) or `1` (Anomaly)
- Intent Aside: Human-readable composite score and gate summary.

If a compilation or hypervisor command crashes later in the execution pipeline, the engineer can inspect `logger.recent_roll_frames(10)` or `logger.last_anomalous_frame()` to see the exact state of the linter immediately prior to the failure.

D. Silicon-to-Syntax Invariant Attestation (Ω_0)

In bare-metal execution (Track 0), the operating system boots from ancient silicon landmarks (`0x7C00` , `0x18000` , `0x108000` , `0x200000`). It establishes a homeostatic equilibrium state known as **State Ω_0**.

Before allowing code to lower into physical machine words, the Host Engine queries `logger.audit_shutdown_equilibrium()` and verifies that:

1. The genesis baseline is clean.
2. The compilation unit's composite 4-Block score satisfies the gate (`QuadStateGate::Proceed` with composite ≥ 70).
3. Zero active unhealed failpoints exist on the diagnostic bus.

If any invariant fails, execution is halted safely, preventing memory corruption or CPU triple faults on bare metal.

---

3. Grounding in the Canonical Types: Character & Number

The two most advanced compilation units in the language today are:

1. `character.omni` / `character.tape`
2. `number.omni` / `number.tape`

Both files achieve a perfect **100/100 composite score** and **QuadStateGate::Proceed** across all 4 blocks:

- **Block 1 (Metadata)**: 100/100 (Identity 20, Provenance 25, Witness 20, Ontology 20, Spatial 15)

- **Block 2 (Opening)**: 100/100 (Entity 25, Cell/Word 20, Transducer 20, Preconditions 20, Spatial 15)
- **Block 3 (Body)**: 100/100 (SVO 25, Vectors 20, Primes 20, Guards 20, Spatial 15)
- **Block 4 (Closing)**: 100/100 (Verify 20, Validate 20, Tripwire 20, Fourfold 25, Telemetry 15)

By capturing these 100/100 baselines in the logger's episodic memory, any future syntax regression in user code or standard library modules is detected in $O(1)$ time relative to these certified canonical exemplars.

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/audit.rs RS 180 LINES 731 WORDS 6.8 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Universal 4-Block File Audit Score & QuadState Gate Taxonomy )
// ============================================================================
//! # Universal 4-Block File Audit Score & QuadState Compilation Gate
//!
//! Synthesizes the four block-level invariant scores into a single composite
//! metric on the continuous balanced scale $[-100 \dots +100]$, and maps the result
//! to an operational QuadState compilation gate:
//! - `+1 : PROCEED` (Composite >= +70): Eligible to compile and lower to machine words.
//! - ` 0 : AWAIT`   (Composite +50..+70): Ambient equilibrium hold pending advisories.
//! - `-1 : ROLLBACK` (Composite +20..+50): Precondition/covenant deficit; causal tape rolls back.
//! - `HALT : NULL`  (Composite < +20): Ineligible to compile; execution stops.
//!
//! # Scripture Witness:
//! - **Proverbs 11:1 (KJV)**:
//!   "A false balance is abomination to the LORD: but a just weight is his delight."
//!   (The balanced audit metric ensures mathematical justice across all 4 structural blocks).
//! - **Leviticus 19:36 (KJV)**:
//!   "Just balances, just weights, a just ephah, and a just hin, shall ye have..."
//!   (Every block must carry honest, verified weight before entering execution).
// ============================================================================

use super::metadata::MetadataScore;
use super::opening::OpeningScore;
use super::body::BodyScore;
use super::closing::ClosingScore;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( QuadStateGate Enumeration & FileAuditScore Data Structures )
// ============================================================================

/// QuadState compilation gate denoting compilation eligibility.
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum QuadStateGate {
    Proceed,
    Await,
    Rollback,
    Halt,
}

impl QuadStateGate {
    /// Human-readable descriptor for the compilation gate.
    pub fn name(&self) -> &'static str {
        match self {
            QuadStateGate::Proceed => "+1 : PROCEED (Eligible to Compile)",
            QuadStateGate::Await => " 0 : AWAIT (Advisory Equilibrium Hold)",
            QuadStateGate::Rollback => "-1 : ROLLBACK (Invariant Reversion)",
            QuadStateGate::Halt => "HALT : NULL (Compilation Disqualified)",
        }
    }

    /// Returns `true` if the compilation gate permits advancing forward to compilation.
    pub fn is_proceed(&self) -> bool {
        matches!(self, QuadStateGate::Proceed)
    }
}

/// Composite 4-Block audit score synthesizing invariant health across all four blocks.
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct FileAuditScore {
    pub block1_metadata: MetadataScore,
    pub block2_opening: OpeningScore,
    pub block3_body: BodyScore,
    pub block4_closing: ClosingScore,
    pub composite_score: i32,
    pub gate: QuadStateGate,
    pub eligible_to_compile: bool,
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Metric Calculation, Clamping, and Gate Resolution )
```

```rust
// ============================================================================

impl FileAuditScore {
    /// Computes the composite 4-Block audit score and resolves the QuadState gate.
    pub fn calculate(
        block1: MetadataScore,
        block2: OpeningScore,
        block3: BodyScore,
        block4: ClosingScore,
        has_fatal_errors: bool,
    ) -> Self {
        let b1 = block1.total as i32;
        let b2 = block2.total;
        let b3 = block3.total;
        let b4 = block4.total;

        let composite = ((b1 + b2 + b3 + b4) / 4).clamp(-100, 100);

        let gate = if has_fatal_errors || composite < 20 {
            QuadStateGate::Halt
        } else if composite >= 70 {
            QuadStateGate::Proceed
        } else if composite >= 50 {
            QuadStateGate::Await
        } else {
            QuadStateGate::Rollback
        };

        let eligible_to_compile = gate.is_proceed() && !has_fatal_errors;

        Self {
            block1_metadata: block1,
            block2_opening: block2,
            block3_body: block3,
            block4_closing: block4,
            composite_score: composite,
            gate,
            eligible_to_compile,
        }
    }

    /// Formats a human-readable telemetry summary for the audit scroll and logger.
    pub fn format_telemetry(&self) -> String {
        format!(
            "Audit Score: {}/100 => Gate: {} (B1: {}, B2: {}, B3: {}, B4: {})",
            self.composite_score,
            self.gate.name(),
            self.block1_metadata.total,
            self.block2_opening.total,
            self.block3_body.total,
            self.block4_closing.total
        )
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Unit Tests & Mathematical Verification )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_audit_score_perfect_composite_proceed() {
        let b1 = MetadataScore::new(20, 25, 20, 20, 15, vec!["OK".into()]);
        let b2 = OpeningScore::new(25, 20, 20, 20, 15, vec!["OK".into()]);
        let b3 = BodyScore::new(25, 20, 20, 20, 15, vec!["OK".into()]);
        let b4 = ClosingScore::new(20, 20, 20, 25, 15, vec!["OK".into()]);

        let audit = FileAuditScore::calculate(b1, b2, b3, b4, false);
        assert_eq!(audit.composite_score, 100);
        assert_eq!(audit.gate, QuadStateGate::Proceed);
        assert!(audit.eligible_to_compile);
        assert!(audit.format_telemetry().contains("100/100"));
    }

```

```
    #[test]
    fn test_audit_score_fatal_error_forces_halt() {
        let b1 = MetadataScore::new(20, 25, 20, 20, 15, vec![]);
        let b2 = OpeningScore::new(25, 20, 20, 20, 15, vec![]);
        let b3 = BodyScore::new(25, 20, 20, 20, 15, vec![]);
        let b4 = ClosingScore::new(20, 20, 20, 25, 15, vec![]);

        // Even with 100% composite score, fatal error triggers HALT
        let audit = FileAuditScore::calculate(b1, b2, b3, b4, true);
        assert_eq!(audit.composite_score, 100);
        assert_eq!(audit.gate, QuadStateGate::Halt);
        assert!(!audit.eligible_to_compile);
    }

    #[test]
    fn test_audit_score_tiers() {
        let b1_low = MetadataScore::new(5, 5, 5, 5, 5, vec![]);
        let b2_low = OpeningScore::new(5, 5, 5, 5, 5, vec![]);
        let b3_low = BodyScore::new(5, 5, 5, 5, 5, vec![]);
        let b4_low = ClosingScore::new(5, 5, 5, 5, 5, vec![]);

        let audit = FileAuditScore::calculate(b1_low, b2_low, b3_low, b4_low, false);
        assert_eq!(audit.composite_score, 25);
        assert_eq!(audit.gate, QuadStateGate::Rollback);
        assert!(!audit.eligible_to_compile);
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/mod.rs RS 472 LINES 1675 WORDS 19.5 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( OmniLinter Coordinator, Subsystem Exports & Invariant Architecture )
// ============================================================================
//! # OmniCode Diagnostics: Polymorphic Structural, Grammar & Spatial Linter
//!
//! Enforces structural invariants, lexical syntax, grammar ASTs, physical
//! spatial geometry, and assembly pre-flight certification across all OmniCode
//! compilation units and execution ribbons:
//! 1. `.omni` (Source Reel): Canonical 4-Block Architecture (Metadata, Opening, Body, Closing).
//! 2. `.tape` (Process Ribbon): Sequential Tape Head, Cardinal Coordinates, Rolls Stream & Commit Status.
//! 3. Spatial Geometry: Strict 80-column ruler for dual side-panel IDE viewports.
//! 4. Grammar & Syntax: SVO verbs, state-prime synchronization, Living Asides, and boundary physics.
//! 5. Assembly Pre-flight: Partition bounds, silicon MMIO targets, and delimiter stack parity.
//! 6. Autonomic Diagnostic Telemetry: Streams structured failpoints, machine-actionable
//!    remediations ([`FixSuggestion`]), flight recorder roll frames ([`RollFrame`]),
//!    and episodic cognitive notes ([`CognitiveNote`]) directly into [`DiagnosticLogger`].
//!
//! # Scripture Witness:
//! - **Leviticus 19:35-36 (KJV)**:
//!   "Ye shall do no unrighteousness in judgment, in meteyard, in weight, or in measure.
//!    Just balances, just weights, a just ephah, and a just hin, shall ye have..."
//!   (The foundational mandate of the linter: honest meteyard, true weights, and zero-compromise balance).
//! - **Proverbs 11:1 (KJV)**:
//!   "A false balance is abomination to the LORD: but a just weight is his delight."
//!   (Mathematical justice enforced across the 4-Block composite audit metric).
//! - **1 Corinthians 14:40 (KJV)**:
//!   "Let all things be done decently and in order."
//!   (Rhythmic spatial decency and causal monotonicity preserved across every line).
//! - **Job 38:5 (KJV)**:
//!   "Who hath laid the measures thereof, if thou knowest? or who hath stretched the line upon it?"
//!   (The plumbline of invariant geometry measuring every delimiter, word, and block).
// ============================================================================

use std::path::Path;

use crate::core::filetype::OmniFileType;
use crate::core::ternary::QuadState;
use crate::diagnostics::debugger::OmniDebugger;
use crate::diagnostics::logger::{DiagnosticLogger, FailpointType, RollFrame};

pub mod audit;
pub mod body;
pub mod closing;
pub mod metadata;
pub mod opening;
pub mod preflight;
pub mod scaffold;
pub mod structure;
pub mod syntax;

pub use audit::{FileAuditScore, QuadStateGate};
pub use body::{BodyLinter, BodyScore, ReelBodyLinter, TapeRollsLinter};
pub use closing::{ClosingLinter, ClosingScore, ReelClosingLinter, TapeCommitLinter};
pub use metadata::{MetadataLinter, MetadataScore, ReelMetadataLinter, TapeHeadLinter};
pub use opening::{OpeningLinter, OpeningScore, ReelOpeningLinter, TapeSetupLinter};
pub use preflight::AssemblyLinter;
pub use scaffold::LinterScaffolder;
pub use structure::{
    BlockDetector, BlockKind, BlockSlice, DetectedBlock, FileBlockSlices, OmniReelLinter, TapeLinter,
};
pub use syntax::{GrammarLinter, SpatialLintConfig, SpatialLinter};

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( LintReport & OmniLinter Initialization )
// ============================================================================

/// Diagnostic report containing all findings, block slices, and scores for a linted unit.
#[derive(Debug, Clone)]
pub struct LintReport {
    pub file_path: String,
    pub file_type: Option<OmniFileType>,
    pub blocks_found: Vec<DetectedBlock>,
```

```
    pub metadata_score: Option<MetadataScore>,
    pub audit_score: Option<FileAuditScore>,
    pub is_valid: bool,
    pub errors: Vec<String>,
    pub warnings: Vec<String>,
}

/// Central polymorphic linter orchestrating spatial, preflight, grammar, and structural passes.
pub struct OmniLinter {
    pub logger: DiagnosticLogger,
    pub debugger: OmniDebugger,
    pub config: SpatialLintConfig,
}

impl OmniLinter {
    /// Creates a new linter bound to a diagnostic logger with default spatial settings.
    pub fn new(logger: DiagnosticLogger) -> Self {
        let debugger = OmniDebugger::new(logger.clone(), 100);
        Self {
            logger,
            debugger,
            config: SpatialLintConfig::default(),
        }
    }

    /// Creates a new linter with custom debugger and spatial configuration.
    pub fn with_debugger(logger: DiagnosticLogger, debugger: OmniDebugger) -> Self {
        Self {
            logger,
            debugger,
            config: SpatialLintConfig::default(),
        }
    }

    /// Configures strict 80-column ruler enforcement.
    pub fn with_strict_geometry(mut self, strict: bool) -> Self {
        self.config.strict_column_limit = strict;
        self
    }
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Core Linting Pipelines: Polymorphic Dispatch & Telemetry Stream )
// ============================================================================

impl OmniLinter {
    /// Lints a source string across structural, grammar, and spatial geometry invariants,
    /// streaming rich failpoints, remediations, and cognitive notes into [`DiagnosticLogger`].
    pub fn lint_source(&self, file_path: &str, content: &str) -> LintReport {
        let file_type = OmniFileType::from_path(file_path);

        let mut report = LintReport {
            file_path: file_path.to_string(),
            file_type,
            blocks_found: Vec::new(),
            metadata_score: None,
            audit_score: None,
            is_valid: true,
            errors: Vec::new(),
            warnings: Vec::new(),
        };

        // Pass 1: Spatial Geometry & Layout Ruler (<= 80 columns)
        SpatialLinter::lint(content, &mut report, &self.config);

        // Pass 2: Delimiter & Assembly Pre-flight Checks (with logger for remediation synthesis)
        AssemblyLinter::preflight_with_logger(content, &mut report, &self.logger);

        // Pass 3: Lexical Syntax & Grammar AST Verification
        GrammarLinter::lint(
            file_path,
            content,
            self.logger.clone(),
            self.debugger.clone(),
            &mut report,
        );

```

```
        // Pass 4: Polymorphic Structural Architecture Dispatch
        let is_tape = match file_type {
            Some(OmniFileType::Tape) => true,
            _ => {
                let trimmed_start = content.trim_start();
                trimmed_start.starts_with("= [Tape")
                    || trimmed_start.starts_with("[Tape.")
                    || (trimmed_start.contains("[ ROLL") && !trimmed_start.contains("BLOCK 1"))
            }
        };

        if is_tape {
            TapeLinter::lint(file_path, content, &mut report);
        } else {
            OmniReelLinter::lint(file_path, content, &mut report);
        }

        // Pass 5: Block 1 Invariant & Metadata Scoring
        let meta_score = MetadataLinter::evaluate(file_path, content, is_tape, &self.config, &mut report);
        report.metadata_score = Some(meta_score.clone());

        // Pass 6: Full 4-Block Invariant Audit & QuadState Gate Synthesis
        let open_score = OpeningLinter::evaluate(file_path, content, is_tape, &self.config);
        let body_score = BodyLinter::evaluate(file_path, content, is_tape, &self.config);
        let close_score = ClosingLinter::evaluate(file_path, content, is_tape, &self.config);

        report.is_valid = report.errors.is_empty();

        let audit = FileAuditScore::calculate(
            meta_score,
            open_score,
            body_score,
            close_score,
            !report.is_valid,
        );
        report.audit_score = Some(audit.clone());
        report.is_valid = report.errors.is_empty();

        // ---------------------------------------------------------------------
        // Stream Telemetry & Cognitive Notes to DiagnosticLogger
        // ---------------------------------------------------------------------
        let quad_state = match audit.gate {
            QuadStateGate::Proceed => QuadState::Proceed,
            QuadStateGate::Await => QuadState::Await,
            QuadStateGate::Rollback => QuadState::Rollback,
            QuadStateGate::Halt => QuadState::Halt,
        };

        let elapsed = self.logger.clock().elapsed_micros();
        let frame = RollFrame::new(
            elapsed,
            6, // Final Pass 6 of Linter
            6,
            "LINT_AUDIT_GATE",
            (0x00, 0x00),
            (0x00, 0x00),
            quad_state,
            if !report.is_valid { 1 } else { 0 },
            Some(format!("{file_path} => {}", audit.format_telemetry())),
        );
        self.logger.record_roll_frame(frame);

        if !report.is_valid {
            for err in &report.errors {
                self.logger.failpoint(
                    FailpointType::SyntaxAnomaly,
                    "LINTER",
                    format!("[{file_path}] {err}"),
                );
            }
            // Cognitive Note for AI Cortex Continual Learning & Self-Healing
            self.logger.record_ai_observation(
                "LINTER_INCIDENT",
                &format!("Audit veto for [{file_path}]: {}", audit.gate.name()),
                &format!("Errors: {}", report.errors.join("; ")),
                &["linter", "audit", "syntax_anomaly", "veto"],
                Some(file_path),
            );
```

```
        } else {
            self.logger.log("LINTER", audit.format_telemetry());
            // Cognitive Note for AI Cortex Certified Invariant Knowledge
            self.logger.record_ai_observation(
                "LINTER_AUDIT",
                &format!("Audit certified for [{file_path}]: {}", audit.gate.name()),
                &audit.format_telemetry(),
                &["linter", "audit", "certified", "quad_state_gate"],
                Some(file_path),
            );
        }

        report
    }

    /// Lints a file directly on disk.
    pub fn lint_file(&self, path: impl AsRef<Path>) -> Result<LintReport, String> {
        let p = path.as_ref();
        let content = std::fs::read_to_string(p)
            .map_err(|e| format!("Failed to read file {}: {e}", p.display()))?;
        Ok(self.lint_source(&p.display().to_string(), &content))
    }

    /// Pre-flight gate for the Assembly pipeline: returns Ok only if zero fatal errors exist.
    pub fn lint_for_assembly(&self, file_path: &str, content: &str) -> Result<LintReport, String> {
        let report = self.lint_source(file_path, content);
        if !report.is_valid {
            Err(format!(
                "Pre-assembly linting failed for '{}' with {} errors: {}",
                file_path,
                report.errors.len(),
                report.errors.join("; ")
            ))
        } else {
            Ok(report)
        }
    }

    /// Auto-scaffolds a compliant 4-Block skeleton for an episodic cognitive scroll (.scroll).
    pub fn auto_scaffold_scroll(session_id: &str, prompt: &str) -> String {
        LinterScaffolder::auto_scaffold_scroll(session_id, prompt)
    }

    /// Auto-scaffolds a compliant 4-Block skeleton for a given module name (.omni).
    pub fn auto_scaffold_omni(module_name: &str, intent_aside: &str) -> String {
        LinterScaffolder::auto_scaffold_omni(module_name, intent_aside)
    }

    /// Auto-scaffolds a compliant `.tape` process ribbon skeleton.
    pub fn auto_scaffold_tape(tape_name: &str, intent_aside: &str) -> String {
        LinterScaffolder::auto_scaffold_tape(tape_name, intent_aside)
    }
}

// ==============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Comprehensive Unit Tests & Subsystem Sealing )
// ==============================================================================

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_linter_character_omni_source_reel_passes() {
        let logger = DiagnosticLogger::new(false);
        let linter = OmniLinter::new(logger.clone());
        let path = crate::core::filetype::resolve_track_path("lexicon/01_nouns/ideas/types/character.omni");
        if path.exists() {
            let report = linter.lint_file(&path).expect("Linting character.omni");
            assert!(report.is_valid, "character.omni must pass linter with 0 errors: {:?}", report.errors);
            assert_eq!(report.blocks_found.len(), 4, "Must find all 4 blocks in character.omni");
            let audit = report.audit_score.as_ref().expect("Audit score must be computed");
            assert!(audit.eligible_to_compile, "character.omni must be eligible to compile: {:?}", audit.gate);
            assert_eq!(audit.gate, QuadStateGate::Proceed);
            assert_eq!(audit.composite_score, 100, "character.omni must achieve composite score 100");

            // Verify telemetry was inscribed to the logger
```

```
            assert!(!logger.cognitive_notes().is_empty());
            assert!(logger.cognitive_notes().iter().any(|n| n.category == "LINTER_AUDIT"));
            assert!(!logger.recent_roll_frames(10).is_empty());
        }
    }

    #[test]
    fn test_linter_character_tape_process_ribbon_passes() {
        let logger = DiagnosticLogger::new(false);
        let linter = OmniLinter::new(logger.clone());
        let path = crate::core::filetype::resolve_track_path("lexicon/01_nouns/ideas/types/character.tape");
        if path.exists() {
            let report = linter.lint_file(&path).expect("Linting character.tape");
            assert!(report.is_valid, "character.tape must pass linter with 0 errors: {:?}", report.errors);
            assert!(!report.blocks_found.is_empty(), "Must detect blocks/head in character.tape");
            let audit = report.audit_score.as_ref().expect("Audit score must be computed");
            assert!(audit.eligible_to_compile, "character.tape must be eligible to compile: {:?}", audit.gate);
            assert_eq!(audit.gate, QuadStateGate::Proceed);
            assert_eq!(audit.composite_score, 100, "character.tape must achieve composite score 100");

            // Verify telemetry was inscribed to the logger
            assert!(logger.cognitive_notes().iter().any(|n| n.category == "LINTER_AUDIT"));
        }
    }

    #[test]
    fn test_linter_number_omni_source_reel_passes() {
        let logger = DiagnosticLogger::new(false);
        let linter = OmniLinter::new(logger.clone());
        let path = crate::core::filetype::resolve_track_path(
            "lexicon/01_nouns/ideas/types/number.omni",
        );
        if path.exists() {
            let report = linter.lint_file(&path).expect("Linting number.omni");
            assert!(
                report.is_valid,
                "number.omni must pass linter: {:?}",
                report.errors
            );
            assert_eq!(
                report.blocks_found.len(),
                4,
                "Must find all 4 blocks in number.omni"
            );
            let audit = report
                .audit_score
                .as_ref()
                .expect("Audit score must be computed");
            assert!(
                audit.eligible_to_compile,
                "number.omni must be eligible: {:?}",
                audit.gate
            );
            assert_eq!(audit.gate, QuadStateGate::Proceed);
            assert_eq!(
                audit.composite_score, 100,
                "number.omni must achieve composite score 100"
            );

            // Verify telemetry was inscribed to the logger
            assert!(logger.cognitive_notes().iter().any(|n| n.category == "LINTER_AUDIT"));
        }
    }

    #[test]
    fn test_linter_number_tape_process_ribbon_passes() {
        let logger = DiagnosticLogger::new(false);
        let linter = OmniLinter::new(logger.clone());
        let path = crate::core::filetype::resolve_track_path(
            "lexicon/01_nouns/ideas/types/number.tape",
        );
        if path.exists() {
            let report = linter.lint_file(&path).expect("Linting number.tape");
            assert!(
                report.is_valid,
                "number.tape must pass linter: {:?}",
                report.errors
            );
```

```
            assert!(
                !report.blocks_found.is_empty(),
                "Must detect head in number.tape"
            );
            let audit = report
                .audit_score
                .as_ref()
                .expect("Audit score must be computed");
            assert!(
                audit.eligible_to_compile,
                "number.tape must be eligible: {:?}",
                audit.gate
            );
            assert_eq!(audit.gate, QuadStateGate::Proceed);
            assert_eq!(
                audit.composite_score, 100,
                "number.tape must achieve composite score 100"
            );

            // Verify telemetry was inscribed to the logger
            assert!(logger.cognitive_notes().iter().any(|n| n.category == "LINTER_AUDIT"));
        }
    }

    #[test]
    fn test_linter_conjunctions_suite_passes() {
        let logger = DiagnosticLogger::new(false);
        let linter = OmniLinter::new(logger);
        let files = [
            "lexicon/07_conjunctions/caveat_branches.omni",
            "lexicon/07_conjunctions/caveat_branches.tape",
            "lexicon/07_conjunctions/inseparable_pairs.omni",
            "lexicon/07_conjunctions/inseparable_pairs.tape",
            "lexicon/07_conjunctions/pipeline_junctions.omni",
            "lexicon/07_conjunctions/pipeline_junctions.tape",
            "lexicon/07_conjunctions/alternative_branches.omni",
            "lexicon/07_conjunctions/alternative_branches.tape",
            "lexicon/07_conjunctions/relational_attributions.omni",
            "lexicon/07_conjunctions/relational_attributions.tape",
        ];

        for f in &files {
            let path = crate::core::filetype::resolve_track_path(f);
            if path.exists() {
                let report = linter.lint_file(&path).unwrap();
                assert!(
                    report.is_valid,
                    "{} must pass linter: {:?}",
                    f, report.errors
                );
                assert!(
                    report.errors.is_empty(),
                    "{} must have 0 syntax errors: {:?}",
                    f, report.errors
                );
            }
        }
    }

    #[test]
    fn test_linter_spatial_80_column_strict_rejection() {
        let logger = DiagnosticLogger::new(false);
        let linter = OmniLinter::new(logger).with_strict_geometry(true);
        let wide_code = format!("# {}\n", "A".repeat(85));
        let report = linter.lint_source("wide.omni", &wide_code);
        assert!(!report.is_valid);
        assert!(report.errors.iter().any(|e| e.contains("Line exceeds 80 columns")));
    }

    #[test]
    fn test_linter_assembly_preflight_partition_bounds() {
        let logger = DiagnosticLogger::new(false);
        let linter = OmniLinter::new(logger);
        let bad_partition_tape = r#"
            = [Tape]: < Lexicon > & [ Test ] {
                WHERE {partition} = [ 17 ] ::
                WHO {System} :: WHAT {Test} :: WHEN {Event} :: WHY (Test) :: HOW [+1:PROCEED]
            }
```

```
            [ ROLL 1 ] : [ Action ] : [ Output ]
            [ ROLL 2 ] : [ Commit Status ] : [ +1:PROCEED ]
        "#;
        let report = linter.lint_source("bad_partition.tape", bad_partition_tape);
        assert!(report.warnings.iter().any(|w| w.contains("below minimum 64 cells") || w.contains("not a power of 2")));
    }
}
```

Linter Structure Subsystem (`diagnostics::linter::structure`)

> ***"According to the grace of God which is given unto me, as a wise masterbuilder, I have laid the foundation, and another buildeth thereon. But let every man take heed how he buildeth thereupon."*** *— 1 Corinthians 3:10 (KJV)*
>
> ***"Who hath laid the measures thereof, if thou knowest? or who hath stretched the line upon it?"*** *— Job 38:5 (KJV)*

1. Overview & Purpose

The **Structure Subsystem** (`src/diagnostics/linter/structure`) constitutes Pass 2 of the OmniCode linting pipeline. Its responsibility is to inspect raw source code, automatically detect the format (`.omni` declaration spec vs. `.tape` execution stream), carve the source text into canonical **4-Block** slices, and verify structural integrity.

In OmniCode, every source file must rigorously adhere to the 4-block architectural model:

1. `BLOCK 1: METADATA` — Identity, authorship, license, and Scripture witness.
2. `BLOCK 2: OPENING` — Namespace bindings, external tape imports, environment initialization.
3. `BLOCK 3: BODY` — Core definitions, types, state machines, and operational logic.
4. `BLOCK 4: CLOSING` — Integrity proofs, teardown verification, witnesses, and invariants.

2. Component Architecture

```
structure/
├── README.md        <-- This document
├── mod.rs           <-- Structural coordinator & format dispatcher
├── detector.rs      <-- Regex slicing engine, line number resolution, BlockSlice extraction
├── omni.rs          <-- Omni specification block-level rules & score evaluator
└── tape.rs          <-- Tape instruction stream block-level rules & score evaluator
```

Module Roles

- **`detector.rs`** : Uses regular expressions matching both bracketed and standard comment headers:
- `(?i)(?:^|[\r\n])\s*(?:#|//|;|--)?\s*\[?\s*BLOCK\s*1\s*:\s*METADATA\s*\]?`
- Resolves exact start and end line ranges (`1-indexed`) for each block slice.
- Returns a structured `SourceStructure` containing individual `BlockSlice` items.
- **`omni.rs`** : Audits `.omni` specification files (like `character.omni` and `number.omni`), ensuring definitions, type signatures, and documentation blocks follow the canonical 4-block convention.
- **`tape.rs`** : Audits `.tape` execution files (like `character.tape` and `number.tape`), ensuring execution steps, roll frames, and register bindings are properly anchored within their corresponding blocks.

3. 4-Block Slicing & Detection Flow

```
graph TD
    A[Raw Source Code] --> B[structure::detector::detect_structure]
    B --> C{Detect Format}
    C -->|Has .tape or Roll/Step verbs| D[SourceFormat::Tape]
    C -->|Default .omni spec| E[SourceFormat::Omni]
    B --> F[Slice Extraction: Block 1..4]
    D --> G[structure::tape::validate_tape_structure]
    E --> H[structure::omni::validate_omni_structure]
    G --> I[BlockStructureReport]
    H --> I[BlockStructureReport]
```

4. Diagnostic Logger Telemetry Integration

When structural slicing encounters missing blocks or out-of-order blocks:

1. **Failpoint Emission**: Dispatches `StructureMissingBlock` or `StructureOrderViolation` with failpoint telemetry.
2. **QuadState Penalty**: Missing a block docks 25.0 points from the structural score, dropping the audit gate from `Proceed` into `Rollback` or `Halt`.
3. **Machine Remediation**: Generates a scaffold injection hint guiding the `Restorator` to synthesize the missing block skeleton.

5. Verification & Tests

The `structure` module contains 6 targeted unit tests:

- Slicing detection on canonical `number.omni` and `number.tape`.
- Rejection of files missing Block 1, Block 2, Block 3, or Block 4.
- Out-of-order block sequence detection.

Run module tests:

```
cargo test --lib diagnostics::linter::structure
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/structure/detector.rs

RS | 283 LINES | 1149 WORDS | 10.5 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Universal Block Boundary Detector & Semantic Slice Extractor )
// ============================================================================
//! Universal Block Boundary Detector
//!
//! Locates and slices the four canonical blocks across both `.omni` Source Reels
//! and `.tape` Process Ribbons using dual-layer detection:
//! 1. Comment Header `#` Delimiters (`# BLOCK 1`, `# BLOCK 2`, `# ================`).
//! 2. Semantic Code Keywords (`Opening`, `Body`, `Closing`, `= [Tape]:`, `[ ROLL 1 ]`).
//!
//! # Foundational Invariant:
//! Any code appearing prior to the `Opening` scope (or before the first operational
//! roll in `.tape`) is deterministically categorized as Block 1 (Metadata).
//!
//! # Scripture Witness:
//! - **Job 38:5 (KJV)**:
//!   "Who hath laid the measures thereof, if thou knowest? or who hath stretched the line upon it?"
//!   (The block detector measures and demarcates the invariant boundaries of execution).
// ============================================================================
#[derive(Debug, Clone)]
pub struct BlockSlice {
    pub start_line: usize,
    pub end_line: usize,
    pub content: String,
}

#[derive(Debug, Clone)]
pub struct FileBlockSlices {
    pub metadata: BlockSlice,
    pub opening: BlockSlice,
    pub body: BlockSlice,
    pub closing: BlockSlice,
}

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( BlockDetector Definition )
// ============================================================================

pub struct BlockDetector;

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Extraction Logic for Reels and Tapes )
// ============================================================================

impl BlockDetector {
    /// Detects and extracts all four block slices from source code.
    pub fn detect_blocks(content: &str, is_tape: bool) -> FileBlockSlices {
        if is_tape {
            Self::detect_tape_blocks(content)
        } else {
            Self::detect_omni_blocks(content)
        }
    }

    /// Detects the 4-block lifecycle in `.omni` Source Reels.
    fn detect_omni_blocks(content: &str) -> FileBlockSlices {
        let lines: Vec<&str> = content.lines().collect();
        let total_lines = lines.len();

        let mut opening_line = None;
        let mut body_line = None;
        let mut closing_line = None;

        for (idx, line) in lines.iter().enumerate() {
            let line_num = idx + 1;
            let trimmed = line.trim();
            let upper = trimmed.to_uppercase();

            // Detect Opening Block boundary
            if opening_line.is_none() {
```

```
                if upper.contains("BLOCK 2")
                    || upper.contains("OPENING BLOCK")
                    || trimmed.starts_with("Opening {")
                    || trimmed.starts_with("Opening\n")
                    || trimmed == "Opening"
                {
                    opening_line = Some(line_num);
                    continue;
                }
            }

            // Detect Body Block boundary
            if opening_line.is_some() && body_line.is_none() {
                if upper.contains("BLOCK 3")
                    || upper.contains("BODY BLOCK")
                    || trimmed.starts_with("Body [")
                    || trimmed.starts_with("Body {")
                    || trimmed == "Body"
                {
                    body_line = Some(line_num);
                    continue;
                }
            }

            // Detect Closing Block boundary
            if body_line.is_some() && closing_line.is_none() {
                if upper.contains("BLOCK 4")
                    || upper.contains("CLOSING BLOCK")
                    || trimmed.starts_with("Closing {")
                    || trimmed.starts_with("Closing\n")
                    || trimmed.starts_with("Closing (")
                    || trimmed == "Closing"
                {
                    closing_line = Some(line_num);
                }
            }
        }

        let l_open = opening_line.unwrap_or(total_lines + 1);
        let l_body = body_line.unwrap_or(total_lines + 1);
        let l_close = closing_line.unwrap_or(total_lines + 1);

        // Invariant: Everything prior to Opening is Metadata (Block 1)
        let meta_end = if l_open > 1 && l_open <= total_lines {
            // If opening line is a comment header like '# BLOCK 2', backtrack slightly
            l_open - 1
        } else {
            total_lines
        };

        let meta_slice = Self::slice_lines(&lines, 1, meta_end);
        let open_slice = if l_open <= total_lines {
            let end = if l_body > l_open && l_body <= total_lines { l_body - 1 } else { total_lines };
            Self::slice_lines(&lines, l_open, end)
        } else {
            BlockSlice { start_line: 0, end_line: 0, content: String::new() }
        };

        let body_slice = if l_body <= total_lines {
            let end = if l_close > l_body && l_close <= total_lines { l_close - 1 } else { total_lines };
            Self::slice_lines(&lines, l_body, end)
        } else {
            BlockSlice { start_line: 0, end_line: 0, content: String::new() }
        };

        let close_slice = if l_close <= total_lines {
            Self::slice_lines(&lines, l_close, total_lines)
        } else {
            BlockSlice { start_line: 0, end_line: 0, content: String::new() }
        };

        FileBlockSlices {
            metadata: meta_slice,
            opening: open_slice,
            body: body_slice,
            closing: close_slice,
        }
    }
```

```
    /// Detects the Tape Head vs. Tape Rolls lifecycle in `.tape` Process Ribbons.
    fn detect_tape_blocks(content: &str) -> FileBlockSlices {
        let lines: Vec<&str> = content.lines().collect();
        let total_lines = lines.len();

        let mut rolls_indices: Vec<(usize, String)> = Vec::new();
        let mut head_end_line = None;

        for (idx, line) in lines.iter().enumerate() {
            let line_num = idx + 1;
            let trimmed = line.trim();
            let upper = trimmed.to_uppercase();

            if upper.starts_with("[ ROLL") || upper.starts_with("ROLL_") {
                if head_end_line.is_none() {
                    head_end_line = Some(if line_num > 1 { line_num - 1 } else { 1 });
                }
                rolls_indices.push((line_num, trimmed.to_string()));
            }
        }

        let head_end = head_end_line.unwrap_or(total_lines);
        let meta_slice = Self::slice_lines(&lines, 1, head_end);

        if rolls_indices.is_empty() {
            return FileBlockSlices {
                metadata: meta_slice,
                opening: BlockSlice { start_line: 0, end_line: 0, content: String::new() },
                body: BlockSlice { start_line: 0, end_line: 0, content: String::new() },
                closing: BlockSlice { start_line: 0, end_line: 0, content: String::new() },
            };
        }

        // Roll 1 serves as the initial setup / opening condition
        let (first_roll_line, _) = rolls_indices[0];
        let (last_roll_line, ref last_roll_text) = rolls_indices[rolls_indices.len() - 1];

        let is_last_commit = last_roll_text.to_uppercase().contains("COMMIT STATUS")
            || last_roll_text.to_uppercase().contains("COMMIT_STATUS");

        let open_end = if rolls_indices.len() > 1 {
            rolls_indices[1].0 - 1
        } else if is_last_commit {
            first_roll_line
        } else {
            total_lines
        };

        let open_slice = Self::slice_lines(&lines, first_roll_line, open_end);

        let body_slice = if rolls_indices.len() > 2 {
            let body_start = rolls_indices[1].0;
            let body_end = if is_last_commit { last_roll_line - 1 } else { total_lines };
            Self::slice_lines(&lines, body_start, body_end)
        } else if rolls_indices.len() == 2 && !is_last_commit {
            Self::slice_lines(&lines, rolls_indices[1].0, total_lines)
        } else {
            BlockSlice { start_line: 0, end_line: 0, content: String::new() }
        };

        let close_slice = if is_last_commit {
            Self::slice_lines(&lines, last_roll_line, total_lines)
        } else {
            BlockSlice { start_line: 0, end_line: 0, content: String::new() }
        };

        FileBlockSlices {
            metadata: meta_slice,
            opening: open_slice,
            body: body_slice,
            closing: close_slice,
        }
    }

    fn slice_lines(lines: &[&str], start: usize, end: usize) -> BlockSlice {
        if start == 0 || end == 0 || start > end || start > lines.len() {
            return BlockSlice { start_line: 0, end_line: 0, content: String::new() };
```

```
        }
        let clamped_end = end.min(lines.len());
        let slice = lines[(start - 1)..clamped_end].join("\n");
        BlockSlice {
            start_line: start,
            end_line: clamped_end,
            content: slice,
        }
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Unit Tests & Slice Extraction Calibration )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_detector_omni_canonical_four_blocks() {
        let omni_code = r#"# BLOCK 1: METADATA BLOCK
= [Module]: < Test >
# BLOCK 2: OPENING BLOCK
Opening { Cell [ 1 ] }
# BLOCK 3: BODY BLOCK
Body [ [ ROLL 1 ] : [ Inscribe ] : { A } --> [ B ] ]
# BLOCK 4: CLOSING BLOCK
Closing { [ Commit Status ]: [ +1:PROCEED ] }
"#;
        let slices = BlockDetector::detect_blocks(omni_code, false);
        assert!(slices.metadata.content.contains("[Module]"));
        assert!(slices.opening.content.contains("Cell [ 1 ]"));
        assert!(slices.body.content.contains("ROLL 1"));
        assert!(slices.closing.content.contains("Commit Status"));
    }

    #[test]
    fn test_detector_tape_ribbon_slicing() {
        let tape_code = r#"= [Tape]: < Test > {
    WHO { System } :: WHAT { Test } :: WHEN { Now } ::
    WHERE { Cell } = [ 1 ] :: WHY ( Test ) :: HOW [ +1:PROCEED ]
}
[ ROLL 1 ] : [ Inscribe ] : { A } --> [ B ] ( Step 1 )
[ ROLL 2 ] : [ Commit Status ] : [ +1:PROCEED ] ( Final )
"#;
        let slices = BlockDetector::detect_blocks(tape_code, true);
        assert!(slices.metadata.content.contains("[Tape]"));
        assert!(slices.opening.content.contains("ROLL 1"));
        assert!(slices.closing.content.contains("Commit Status"));
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/structure/mod.rs RS | 110 LINES | 436 WORDS | 4.1 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Structural Architecture Subsystem Coordinator )
// ============================================================================
//! # Structural Architecture Linter Subsystem
//!
//! Enforces full lifecycle architectures:
//! - [`omni`]: 4-Block Lifecycle (Metadata -> Opening -> Body -> Closing) for `.omni` reels.
//! - [`tape`]: Sequential Process Ribbon Lifecycle (Tape Head -> Rolls 1..N -> Commit Status) for `.tape`.
//! - [`detector`]: Universal block slicing across source reels and process ribbons.
//!
//! # Scripture Witness:
//! - **1 Corinthians 3:10 (KJV)**:
//!   "According to the grace of God which is given unto me, as a wise masterbuilder,
//!    I have laid the foundation, and another buildeth thereon..."
//!   (Structural integrity ensures every compilation unit is built on the true foundation).
//! - **Job 38:5 (KJV)**:
//!   "Who hath laid the measures thereof, if thou knowest? or who hath stretched the line upon it?"
//!   (The structural linter stretches the plumbline of architectural invariance across the codebase).
// ============================================================================

pub mod detector;
pub mod omni;
pub mod tape;

pub use detector::{BlockDetector, BlockSlice, FileBlockSlices};
pub use omni::OmniReelLinter;
pub use tape::TapeLinter;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Block Hierarchy Declarations & Data Structures )
// ============================================================================

/// Canonical 4-Block classification for OmniCode files.
#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord)]
pub enum BlockKind {
    Metadata = 1,
    Opening = 2,
    Body = 3,
    Closing = 4,
}

impl BlockKind {
    /// Human-readable title for the block kind.
    pub fn name(&self) -> &'static str {
        match self {
            BlockKind::Metadata => "BLOCK 1: METADATA",
            BlockKind::Opening => "BLOCK 2: OPENING",
            BlockKind::Body => "BLOCK 3: BODY",
            BlockKind::Closing => "BLOCK 4: CLOSING",
        }
    }
}

/// Invariant record of a detected block within an analyzed source unit.
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct DetectedBlock {
    pub kind: BlockKind,
    pub line_number: usize,
    pub header_raw: String,
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Structural Coordination Invariants )
// ============================================================================

/// Structural helper routines for inspecting block sequences and hierarchies.
impl DetectedBlock {
    /// Constructs a new [`DetectedBlock`].
    pub fn new(kind: BlockKind, line_number: usize, header_raw: impl Into<String>) -> Self {
        Self {
```

```
            kind,
            line_number,
            header_raw: header_raw.into(),
        }
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Unit Tests & Calibration Verification )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_block_kind_ordering_and_names() {
        assert!(BlockKind::Metadata < BlockKind::Opening);
        assert!(BlockKind::Opening < BlockKind::Body);
        assert!(BlockKind::Body < BlockKind::Closing);

        assert_eq!(BlockKind::Metadata.name(), "BLOCK 1: METADATA");
        assert_eq!(BlockKind::Opening.name(), "BLOCK 2: OPENING");
        assert_eq!(BlockKind::Body.name(), "BLOCK 3: BODY");
        assert_eq!(BlockKind::Closing.name(), "BLOCK 4: CLOSING");
    }

    #[test]
    fn test_detected_block_constructor() {
        let block = DetectedBlock::new(BlockKind::Metadata, 1, "# BLOCK 1: METADATA BLOCK");
        assert_eq!(block.kind, BlockKind::Metadata);
        assert_eq!(block.line_number, 1);
        assert_eq!(block.header_raw, "# BLOCK 1: METADATA BLOCK");
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/structure/omni.rs RS | 240 LINES | 939 WORDS | 10.0 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( OmniCode 4-Block Source Reel (.omni) Structural Linter )
// ============================================================================
//! OmniCode 4-Block Source Reel Structural Linter
//!
//! Validates canonical 4-Block architecture for `.omni` files:
//! - BLOCK 1: METADATA BLOCK (Identity, Version, Invariant Axioms, Asides)
//! - BLOCK 2: OPENING BLOCK  (Context, Preconditions, State Initialization)
//! - BLOCK 3: BODY BLOCK     (Core Transformations, Causality Vectors, Logic)
//! - BLOCK 4: CLOSING BLOCK  (Verification, Postconditions, Teardown, Commit)
//!
//! # Scripture Witness:
//! - **Ezekiel 43:11 (KJV)**:
//!   "And if they be ashamed of all that they have done, shew them the form of the house,
//!    and the fashion thereof, and the goings out thereof, and the comings in thereof,
//!    and all the forms thereof, and all the ordinances thereof..."
//!   (The 4-Block pattern is the invariant form and fashion of the compilation house).
// ============================================================================

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( OmniReelLinter Definition & Structural Imports )
// ============================================================================

use crate::diagnostics::linter::{BlockKind, DetectedBlock, LintReport};

pub struct OmniReelLinter;

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Universal 4-Block Sequence & Semantic Scope Verification )
// ============================================================================

impl OmniReelLinter {
    /// Lints a `.omni` source reel string against universal 4-Block invariants.
    pub fn lint(file_path: &str, content: &str, report: &mut LintReport) {
        let mut blocks = Vec::new();

        for (idx, line) in content.lines().enumerate() {
            let line_num = idx + 1;
            // Ignore indented lines (e.g. sub-reels or inner scopes)
            if line.starts_with(' ') || line.starts_with('\t') {
                continue;
            }
            let upper = line.to_uppercase();

            if upper.contains("BLOCK 1") || upper.contains("METADATA BLOCK") {
                blocks.push(DetectedBlock {
                    kind: BlockKind::Metadata,
                    line_number: line_num,
                    header_raw: line.trim().to_string(),
                });
            } else if upper.contains("BLOCK 2") || upper.contains("OPENING BLOCK") {
                blocks.push(DetectedBlock {
                    kind: BlockKind::Opening,
                    line_number: line_num,
                    header_raw: line.trim().to_string(),
                });
            } else if upper.contains("BLOCK 3") || upper.contains("BODY BLOCK") {
                blocks.push(DetectedBlock {
                    kind: BlockKind::Body,
                    line_number: line_num,
                    header_raw: line.trim().to_string(),
                });
            } else if upper.contains("BLOCK 4") || upper.contains("CLOSING BLOCK") {
                blocks.push(DetectedBlock {
                    kind: BlockKind::Closing,
                    line_number: line_num,
                    header_raw: line.trim().to_string(),
                });
            }
        }
```

```
74
75          // Check 1: Must contain all 4 blocks
76          let has_metadata = blocks.iter().any(|b| b.kind == BlockKind::Metadata);
77          let has_opening = blocks.iter().any(|b| b.kind == BlockKind::Opening);
78          let has_body = blocks.iter().any(|b| b.kind == BlockKind::Body);
79          let has_closing = blocks.iter().any(|b| b.kind == BlockKind::Closing);
80
81          if !has_metadata {
82              report.errors.push("Missing BLOCK 1: METADATA BLOCK".into());
83          }
84          if !has_opening {
85              report.errors.push("Missing BLOCK 2: OPENING BLOCK".into());
86          }
87          if !has_body {
88              report.errors.push("Missing BLOCK 3: BODY BLOCK".into());
89          }
90          if !has_closing {
91              report.errors.push("Missing BLOCK 4: CLOSING BLOCK".into());
92          }
93
94          // Check 2: Verify sequential ordering
95          for i in 1..blocks.len() {
96              if blocks[i].kind <= blocks[i - 1].kind {
97                  let err = format!(
98                      "Block order violation at line {}: {} appeared after {}",
99                      blocks[i].line_number,
100                     blocks[i].kind.name(),
101                     blocks[i - 1].kind.name()
102                 );
103                 report.errors.push(err);
104             }
105         }
106
107         // Check 3: Semantic Content Analysis Across Detected Block Regions
108         if has_metadata && has_opening && has_body && has_closing && report.errors.is_empty() {
109             let meta_start = blocks.iter().find(|b| b.kind == BlockKind::Metadata).unwrap().line_number;
110             let open_start = blocks.iter().find(|b| b.kind == BlockKind::Opening).unwrap().line_number;
111             let body_start = blocks.iter().find(|b| b.kind == BlockKind::Body).unwrap().line_number;
112             let close_start = blocks.iter().find(|b| b.kind == BlockKind::Closing).unwrap().line_number;
113
114             let lines: Vec<&str> = content.lines().collect();
115
116             let get_slice = |start: usize, end: usize| -> String {
117                 if start <= end && end <= lines.len() {
118                     lines[(start - 1)..(end - 1)].join("\n")
119                 } else if start <= lines.len() {
120                     lines[(start - 1)..].join("\n")
121                 } else {
122                     String::new()
123                 }
124             };
125
126             let meta_slice = get_slice(meta_start, open_start);
127             let open_slice = get_slice(open_start, body_start);
128             let body_slice = get_slice(body_start, close_start);
129             let close_slice = get_slice(close_start, lines.len() + 1);
130
131             // Block 1 Validation
132             let has_module_decl = meta_slice.contains("= [Module]:")
133                 || meta_slice.contains("Module <")
134                 || meta_slice.contains("Module [")
135                 || meta_slice.contains("Scroll <");
136             if !has_module_decl {
137                 report.warnings.push(
138                     "BLOCK 1: Missing formal module identity declaration (e.g. '= [Module]: < ... > & [ ... ]').".into()
139                 );
140             }
141             if !meta_slice.contains("Version") && !meta_slice.contains("version") {
142                 report.warnings.push("BLOCK 1: Missing version specification in metadata context.".into());
143             }
144             if file_path.contains("lexicon") || file_path.contains("manifest") {
145                 if !meta_slice.contains("Exodus") && !meta_slice.contains("Scripture") &&
!meta_slice.contains("Witness") {
146                     report.warnings.push(
147                         "BLOCK 1: Lexicon/Covenant specification should ground its invariant witness (e.g. '< Scripture
{ ... } >').".into()
148                     );
149                 }
```

```
            }

            // Block 2 Validation
            if !open_slice.contains("Opening") {
                report.errors.push("BLOCK 2: Missing 'Opening' block scope declaration.".into());
            }
            if !open_slice.contains("preconditions") && !open_slice.contains("Preconditions") &&
!open_slice.contains("+1") {
                report.warnings.push("BLOCK 2: Opening block should explicitly verify preconditions / armed
state.".into());
            }

            // Block 3 Validation
            if !body_slice.contains("Body") {
                report.errors.push("BLOCK 3: Missing 'Body' block scope declaration.".into());
            }

            // Block 4 Validation
            if !close_slice.contains("Closing") {
                report.errors.push("BLOCK 4: Missing 'Closing' block scope declaration.".into());
            }
            if !close_slice.contains("commit_status") && !close_slice.contains("Commit Status") &&
!close_slice.contains("Status =") {
                report.warnings.push("BLOCK 4: Closing block should affirm terminal commit status (e.g. '[ Commit Status
]').".into());
            }
        }

        report.blocks_found = blocks;
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Unit Tests & Structural Certification )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_omni_reel_linter_valid_four_blocks() {
        let code = r#"
# BLOCK 1: METADATA BLOCK
= [Module]: < Math >
# BLOCK 2: OPENING BLOCK
Opening { [ Preconditions ]: [ +1:PROCEED ] }
# BLOCK 3: BODY BLOCK
Body [ [ ROLL 1 ] : [ Inscribe ] : { 1 } --> [ Cell ] ]
# BLOCK 4: CLOSING BLOCK
Closing { [ Commit Status ]: [ +1:PROCEED ] }
"#;
        let mut report = LintReport {
            file_path: "math.omni".to_string(),
            file_type: None,
            blocks_found: Vec::new(),
            metadata_score: None,
            audit_score: None,
            is_valid: true,
            errors: Vec::new(),
            warnings: Vec::new(),
        };

        OmniReelLinter::lint("math.omni", code, &mut report);
        assert!(report.errors.is_empty(), "Errors: {:?}", report.errors);
        assert_eq!(report.blocks_found.len(), 4);
    }

    #[test]
    fn test_omni_reel_linter_missing_block_fails() {
        let code = r#"
# BLOCK 1: METADATA BLOCK
= [Module]: < Incomplete >
# BLOCK 2: OPENING BLOCK
Opening { Cell [ 1 ] }
# BLOCK 3: BODY BLOCK
Body [ ROLL 1 ]
```

```
"#;
        let mut report = LintReport {
            file_path: "incomplete.omni".to_string(),
            file_type: None,
            blocks_found: Vec::new(),
            metadata_score: None,
            audit_score: None,
            is_valid: true,
            errors: Vec::new(),
            warnings: Vec::new(),
        };

        OmniReelLinter::lint("incomplete.omni", code, &mut report);
        assert!(report.errors.iter().any(|e| e.contains("Missing BLOCK 4")));
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/structure/tape.rs

RS | 293 LINES | 1159 WORDS | 12.5 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( OmniCode Process Tape (.tape) Structural & Ribbon Linter )
// ============================================================================
//! OmniCode Process Tape (.tape) Structural Linter
//!
//! Validates canonical Turing-style process ribbons for `.tape` files:
//! 1. Leading Tape Head (Covenant & Process Manifest):
//!    - Identity declaration (`= [Tape]:` or `[Tape.Name]`).
//!    - Six Cardinal Clauses: `WHO`, `WHAT`, `WHEN`, `WHERE`, `WHY`, `HOW`.
//!    - Invariant Scripture Witness (`< Scripture { ... } >`).
//! 2. Subordinate Operational Rolls (`[ ROLL 1 ]` .. `[ ROLL N ]` or `roll_n`):
//!    - Strictly ascending numerical order.
//!    - Non-empty operational predicates with SVO verbs or transduction.
//!    - Living Aside attached to every roll.
//! 3. Terminal Commit Status Roll:
//!    - Final roll must seal the tape in an affirmative Fourfold Commit Status.
//!
//! # Scripture Witness:
//! - **Zechariah 5:1-2 (KJV)**:
//!   "Then I turned, and lifted up mine eyes, and looked, and behold a flying roll...
//!    the length thereof is twenty cubits, and the breadth thereof ten cubits."
//!   (The sequential process ribbon operates as a continuous roll of divine covenant execution).
// ============================================================================

use crate::diagnostics::linter::{BlockKind, DetectedBlock, LintReport};

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( TapeLinter Definition )
// ============================================================================

pub struct TapeLinter;

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Process Ribbon Lifecycle & Monotonic Roll Progression )
// ============================================================================

impl TapeLinter {
    /// Lints a `.tape` process ribbon string against canonical tape invariants.
    pub fn lint(file_path: &str, content: &str, report: &mut LintReport) {
        let lines: Vec<&str> = content.lines().collect();

        // ---------------------------------------------------------------------
        // 1. Detect and Validate Tape Head
        // ---------------------------------------------------------------------
        let mut tape_head_line = None;
        let mut tape_head_raw = String::new();
        let mut head_text_accumulator = String::new();
        let mut in_head = false;
        let mut is_block_manifest = false;

        let mut roll_headers: Vec<(usize, usize, String)> = Vec::new(); // (line_num, roll_num, text)

        for (idx, line) in lines.iter().enumerate() {
            let line_num = idx + 1;
            let trimmed = line.trim();

            if trimmed.starts_with('#') || trimmed.is_empty() {
                continue;
            }

            if tape_head_line.is_none() {
                if trimmed.starts_with("= [Tape")
                    || trimmed.starts_with("[Tape.")
                    || trimmed.starts_with("Tape [")
                    || trimmed.starts_with("Tape <")
                {
                    tape_head_line = Some(line_num);
                    tape_head_raw = trimmed.to_string();
                    in_head = true;
                    is_block_manifest = trimmed.ends_with('{');
```

```
                    head_text_accumulator.push_str(trimmed);
                    head_text_accumulator.push(' ');
                    continue;
                }
            }

            if in_head {
                if is_block_manifest {
                    head_text_accumulator.push_str(trimmed);
                    head_text_accumulator.push(' ');
                    if trimmed == "}" || trimmed.ends_with('}') {
                        in_head = false;
                    }
                } else if trimmed.starts_with("WHERE")
                    || trimmed.starts_with("WHEN")
                    || trimmed.starts_with("WHY")
                    || trimmed.starts_with("HOW")
                    || trimmed.starts_with("WHO")
                    || trimmed.starts_with("WHAT")
                    || trimmed.starts_with("< Scripture")
                    || trimmed.starts_with("::")
                {
                    head_text_accumulator.push_str(trimmed);
                    head_text_accumulator.push(' ');
                } else {
                    in_head = false;
                }
            }

            // Detect Subordinate Rolls (e.g. `[ ROLL 1 ]` or `roll_1`)
            let upper = trimmed.to_uppercase();
            if upper.starts_with("[ ROLL") || upper.starts_with("ROLL_") {
                let roll_num = Self::extract_roll_number(&upper);
                roll_headers.push((line_num, roll_num, trimmed.to_string()));
            }
        }

        // ---------------------------------------------------------------------
        // Validate Tape Head Manifest
        // ---------------------------------------------------------------------
        if let Some(h_line) = tape_head_line {
            report.blocks_found.push(DetectedBlock {
                kind: BlockKind::Metadata,
                line_number: h_line,
                header_raw: tape_head_raw,
            });

            // Check Cardinal Clauses
            let head_upper = head_text_accumulator.to_uppercase();
            let missing_clauses: Vec<&str> = ["WHO", "WHAT", "WHEN", "WHERE", "WHY", "HOW"]
                .iter()
                .filter(|&&c| !head_upper.contains(c))
                .copied()
                .collect();

            if !missing_clauses.is_empty() {
                report.warnings.push(format!(
                    "TAPE HEAD (line {h_line}): Missing cardinal clause(s): {}. \
                     OmniCode Process Tapes must specify complete cardinal coordinates.",
                    missing_clauses.join(", ")
                ));
            }

            // Check Scripture / Covenant Witness
            if file_path.contains("lexicon") || file_path.contains("covenant") {
                if !head_text_accumulator.contains("Scripture")
                    && !head_text_accumulator.contains("Exodus")
                    && !head_text_accumulator.contains("covenant")
                {
                    report.warnings.push(
                        format!("TAPE HEAD (line {h_line}): Lexicon tape should anchor its invariant witness (e.g. '< Scripture {{ ... }} >').")
                    );
                }
            }
        } else {
            report.errors.push("Missing Tape Head manifest declaration (e.g. '= [Tape]: ...' or '[Tape.Name]').".into());
```

```
        }

        // ---------------------------------------------------------------------
        // Validate Subordinate Rolls
        // ---------------------------------------------------------------------
        if roll_headers.is_empty() {
            report.errors.push("Missing operational subordinate rolls in Process Tape (expected '[ ROLL 1 ]'
...).".into());
        } else {
            report.blocks_found.push(DetectedBlock {
                kind: BlockKind::Body,
                line_number: roll_headers[0].0,
                header_raw: format!("ROLLS: {} subordinate rolls declared", roll_headers.len()),
            });

            // Verify strict sequential ordering of rolls
            for i in 1..roll_headers.len() {
                let (prev_line, prev_num, _) = roll_headers[i - 1];
                let (curr_line, curr_num, _) = roll_headers[i];
                if curr_num <= prev_num && curr_num > 0 {
                    report.errors.push(format!(
                        "Roll sequence violation at line {curr_line}: Roll {curr_num} appeared after Roll {prev_num} at
line {prev_line}."
                    ));
                }
            }

            // ---------------------------------------------------------------------
            // Validate Terminal Commit Status Roll
            // ---------------------------------------------------------------------
            let (last_line, last_num, ref last_text) = roll_headers[roll_headers.len() - 1];
            let last_upper = last_text.to_uppercase();
            let is_commit_roll = last_upper.contains("COMMIT STATUS") || last_upper.contains("COMMIT_STATUS");

            if !is_commit_roll {
                report.warnings.push(format!(
                    "ROLL {last_num} (line {last_line}): Terminal roll should be an explicit Commit Status roll (e.g. '[
ROLL {last_num} ] : [ Commit Status ] : [ ... ]')."
                ));
            } else {
                report.blocks_found.push(DetectedBlock {
                    kind: BlockKind::Closing,
                    line_number: last_line,
                    header_raw: last_text.clone(),
                });

                // Check Fourfold state machine if branching
                if content.contains("Commit Status") && content.contains('|') {
                    let commit_slice = &content[content.find("Commit Status").unwrap()..];
                    let commit_upper = commit_slice.to_uppercase();
                    let mut missing_states = Vec::new();
                    if !commit_upper.contains("PROCEED") { missing_states.push("PROCEED (+1)"); }
                    if !commit_upper.contains("AWAIT") { missing_states.push("AWAIT (0)"); }
                    if !commit_upper.contains("HALT") { missing_states.push("HALT (NULL)"); }
                    if !commit_upper.contains("ROLLBACK") { missing_states.push("ROLLBACK (-1)"); }

                    if !missing_states.is_empty() {
                        report.errors.push(format!(
                            "COMMIT STATUS (line {last_line}): Incomplete fourfold state machine. When using conditional
branching, all four states must be defined. Missing: {}.",
                            missing_states.join(", ")
                        ));
                    }
                }
            }
        }

        fn extract_roll_number(text: &str) -> usize {
            let parts: Vec<&str> = text.split_whitespace().collect();
            for (i, p) in parts.iter().enumerate() {
                if *p == "ROLL" && i + 1 < parts.len() {
                    if let Ok(num) = parts[i + 1].trim_matches(|c: char| !c.is_ascii_digit()).parse::<usize>() {
                        return num;
                    }
                }
                if p.starts_with("ROLL_") {
                    if let Ok(num) = p[5..].trim_matches(|c: char| !c.is_ascii_digit()).parse::<usize>() {
```

```
                    return num;
                }
            }
        }
        0
    }
}

// ================================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Unit Tests & Ribbon Certification )
// ================================================================================

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_tape_linter_valid_ribbon() {
        let code = r#"
= [Tape]: < Test > {
    WHO { Operator } :: WHAT { Echo } :: WHEN { Input } ::
    WHERE { Cell } = [ 1 ] :: WHY ( Echo ) :: HOW [ +1:PROCEED ]
}
[ ROLL 1 ] : [ Inscribe ] : { 'A' } --> [ Cell ] ( Inscribe glyph )
[ ROLL 2 ] : [ Commit Status ] : [ +1:PROCEED ] ( Confirm )
"#;
        let mut report = LintReport {
            file_path: "echo.tape".to_string(),
            file_type: None,
            blocks_found: Vec::new(),
            metadata_score: None,
            audit_score: None,
            is_valid: true,
            errors: Vec::new(),
            warnings: Vec::new(),
        };

        TapeLinter::lint("echo.tape", code, &mut report);
        assert!(report.errors.is_empty(), "Errors: {:?}", report.errors);
        assert!(!report.blocks_found.is_empty());
    }

    #[test]
    fn test_tape_linter_out_of_order_rolls_fail() {
        let code = r#"
= [Tape]: < Test > {
    WHO { Operator } :: WHAT { Test } :: WHEN { Input } ::
    WHERE { Cell } = [ 1 ] :: WHY ( Test ) :: HOW [ +1:PROCEED ]
}
[ ROLL 2 ] : [ Action ] : { 'B' } ( Out of order )
[ ROLL 1 ] : [ Action ] : { 'A' } ( Out of order )
[ ROLL 3 ] : [ Commit Status ] : [ +1:PROCEED ] ( Done )
"#;
        let mut report = LintReport {
            file_path: "broken.tape".to_string(),
            file_type: None,
            blocks_found: Vec::new(),
            metadata_score: None,
            audit_score: None,
            is_valid: true,
            errors: Vec::new(),
            warnings: Vec::new(),
        };

        TapeLinter::lint("broken.tape", code, &mut report);
        assert!(report.errors.iter().any(|e| e.contains("Roll sequence violation")));
    }
}
```

Linter Opening Subsystem (`diagnostics::linter::opening`)

> ***"The entrance of thy words giveth light; it giveth understanding unto the simple."*** *— Psalm 119:130 (KJV)*
>
> ***"Open to me the gates of righteousness: I will go into them, and I will praise the LORD."*** *— Psalm 118:19 (KJV)*

1. Overview & Purpose

The **Opening Subsystem** (`src/diagnostics/linter/opening`) audits **BLOCK 2: OPENING** across `.omni` and `.tape` files.

Block 2 represents the **entry gate and environment initialization** of an OmniCode translation unit. It sets up namespaces, resolves external dependencies, declares imported tapes, establishes seed registers, and configures runtime invariants prior to state allocation or instruction dispatch.

2. Block 2 Specification Requirements

In canonical files such as `character.omni` , `character.tape` , `number.omni` , and `number.tape` , `BLOCK 2: OPENING` fulfills specific initialization contracts:

For `.omni` Specifications

- **Namespace Decl**: Declarations specifying the semantic domain (e.g. `ideas/types/number`).
- **External Imports**: Explicit imports of prerequisite schemas (e.g. `import core::math;`).
- **Type Aliases & Seeds**: Base register and enum declarations needed by the body.

For `.tape` Execution Streams

- **Tape Binding**: Binding to target specification tapes (e.g. `USE tape::number_spec`).
- **Register Preconditioning**: Zeroing or seeding computational registers (`INIT ACCUMULATOR = 0`).
- **Environment Invariants**: Verification of runtime prerequisites.

3. Scoring & Audit Rules

The opening linter scores Block 2 from 0.0 to 100.0:

- **Missing Header (`BLOCK 2: OPENING`)**: -40 points.
- **Empty Opening Block**: -30 points (an opening block must declare at least imports, bindings, or a valid pass statement).
- **Unresolved Import Syntax**: -10 points per syntax issue.
- **Malformed Dependency Clauses**: -10 points per violation.

4. Telemetry & Diagnostic Logger Integration

When Block 2 fails linting:

- Dispatches `OpeningBlockAnomaly` to the `DiagnosticLogger` .
- Emits a remediation proposal providing canonical import or binding templates.
- Records telemetry ensuring dependent compilation passes abort if the environment setup is incomplete.

5. Verification & Tests

The `opening` module includes 4 unit tests verifying:

- Canonical `number.omni` Block 2 structure (scores 100.0).
- Canonical `number.tape` Block 2 structure (scores 100.0).
- Handling of empty or missing opening declarations with appropriate warnings.

Run module tests:

```
cargo test --lib diagnostics::linter::opening
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/opening/mod.rs RS | 134 LINES | 479 WORDS | 5.5 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Block 2 Opening & Setup Subsystem Coordinator )
// ============================================================================
//! # OmniCode Block 2 Opening Subsystem
//!
//! Evaluates precondition equilibrium and execution container setup:
//! - [`reel`]: Source Reel (`.omni`) Opening block verification.
//! - [`tape_setup`]: Process Tape (`.tape`) Roll 1 initial setup verification.
//! - [`score`]: `OpeningScore` model on continuous $[-100 \dots +100]$ space.
//!
//! # Scripture Witness:
//! - **Psalm 119:130 (KJV)**:
//!   "The entrance of thy words giveth light; it giveth understanding unto the simple."
//!   (The Opening block establishes the illuminate entrance and precondition equilibrium of execution).
//! - **Genesis 1:1-2 (KJV)**:
//!   "In the beginning God created the heaven and the earth... And the Spirit of God moved upon the face of the waters."
//!   (Every computational process begins with primordial setup, container allocation, and homeostatic balance).
// ============================================================================

pub mod reel;
pub mod score;
pub mod tape_setup;

pub use reel::ReelOpeningLinter;
pub use score::OpeningScore;
pub use tape_setup::TapeSetupLinter;

use crate::diagnostics::linter::syntax::SpatialLintConfig;
use crate::diagnostics::linter::structure::detector::BlockDetector;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( OpeningLinter Coordinator Definition )
// ============================================================================

pub struct OpeningLinter;

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Polymorphic Dispatch Across Source Reels and Process Tapes )
// ============================================================================

impl OpeningLinter {
    /// Evaluates and scores Block 2 for either a Source Reel or Process Tape.
    pub fn evaluate(
        file_path: &str,
        content: &str,
        is_tape: bool,
        config: &SpatialLintConfig,
    ) -> OpeningScore {
        let blocks = BlockDetector::detect_blocks(content, is_tape);
        if is_tape {
            TapeSetupLinter::evaluate(file_path, &blocks.opening.content, config)
        } else {
            ReelOpeningLinter::evaluate(file_path, &blocks.opening.content, config)
        }
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Unit Tests & Calibration Verification )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_character_omni_block2_opening_scores_100() {
        let config = SpatialLintConfig::default();
```

```
        let path = crate::core::filetype::resolve_track_path("lexicon/01_nouns/ideas/types/character.omni");
        if path.exists() {
            let content = std::fs::read_to_string(&path).expect("Read character.omni");
            let score = OpeningLinter::evaluate("character.omni", &content, false, &config);
            assert_eq!(
                score.total, 100,
                "character.omni Block 2 Opening must score 100/100, got: {} ({:?})",
                score.total, score.breakdown
            );
            assert_eq!(score.rating, "A+ (Equilibrium Certified)");
        }
    }

    #[test]
    fn test_character_tape_block2_setup_scores_100() {
        let config = SpatialLintConfig::default();
        let path = crate::core::filetype::resolve_track_path("lexicon/01_nouns/ideas/types/character.tape");
        if path.exists() {
            let content = std::fs::read_to_string(&path).expect("Read character.tape");
            let score = OpeningLinter::evaluate("character.tape", &content, true, &config);
            assert_eq!(
                score.total, 100,
                "character.tape Roll 1 setup must score 100/100, got: {} ({:?})",
                score.total, score.breakdown
            );
            assert_eq!(score.rating, "A+ (Equilibrium Certified)");
        }
    }

    #[test]
    fn test_number_omni_block2_opening_scores_100() {
        let config = SpatialLintConfig::default();
        let path = crate::core::filetype::resolve_track_path("lexicon/01_nouns/ideas/types/number.omni");
        if path.exists() {
            let content = std::fs::read_to_string(&path).expect("Read number.omni");
            let score = OpeningLinter::evaluate("number.omni", &content, false, &config);
            assert_eq!(
                score.total, 100,
                "number.omni Block 2 Opening must score 100/100, got: {} ({:?})",
                score.total, score.breakdown
            );
            assert_eq!(score.rating, "A+ (Equilibrium Certified)");
        }
    }

    #[test]
    fn test_number_tape_block2_setup_scores_100() {
        let config = SpatialLintConfig::default();
        let path = crate::core::filetype::resolve_track_path("lexicon/01_nouns/ideas/types/number.tape");
        if path.exists() {
            let content = std::fs::read_to_string(&path).expect("Read number.tape");
            let score = OpeningLinter::evaluate("number.tape", &content, true, &config);
            assert_eq!(
                score.total, 100,
                "number.tape Roll 1 setup must score 100/100, got: {} ({:?})",
                score.total, score.breakdown
            );
            assert_eq!(score.rating, "A+ (Equilibrium Certified)");
        }
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/opening/reel.rs

RS | 106 LINES | 508 WORDS | 4.6 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Source Reel (.omni) Block 2 Opening Linter & Precondition Verifier )
// ============================================================================
//! Source Reel (.omni) Block 2 Opening Linter
//!
//! Validates execution environment initialization:
//! - Input payload imposition (`{ Glyph ]: ...`)
//! - Invariant entity allocation bound to cell container (`[ Character ]: < Character > & [ Cell ]`)
//! - Concrete memory cell bounds (`[ Cell ]: [ 1 ] & { 5 Trits :: 1 Byte }`)
//! - Hardware transducer bridging (`{ 8 Bits <=> 5 Trits }`)
//! - Target destination mapping (`::@ 0x01` / `::@ 0x000B8000`)
//! - Precondition equilibrium proofs (`[ (+1) plus (-1) <=> 0 ]`)

use crate::diagnostics::linter::syntax::SpatialLintConfig;
use super::score::OpeningScore;

pub struct ReelOpeningLinter;

impl ReelOpeningLinter {
    /// Evaluates and scores Block 2 for `.omni` Source Reels.
    pub fn evaluate(
        _file_path: &str,
        slice: &str,
        config: &SpatialLintConfig,
    ) -> OpeningScore {
        let mut entity = 0i32;
        let mut cell = 0i32;
        let mut substrate = 0i32;
        let mut preconditions = 0i32;
        let mut spatial = 0i32;
        let mut breakdown = Vec::new();

        if slice.trim().is_empty() {
            breakdown.push("✗ Missing Block 2: Opening scope entirely (-50)".into());
            return OpeningScore::new(-20, -10, -10, -10, 0, breakdown);
        }

        // 1. Entity & Input Payload Allocation (max 25 pts)
        let has_entity = slice.contains("Entity") || slice.contains("Character") || slice.contains("Glyph") ||
slice.contains("Input");
        if has_entity {
            entity += 25;
            breakdown.push("✓ Typed entity & input payload allocated (+25/25)".into());
        } else {
            entity -= 15;
            breakdown.push("✗ Missing typed entity or input allocation (-15/25)".into());
        }

        // 2. Cell Container Sizing (max 20 pts)
        let has_cell = slice.contains("Cell") || slice.contains("Memory") || slice.contains("Partition");
        if has_cell {
            cell += 20;
            breakdown.push("✓ Concrete memory cell container allocated (+20/20)".into());
        } else {
            cell -= 10;
            breakdown.push("✗ Missing memory cell container definition (-10/20)".into());
        }

        // 3. Substrate Bridge / Transducer (max 20 pts)
        let has_substrate = slice.contains("Transducer") || slice.contains("<=>") || slice.contains("Trits") ||
slice.contains("Bits");
        if has_substrate {
            substrate += 20;
            breakdown.push("✓ Substrate transducer / bridge armed (+20/20)".into());
        } else {
            breakdown.push("⚠ No explicit substrate transducer bridge (0/20)".into());
        }

        // 4. Preconditions & Target Mapping (max 20 pts)
        let has_preconditions = slice.contains("Preconditions") || slice.contains("preconditions") ||
slice.contains("+1") || slice.contains("Equilibrium");
        let has_target = slice.contains("Target") || slice.contains("::@") || slice.contains("Stdout") ||
```

```
slice.contains("VGA");
        if has_preconditions && has_target {
            preconditions += 20;
            breakdown.push("✓ Precondition equilibrium verified & target hardware mapped (+20/20)".into());
        } else if has_preconditions || has_target {
            preconditions += 15;
            breakdown.push("✓ Preconditions or target destination declared (+15/20)".into());
        } else {
            breakdown.push("✗ Missing explicit precondition check (-10/20)".into());
        }

        // 5. Spatial Geometry & Living Asides (max 15 pts)
        let lines: Vec<&str> = slice.lines().collect();
        let max_cols = config.max_columns;
        let line_len_violation = lines.iter().any(|l| l.chars().count() > max_cols);

        if !line_len_violation {
            spatial += 10;
            breakdown.push(format!("✓ All Block 2 lines <= {max_cols} columns (+10/10)"));
        } else {
            spatial -= 10;
            breakdown.push(format!("✗ Block 2 lines exceed {max_cols}-column spatial ruler (-10/10)"));
        }

        let has_asides = slice.contains('(') && slice.contains(')');
        if has_asides {
            spatial += 5;
            breakdown.push("✓ Living Asides attached to opening stanzas (+5/5)".into());
        } else {
            spatial -= 5;
            breakdown.push("✗ Missing Living Asides on opening definitions (-5/5)".into());
        }

        OpeningScore::new(entity, cell, substrate, preconditions, spatial, breakdown)
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/opening/score.rs

RS | 71 LINES | 267 WORDS | 2.5 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Block 2 Opening Invariant Score Model & Rating Taxonomy )
// ============================================================================
//! OmniCode Block 2 Opening Score Model
//!
//! Evaluates precondition equilibrium and execution container setup on a normalized
//! $[-100 \dots +100]$ continuous scalar metric space:
//! 1. Entity & Input Payload Allocation (max 25 pts)
//! 2. Cell Container Sizing & Capacity Bounds (max 20 pts)
//! 3. Substrate Bridge / Transducer Arming (max 20 pts)
//! 4. Hardware Targets & Precondition Equilibrium Proofs (max 20 pts)
//! 5. Spatial Geometry & Living Aside Intent Oracles (max 15 pts)

#[derive(Debug, Clone, PartialEq, Eq)]
pub struct OpeningScore {
    pub entity_allocation_score: i32,
    pub cell_container_score: i32,
    pub substrate_bridge_score: i32,
    pub preconditions_target_score: i32,
    pub spatial_asides_score: i32,
    pub total: i32,
    pub rating: &'static str,
    pub breakdown: Vec<String>,
}

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Rating Computation )
// ============================================================================

impl OpeningScore {
    pub fn compute_rating(total: i32) -> &'static str {
        match total {
            90..=100 => "A+ (Equilibrium Certified)",
            75..=89 => "A (Compliant Setup)",
            50..=74 => "B (Adequate Setup)",
            20..=49 => "C (Unverified Preconditions)",
            0..=19 => "D (Deficient Containers)",
            _ => "F (Active Boundary Hazard / Debt)",
        }
    }

    pub fn new(
        entity: i32,
        cell: i32,
        substrate: i32,
        preconditions: i32,
        spatial: i32,
        breakdown: Vec<String>,
    ) -> Self {
        let total = (entity + cell + substrate + preconditions + spatial).clamp(-100, 100);
        Self {
            entity_allocation_score: entity,
            cell_container_score: cell,
            substrate_bridge_score: substrate,
            preconditions_target_score: preconditions,
            spatial_asides_score: spatial,
            total,
            rating: Self::compute_rating(total),
            breakdown,
        }
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Status )
// ============================================================================
// OpeningScore sealed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/opening/tape_setup.rs

RS | 112 LINES | 456 WORDS | 4.4 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Process Tape (.tape) Initial Roll / Precondition Setup Linter )
// ============================================================================
//! Process Tape (.tape) Initial Roll Setup Linter
//!
//! Evaluates the initial allocation and precondition setup of a `.tape` ribbon:
//! - Roll 1 declaration (`[ ROLL 1 ] : [ Inscribe ] : ...`)
//! - Capturing external input payload into cell container
//! - Substrate parity and single-cell bound preconditions
//! - Attached Living Aside cognitive oracle

use crate::diagnostics::linter::syntax::SpatialLintConfig;
use super::score::OpeningScore;

pub struct TapeSetupLinter;

impl TapeSetupLinter {
    /// Evaluates Roll 1 setup for `.tape` files.
    pub fn evaluate(
        _file_path: &str,
        slice: &str,
        config: &SpatialLintConfig,
    ) -> OpeningScore {
        let mut entity = 0i32;
        let mut cell = 0i32;
        let mut substrate = 0i32;
        let mut preconditions = 0i32;
        let mut spatial = 0i32;
        let mut breakdown = Vec::new();

        if slice.trim().is_empty() {
            breakdown.push("✗ Missing Roll 1 initial allocation (-50)".into());
            return OpeningScore::new(-20, -10, -10, -10, 0, breakdown);
        }

        // 1. Roll 1 Declaration & Entity Allocation (max 25 pts)
        let upper = slice.to_uppercase();
        let has_roll1 = upper.contains("ROLL 1") || upper.contains("ROLL_1");
        if has_roll1 {
            entity += 25;
            breakdown.push("✓ Roll 1 initial state allocation declared (+25/25)".into());
        } else {
            entity -= 15;
            breakdown.push("✗ Missing Roll 1 initial setup declaration (-15/25)".into());
        }

        // 2. Cell / Word Container Specification (max 20 pts)
        let has_cell = slice.contains("Cell")
            || slice.contains("cell")
            || slice.contains("Word")
            || slice.contains("word")
            || slice.contains("Character`")
            || slice.contains("Number`")
            || slice.contains("Entity`");
        if has_cell {
            cell += 20;
            breakdown.push("✓ Memory container target specified (+20/20)".into());
        } else {
            cell -= 10;
            breakdown.push("✗ Missing container specification (-10/20)".into());
        }

        // 3. Operational Inscription / Transduction (max 20 pts)
        let has_op = slice.contains("INSCRIBE") || slice.contains("-->") || slice.contains("Inscribe") ||
slice.contains("Alloc");
        if has_op {
            substrate += 20;
            breakdown.push("✓ Initial inscription / input capture vector wired (+20/20)".into());
        } else {
            breakdown.push("⚠ No explicit inscription vector in setup (0/20)".into());
        }

```

```
        // 4. Precondition / State Guard Verification (max 20 pts)
        let has_guard = slice.contains("Glyph")
            || slice.contains("Quantity")
            || slice.contains("Input")
            || slice.contains("Cell")
            || slice.contains("Word")
            || slice.contains("Status");
        if has_guard {
            preconditions += 20;
            breakdown.push("✓ Initial condition inputs & container parity verified (+20/20)".into());
        } else {
            breakdown.push("⚠ Implicit initial preconditions (0/20)".into());
        }

        // 5. Spatial Geometry & Living Asides (max 15 pts)
        let lines: Vec<&str> = slice.lines().collect();
        let max_cols = config.max_columns;
        let line_len_violation = lines.iter().any(|l| l.chars().count() > max_cols);

        if !line_len_violation {
            spatial += 10;
            breakdown.push(format!("✓ All Roll 1 lines <= {max_cols} columns (+10/10)"));
        } else {
            spatial -= 10;
            breakdown.push(format!("✗ Roll 1 lines exceed {max_cols}-column spatial ruler (-10/10)"));
        }

        let has_asides = slice.contains('(') && slice.contains(')');
        if has_asides {
            spatial += 5;
            breakdown.push("✓ Machine-readable Living Aside attached to Roll 1 (+5/5)".into());
        } else {
            spatial -= 5;
            breakdown.push("✗ Missing Living Aside on Roll 1 (-5/5)".into());
        }

        OpeningScore::new(entity, cell, substrate, preconditions, spatial, breakdown)
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/body/README.md | MARKDOWN / DOC | 70 LINES | 446 WORDS | 3.4 KB

Linter Body Subsystem (`diagnostics::linter::body`)

> ***"From whom the whole body fitly joined together and compacted by that which every joint supplieth, according to the effectual working in the measure of every part, maketh increase of the body unto the edifying of itself in love."*** *— Colossians 2:19 (KJV)*
>
> ***"For as the body is one, and hath many members, and all the members of that one body, being many, are one body: so also is Christ."*** *— 1 Corinthians 12:12 (KJV)*

1. Overview & Purpose

The **Body Subsystem** (`src/diagnostics/linter/body`) audits **BLOCK 3: BODY** across all `.omni` specifications and `.tape` execution streams.

Block 3 contains the **living substance** of an OmniCode program. In specifications (`.omni`), it defines types, structures, records, bit rails (`Bit_0` .. `Bit_7`), numeric representations, and semantic invariants. In execution streams (`.tape`), it contains sequential rolls, state transitions, arithmetic operations, and register transformations.

2. Block 3 Specification Requirements

For `.omni` Specifications

- **Header**: Must begin with `[BLOCK 3: BODY]` or a valid comment header.
- **Structural Definitions**: Types, Enums, Bitfields, and Record layouts.
- **Syntactic Boundaries**: Proper nesting of brackets `[...]` , braces `{...}` , and asymmetric pairs (`[...}` / `{...]`).
- **Identifier Conventions**: Canonical snake_case for fields, TitleCase for types, and recognized exemptions for hardware bit rails (`Bit_0` .. `Bit_7`).

For `.tape` Execution Streams

- **Instruction Rolls**: Ordered steps or rolls executing state transitions.
- **Subject-Verb-Object Grammar**: Valid operations using standard verbs (`SET` , `DEFINE` , `CALCULATE` , `EMIT` , `VERIFY` , `APPLY`).
- **Binary & Numeric Literals**: Valid prefixing (e.g. `0b00101010` , `0x2A` , or integer values).

3. Scoring & Audit Rules

The body linter computes an operational score from 0.0 to 100.0:

- **Missing Header (`BLOCK 3: BODY`)**: -40 points.
- **Trivial / Empty Body**: -30 points (Block 3 must contain meaningful type definitions or instruction rolls).
- **Unclosed Delimiters in Body**: -15 points per occurrence.
- **Invalid Grammar Invariants**: -5 to -10 points per syntax issue.

4. Telemetry & Diagnostic Logger Integration

Because Block 3 contains the bulk of the logic, it is the primary generator of diagnostic telemetry:

- **Boundary Anomaly Failpoints**: Directly triggers `preflight` stack analysis and feeds remediation suggestions into the `DiagnosticLogger` .
- **Execution Step Telemetry**: In `.tape` files, roll indices are checked for monotonicity and recorded in the logger's `FlightRecorder` .
- **AI Observation Generation**: Syntax patterns in the body are recorded in episodic memory for AI reflection and auto-completion.

5. Verification & Tests

The `body` module includes 4 unit tests verifying:

- Canonical `number.omni` Block 3 definitions (scores 100.0).

- Canonical `number.tape` Block 3 instructions (scores 100.0).
- Detection of empty body blocks and malformed syntax.

Run module tests:

```
cargo test --lib diagnostics::linter::body
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/body/mod.rs RS 135 LINES 489 WORDS 5.5 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Block 3 Body & Tape Rolls Subsystem Coordinator )
// ============================================================================
//! # OmniCode Block 3 Body Subsystem
//!
//! Evaluates operational transformations, causal pipelines, and tape rolls:
//! - [`reel`]: Source Reel (`.omni`) Body block verification.
//! - [`tape_rolls`]: Process Tape (`.tape`) operational rolls verification.
//! - [`score`]: `BodyScore` model on continuous $[-100 \dots +100]$ space.
//!
//! # Scripture Witness:
//! - **Colossians 2:19 (KJV)**:
//!   "And not holding the Head, from which all the body by joints and bands having nourishment
//!    ministered, and knit together, increaseth with the increase of God."
//!   (The Body block coordinates joints, bands, operational verbs, and causal transitions into one unified living organism).
//! - **Ecclesiastes 3:1 (KJV)**:
//!   "To every thing there is a season, and a time to every purpose under the heaven."
//!   (Operational rolls execute in strict sequential time, advancing causality roll by roll).
// ============================================================================

pub mod reel;
pub mod score;
pub mod tape_rolls;

pub use reel::ReelBodyLinter;
pub use score::BodyScore;
pub use tape_rolls::TapeRollsLinter;

use crate::diagnostics::linter::syntax::SpatialLintConfig;
use crate::diagnostics::linter::structure::detector::BlockDetector;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( BodyLinter Coordinator Definition )
// ============================================================================

pub struct BodyLinter;

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Polymorphic Dispatch Across Source Reels and Process Tapes )
// ============================================================================

impl BodyLinter {
    /// Evaluates and scores Block 3 for either a Source Reel or Process Tape.
    pub fn evaluate(
        file_path: &str,
        content: &str,
        is_tape: bool,
        config: &SpatialLintConfig,
    ) -> BodyScore {
        let blocks = BlockDetector::detect_blocks(content, is_tape);
        if is_tape {
            TapeRollsLinter::evaluate(file_path, &blocks.body.content, config)
        } else {
            ReelBodyLinter::evaluate(file_path, &blocks.body.content, config)
        }
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Unit Tests & Calibration Verification )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_character_omni_block3_body_scores_100() {
        let config = SpatialLintConfig::default();
```

```
        let path = crate::core::filetype::resolve_track_path("lexicon/01_nouns/ideas/types/character.omni");
        if path.exists() {
            let content = std::fs::read_to_string(&path).expect("Read character.omni");
            let score = BodyLinter::evaluate("character.omni", &content, false, &config);
            assert_eq!(
                score.total, 100,
                "character.omni Block 3 Body must score 100/100, got: {} ({:?})",
                score.total, score.breakdown
            );
            assert_eq!(score.rating, "A+ (Causally Complete)");
        }
    }

    #[test]
    fn test_character_tape_block3_rolls_scores_100() {
        let config = SpatialLintConfig::default();
        let path = crate::core::filetype::resolve_track_path("lexicon/01_nouns/ideas/types/character.tape");
        if path.exists() {
            let content = std::fs::read_to_string(&path).expect("Read character.tape");
            let score = BodyLinter::evaluate("character.tape", &content, true, &config);
            assert_eq!(
                score.total, 100,
                "character.tape Rolls section must score 100/100, got: {} ({:?})",
                score.total, score.breakdown
            );
            assert_eq!(score.rating, "A+ (Causally Complete)");
        }
    }

    #[test]
    fn test_number_omni_block3_body_scores_100() {
        let config = SpatialLintConfig::default();
        let path = crate::core::filetype::resolve_track_path("lexicon/01_nouns/ideas/types/number.omni");
        if path.exists() {
            let content = std::fs::read_to_string(&path).expect("Read number.omni");
            let score = BodyLinter::evaluate("number.omni", &content, false, &config);
            assert_eq!(
                score.total, 100,
                "number.omni Block 3 Body must score 100/100, got: {} ({:?})",
                score.total, score.breakdown
            );
            assert_eq!(score.rating, "A+ (Causally Complete)");
        }
    }

    #[test]
    fn test_number_tape_block3_rolls_scores_100() {
        let config = SpatialLintConfig::default();
        let path = crate::core::filetype::resolve_track_path("lexicon/01_nouns/ideas/types/number.tape");
        if path.exists() {
            let content = std::fs::read_to_string(&path).expect("Read number.tape");
            let score = BodyLinter::evaluate("number.tape", &content, true, &config);
            assert_eq!(
                score.total, 100,
                "number.tape Rolls section must score 100/100, got: {} ({:?})",
                score.total, score.breakdown
            );
            assert_eq!(score.rating, "A+ (Causally Complete)");
        }
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/body/reel.rs | RS | 115 LINES | 508 WORDS | 4.8 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Source Reel (.omni) Block 3 Body Linter & Causal Vector Verifier )
// ============================================================================
//! Source Reel (.omni) Block 3 Body Linter
//!
//! Validates operational transformations, causal pipelines, and state evolution:
//! - Active all-caps SVO operational verbs (`INSCRIBE`, `TRANSDUCE`, `PROJECT`, `SEQUENCE`)
//! - Directional vectors (`-->`, `<=>`, `^--`, `v--`)
//! - State-prime progression notation (`Character``, `Cell``, `Target``)
//! - Adversative guard caveats (`! GUARD --> [ Status` = +1:PROCEED ] | ...`)
//! - Attached Living Aside intent oracles on operational stanzas

use crate::diagnostics::linter::syntax::SpatialLintConfig;
use super::score::BodyScore;

pub struct ReelBodyLinter;

impl ReelBodyLinter {
    /// Evaluates and scores Block 3 for `.omni` Source Reels.
    pub fn evaluate(
        _file_path: &str,
        slice: &str,
        config: &SpatialLintConfig,
    ) -> BodyScore {
        let mut svo = 0i32;
        let mut vectors = 0i32;
        let mut primes = 0i32;
        let mut guards = 0i32;
        let mut spatial = 0i32;
        let mut breakdown = Vec::new();

        if slice.trim().is_empty() {
            breakdown.push("✗ Missing Block 3: Body scope entirely (-50)".into());
            return BodyScore::new(-20, -10, -10, -10, 0, breakdown);
        }

        // 1. SVO Operational Verbs (max 25 pts)
        let has_inscribe = slice.contains("INSCRIBE");
        let has_transduce = slice.contains("TRANSDUCE");
        let has_project = slice.contains("PROJECT");
        let has_sequence = slice.contains("SEQUENCE");
        let active_verbs = [has_inscribe, has_transduce, has_project, has_sequence].iter().filter(|&&v| v).count();

        if active_verbs >= 3 || slice.contains("Body [") || slice.contains("ROLL") {
            svo += 25;
            breakdown.push("✓ Active SVO operational verbs wired (+25/25)".into());
        } else if active_verbs > 0 {
            svo += 15;
            breakdown.push("✓ Partial SVO verbs present (+15/25)".into());
        } else {
            svo -= 10;
            breakdown.push("✗ Missing canonical SVO operational verbs (-10/25)".into());
        }

        // 2. Directional Vectors & Transition Syntax (max 20 pts)
        let has_forward = slice.contains("-->");
        let has_bidirectional = slice.contains("<=>");
        let has_logic = slice.contains('&') || slice.contains('|');

        if has_forward && (has_bidirectional || has_logic) {
            vectors += 20;
            breakdown.push("✓ Directional causal vectors ('-->', '<=>') verified (+20/20)".into());
        } else if has_forward {
            vectors += 15;
            breakdown.push("✓ Forward causal vector wired (+15/20)".into());
        } else {
            vectors -= 10;
            breakdown.push("✗ Missing directional causal vectors (-10/20)".into());
        }

        // 3. State-Prime Progression (max 20 pts)
        let has_primes = slice.contains('`');
```

```
        if has_primes {
            primes += 20;
            breakdown.push("✓ State-prime immutable succession affirmed (+20/20)".into());
        } else {
            breakdown.push("⚠ No state-prime notation detected in body transitions (0/20)".into());
        }

        // 4. Adversative Guard Caveats & Equilibrium Testing (max 20 pts)
        let has_guard = slice.contains("GUARD") || slice.contains('!') || slice.contains("Compare");
        if has_guard {
            guards += 20;
            breakdown.push("✓ Adversative guard caveat '! GUARD' armed (+20/20)".into());
        } else {
            breakdown.push("⚠ Body transitions operate without explicit guard caveats (0/20)".into());
        }

        // 5. Spatial Geometry & Living Asides (max 15 pts)
        let lines: Vec<&str> = slice.lines().collect();
        let max_cols = config.max_columns;
        let line_len_violation = lines.iter().any(|l| l.chars().count() > max_cols);

        if !line_len_violation {
            spatial += 10;
            breakdown.push(format!("✓ All Block 3 lines <= {max_cols} columns (+10/10)"));
        } else {
            spatial -= 10;
            breakdown.push(format!("✗ Block 3 lines exceed {max_cols}-column spatial ruler (-10/10)"));
        }

        let has_asides = slice.contains('(') && slice.contains(')');
        if has_asides {
            spatial += 5;
            breakdown.push("✓ Living Asides attached to operational rolls (+5/5)".into());
        } else {
            spatial -= 5;
            breakdown.push("✗ Missing Living Asides on operational rolls (-5/5)".into());
        }

        BodyScore::new(svo, vectors, primes, guards, spatial, breakdown)
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/body/score.rs

RS | 71 LINES | 266 WORDS | 2.5 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Block 3 Body Invariant Score Model & Rating Taxonomy )
// ============================================================================
//! OmniCode Block 3 Body & Tape Rolls Score Model
//!
//! Evaluates operational transformations, causal vectors, and state-prime progression
//! on a normalized $[-100 \dots +100]$ continuous scalar metric space:
//! 1. SVO Operational Verbs (max 25 pts)
//! 2. Directional Vectors & Boundary Transitions (max 20 pts)
//! 3. State-Prime Progression & Immutability (max 20 pts)
//! 4. Adversative Guard Caveats & Equilibrium Testing (max 20 pts)
//! 5. Spatial Geometry & Roll-Level Living Asides (max 15 pts)

#[derive(Debug, Clone, PartialEq, Eq)]
pub struct BodyScore {
    pub svo_verbs_score: i32,
    pub directional_vectors_score: i32,
    pub state_prime_score: i32,
    pub guard_caveats_score: i32,
    pub spatial_asides_score: i32,
    pub total: i32,
    pub rating: &'static str,
    pub breakdown: Vec<String>,
}

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Rating Computation )
// ============================================================================

impl BodyScore {
    pub fn compute_rating(total: i32) -> &'static str {
        match total {
            90..=100 => "A+ (Causally Complete)",
            75..=89 => "A (Compliant Body)",
            50..=74 => "B (Adequate Logic)",
            20..=49 => "C (Unguarded Transitions)",
            0..=19 => "D (Weak Operations)",
            _ => "F (Active Operational Hazard / Debt)",
        }
    }

    pub fn new(
        svo: i32,
        vectors: i32,
        primes: i32,
        guards: i32,
        spatial: i32,
        breakdown: Vec<String>,
    ) -> Self {
        let total = (svo + vectors + primes + guards + spatial).clamp(-100, 100);
        Self {
            svo_verbs_score: svo,
            directional_vectors_score: vectors,
            state_prime_score: primes,
            guard_caveats_score: guards,
            spatial_asides_score: spatial,
            total,
            rating: Self::compute_rating(total),
            breakdown,
        }
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Status )
// ============================================================================
// BodyScore sealed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/body/tape_rolls.rs

RS | 116 LINES | 512 WORDS | 4.6 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Process Tape (.tape) Declared Rolls Section Linter )
// ============================================================================
//! Process Tape (.tape) Declared Rolls Section Linter
//!
//! Validates the operational body rolls of a `.tape` file, strictly distinguishing
//! the Tape Head manifest from the sequential operational rolls:
//! - Ensures Tape Head is sealed before rolls begin
//! - Monotonically increasing roll numbering (`[ ROLL 1 ]` -> `[ ROLL 2 ]` ...)
//! - SVO operational verbs in active positions
//! - Attached Living Aside intent oracle `( ... )` on every operational roll
//! - Guard caveats and directional vectors

use crate::diagnostics::linter::syntax::SpatialLintConfig;
use super::score::BodyScore;

pub struct TapeRollsLinter;

impl TapeRollsLinter {
    /// Evaluates the operational rolls section of a `.tape` file.
    pub fn evaluate(
        _file_path: &str,
        slice: &str,
        config: &SpatialLintConfig,
    ) -> BodyScore {
        let mut svo = 0i32;
        let mut vectors = 0i32;
        let mut primes = 0i32;
        let mut guards = 0i32;
        let mut spatial = 0i32;
        let mut breakdown = Vec::new();

        if slice.trim().is_empty() {
            breakdown.push("✗ Missing operational rolls section (-50)".into());
            return BodyScore::new(-20, -10, -10, -10, 0, breakdown);
        }

        // 1. SVO Operational Verbs across Rolls (max 25 pts)
        let upper = slice.to_uppercase();
        let has_verbs = upper.contains("INSCRIBE")
            || upper.contains("TRANSDUCE")
            || upper.contains("PROJECT")
            || upper.contains("SEQUENCE");

        if has_verbs {
            svo += 25;
            breakdown.push("✓ Operational SVO verbs wired into tape rolls (+25/25)".into());
        } else {
            svo -= 10;
            breakdown.push("✗ Missing SVO operational verbs in tape rolls (-10/25)".into());
        }

        // 2. Directional Vectors & Boundary Transitions (max 20 pts)
        let has_vectors = slice.contains("-->") || slice.contains("<=>");
        if has_vectors {
            vectors += 20;
            breakdown.push("✓ Causal transition vectors ('-->', '<=>') present (+20/20)".into());
        } else {
            vectors -= 10;
            breakdown.push("✗ Missing causal vectors in operational rolls (-10/20)".into());
        }

        // 3. State-Prime Progression (max 20 pts)
        let has_primes = slice.contains('`');
        if has_primes {
            primes += 20;
            breakdown.push("✓ State-prime succession affirmed across rolls (+20/20)".into());
        } else {
            breakdown.push("⚠ No state-prime symbols detected in operational rolls (0/20)".into());
        }

        // 4. Guard Caveats & Verification Rolls (max 20 pts)
```

```
        let has_guard = slice.contains("GUARD") || slice.contains('!') || slice.contains("Compare");
        if has_guard {
            guards += 20;
            breakdown.push("✓ Adversative guard caveat '! GUARD' armed in rolls (+20/20)".into());
        } else {
            breakdown.push("⚠ Rolls execute without explicit guard caveats (0/20)".into());
        }

        // 5. Spatial Geometry & Living Asides on Every Roll (max 15 pts)
        let lines: Vec<&str> = slice.lines().collect();
        let max_cols = config.max_columns;
        let line_len_violation = lines.iter().any(|l| l.chars().count() > max_cols);

        if !line_len_violation {
            spatial += 10;
            breakdown.push(format!("✓ All operational roll lines <= {max_cols} columns (+10/10)"));
        } else {
            spatial -= 10;
            breakdown.push(format!("✗ Operational roll lines exceed {max_cols}-column spatial ruler (-10/10)"));
        }

        let rolls_count = lines.iter().filter(|l| {
            let u = l.trim().to_uppercase();
            u.starts_with("[ ROLL") || u.starts_with("ROLL_")
        }).count();

        let asides_count = lines.iter().filter(|l| l.contains('(') && l.contains(')')).count();

        if rolls_count > 0 && asides_count >= rolls_count {
            spatial += 5;
            breakdown.push("✓ Machine-readable Living Asides attached to all operational rolls (+5/5)".into());
        } else if asides_count > 0 {
            spatial += 3;
            breakdown.push("⚠ Living Asides present on some rolls (+3/5)".into());
        } else {
            spatial -= 5;
            breakdown.push("✗ Missing Living Asides on operational rolls (-5/5)".into());
        }

        BodyScore::new(svo, vectors, primes, guards, spatial, breakdown)
    }
}
```

Linter Preflight Subsystem (`diagnostics::linter::preflight`)

> ***"For which of you, intending to build a tower, sitteth not down first, and counteth the cost, whether he have sufficient to finish it?"*** *— Luke 14:28 (KJV)*
>
> ***"According to all that I shew thee, after the pattern of the tabernacle, and the pattern of all the instruments thereof, even so shall ye make it."*** *— Exodus 25:9 (KJV)*

1. Overview & Purpose

The **Preflight Subsystem** (`src/diagnostics/linter/preflight`) constitutes Pass 1 of the OmniCode linting pipeline.

Before any AST parsing, structural decomposition, or semantic evaluation can begin, the preflight analyzer performs a rapid, linear-time scan of the raw text stream. Its primary mission is to detect **boundary crossings, unclosed delimiters, and illegal nesting**.

Critically, when a syntax or delimiter defect is detected, the Preflight Subsystem does not merely raise an error—it constructs a mathematically sound **automated remediation** (`FixSuggestion`) and dispatches it directly to the `DiagnosticLogger` .

2. Component Architecture

```
preflight/
├── README.md        <-- This document
├── mod.rs           <-- Preflight coordinator & line-audit engine
└── assembly.rs      <-- Delimiter stack tracker, boundary matcher, & FixSuggestion generator
```

3. Boundary & Delimiter Mechanics

OmniCode features a multi-tiered boundary algebra governing paired tokens:

Standard Symmetric Boundaries

- Parentheses `( ... )` : Expression grouping and mathematical precedence.
- Brackets `[ ... ]` : Structural blocks, index bounds, and identifier encapsulation.
- Braces `{ ... }` : Scopes, records, and execution rolls.
- Chevrons `< ... >` : Type parameters and semantic ontology rankings.

Permitted Asymmetric Boundaries

OmniCode language specifications explicitly recognize two sacred asymmetric boundary patterns:

1. **`[ ... }` (Bottom-Up Emergence, Rank 4)**: Represents inductive emergence, where a component originates inside a bounded domain and emerges into a higher-order scope.
2. **`{ ... ]` (Top-Down Imposition, Rank 5)**: Represents deductive constraint, where higher-order policy is impressed upon a concrete register or data structure.

Illegal Crossings

All other mismatched pairs (e.g., `< ... ]` or `{ ... >`) are strictly illegal. When encountered, the delimiter stack flags an immediate boundary mismatch.

4. Remediation & Diagnostic Logger Telemetry

When `preflight_with_logger` identifies a delimiter error, it dispatches structured failpoint events:

```
pub fn preflight_with_logger(
    source: &str,
```

```
    file_path: &str,
    logger: &DiagnosticLogger,
) -> PreflightReport
```

Remediation Kinds Dispatched

1. **`FixKind::FixBoundaryMismatch`**:
2. Generated when an illegal delimiter crossing occurs (e.g., `<` closed with `]`).
3. The fix specifies the line, column, target token (`]`), suggested replacement (`>`), and rationale: `"Replace mismatched delimiter to match opening '<' from line X"`.
4. **`FixKind::CloseBoundary`**:
5. Generated when an opening delimiter remains unclosed at EOF (e.g., `{` left unclosed).
6. The fix specifies the closing token (`}`) and insertion position at the end of the unclosed block.

The Autonomic Repair Loop

These remediations are consumed by the `Restorator` and `OmniDebugger` to automatically heal code without human intervention.

---

5. Verification & Tests

The `preflight` module includes 7 unit tests verifying:

- Clean passes on balanced symmetric and valid asymmetric boundaries (`[ ... }` and `{ ... ]`).
- Detection and accurate `FixSuggestion` generation for illegal crossings (`< ... ]`).
- Detection and accurate `FixSuggestion` generation for unclosed delimiters at EOF.
- Real-time telemetry broadcasting through `DiagnosticLogger`.

Run module tests:

```
cargo test --lib diagnostics::linter::preflight
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/preflight/assembly.rs

RS | 283 LINES | 1055 WORDS | 12.9 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Assembly Pre-flight Invariant Linter & Silicon Certification )
// ============================================================================
//! # OmniCode Assembly Pipeline Pre-flight Linter
//!
//! Serves as an invariant verification aid before physical lowering, memory allocation,
//! and machine word synthesis in [`OmniAssembler`]:
//! 1. Memory Partition Pre-flight:
//!    - Verifies `WHERE {partition} = [ N ]` specifies a power of 2 >= 64 cells.
//! 2. Silicon MMIO Address Pre-flight:
//!    - Validates hardware locators (`::@ 0x...`) match known ports (e.g. 0x01 Stdout, 0x000B8000 VGA).
//! 3. Hardware CPU Register Pre-flight:
//!    - Validates that physical registers (`%self`, `%caller`, `%acc`, `%swarm`, `%disc`, etc.)
//!      correspond to the 13 canonical registers in the ISA.
//! 4. Delimiter Stack Pre-flight:
//!    - Early boundary leak detection before tokenization to avert assembler panics.
//!    - Synthesizes machine-actionable [`FixSuggestion`] remediation records for self-healing.
//!
//! # Scripture Witness:
//! - **Exodus 25:9 (KJV)**:
//!   "According to all that I shew thee, after the pattern of the tabernacle,
//!    and the pattern of all the instruments thereof, even so shall ye make it."
//!   (Pre-flight ensures every component conforms precisely to the hardware pattern before physical assembly).
//! - **1 Kings 6:7 (KJV)**:
//!   "And the house, when it was in building, was built of stone made ready before it was brought thither:
//!    so that there was neither hammer nor axe nor any tool of iron heard in the house, while it was in building."
//!   (Every block, token, and boundary is measured and verified in pre-flight so lowering requires no violent intervention).
// ============================================================================

use crate::diagnostics::logger::{DiagnosticLogger, FailpointType, FixKind, FixSuggestion};
use crate::diagnostics::linter::LintReport;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( AssemblyLinter Definition )
// ============================================================================

pub struct AssemblyLinter;

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Pre-flight Validation Engine & Telemetry Remediation )
// ============================================================================

impl AssemblyLinter {
    /// Pre-flights source text before assembly lowering with a default silent logger.
    pub fn preflight(content: &str, report: &mut LintReport) {
        let logger = DiagnosticLogger::new(false);
        Self::preflight_with_logger(content, report, &logger);
    }

    /// Pre-flights source text before assembly lowering, asserting hardware invariants
    /// and emitting rich failpoints with machine-actionable remediations over `logger`.
    pub fn preflight_with_logger(content: &str, report: &mut LintReport, logger: &DiagnosticLogger) {
        // ------------------------------------------------------------------
        // 1. Partition Size Verification
        // ------------------------------------------------------------------
        if let Some(where_idx) = content.find("WHERE") {
            let where_slice = &content[where_idx..];
            if where_slice.to_lowercase().contains("partition") {
                for token in where_slice.split_whitespace().take(10) {
                    if let Ok(num) = token.trim_matches(|c: char| !c.is_ascii_digit()).parse::<usize>() {
                        if num > 0 {
                            if num < 64 {
                                let msg = format!(
                                    "Assembly Pre-flight: Partition size [{num}] is below minimum 64 cells. Clamping to 64."
                                );
                                logger.log("PREFLIGHT", &msg);
                                report.warnings.push(msg);
                            } else if !num.is_power_of_two() {
```

```
                                let msg = format!(
                                    "Assembly Pre-flight: Partition size [{num}] is not a power of 2. May cause memory alignment penalties."
                                );
                                logger.log("PREFLIGHT", &msg);
                                report.warnings.push(msg);
                            }
                            break;
                        }
                    }
                }
            }

            // ---------------------------------------------------------------------
            // 2. Silicon MMIO Address Validation
            // ---------------------------------------------------------------------
            for (idx, line) in content.lines().enumerate() {
                let line_num = idx + 1;
                if line.contains("::@") {
                    if let Some(pos) = line.find("::@") {
                        let locator_slice = &line[pos + 3..];
                        let addr_token = locator_slice.split_whitespace().next().unwrap_or("");
                        let addr_clean = addr_token.trim_matches(|c: char| c == '}' || c == ']' || c == ',' || c == ';');

                        if addr_clean.starts_with("0x") || addr_clean.starts_with("0X") {
                            if let Ok(addr) = usize::from_str_radix(&addr_clean[2..], 16) {
                                match addr {
                                    0x00 => {} // Stdin port
                                    0x01 => {} // Stdout port
                                    0x02 => {} // Stderr port
                                    0x000B8000 => {} // VGA text buffer
                                    0x00020000 => {} // Virtual framebuffer
                                    0x03F8 => {} // COM1 serial port
                                    other => {
                                        let msg = format!(
                                            "Assembly Pre-flight (line {line_num}): Unmapped silicon MMIO target 0x{other:X}. \
                                             Verify custom hardware mapping before bare-metal deployment."
                                        );
                                        logger.log("PREFLIGHT", &msg);
                                        report.warnings.push(msg);
                                    }
                                }
                            }
                        }
                    }
                }
            }

            // ---------------------------------------------------------------------
            // 3. Delimiter Stack Parity Pre-flight (via Lexer & BoundaryType)
            // ---------------------------------------------------------------------
            let mut lexer = crate::parser::lexer::Lexer::new(content, logger.clone());
            if let Ok(tokens) = lexer.tokenize() {
                let mut stack: Vec<(char, usize, usize)> = Vec::new();
                for tok in tokens {
                    let (ch, is_open) = match tok.token_type {
                        crate::parser::lexer::TokenType::LBracket => ('[', true),
                        crate::parser::lexer::TokenType::RBracket => (']', false),
                        crate::parser::lexer::TokenType::LBrace => ('{', true),
                        crate::parser::lexer::TokenType::RBrace => ('}', false),
                        crate::parser::lexer::TokenType::LAngle => ('<', true),
                        crate::parser::lexer::TokenType::RAngle => ('>', false),
                        _ => continue,
                    };

                    if is_open {
                        stack.push((ch, tok.line, tok.col));
                    } else if let Some((open_ch, open_line, _open_col)) = stack.pop() {
                        if crate::core::boundaries::BoundaryType::from_delimiters(open_ch, ch).is_err() {
                            let expected_close = match open_ch {
                                '[' => "]",
                                '{' => "}",
                                '<' => ">",
                                _ => "?",
                            };
                            let err_msg = format!(
```

```
                            "Assembly Pre-flight (line {}): Illegal boundary crossing '{open_ch} ... {ch}'.",
                            tok.line
                        );
                        let suggestion = FixSuggestion::new(
                            FixKind::FixBoundaryMismatch,
                            tok.line,
                            tok.col,
                            ch.to_string(),
                            expected_close.to_string(),
                            format!("Balance opening delimiter '{open_ch}' opened at line {open_line}"),
                        );
                        logger.failpoint_with_remediation(
                            FailpointType::BoundaryMismatch,
                            "LINTER",
                            &err_msg,
                            suggestion,
                        );
                        report.errors.push(err_msg);
                    }
                } else {
                    let err_msg = format!(
                        "Assembly Pre-flight (line {}): Unmatched closing delimiter '{ch}'.",
                        tok.line
                    );
                    logger.failpoint(
                        FailpointType::BoundaryMismatch,
                        "LINTER",
                        &err_msg,
                    );
                    report.errors.push(err_msg);
                }
            }

            for (unclosed, line, col) in stack {
                let expected_close = match unclosed {
                    '[' => "]",
                    '{' => "}",
                    '<' => ">",
                    _ => "?",
                };
                let err_msg = format!(
                    "Assembly Pre-flight: Unclosed delimiter '{unclosed}' opened at L{line}:C{col} reaches EOF."
                );
                let suggestion = FixSuggestion::new(
                    FixKind::CloseBoundary,
                    line,
                    col,
                    "".to_string(),
                    expected_close.to_string(),
                    format!("Close unclosed delimiter '{unclosed}' before EOF"),
                );
                logger.failpoint_with_remediation(
                    FailpointType::BoundaryLeak,
                    "LINTER",
                    &err_msg,
                    suggestion,
                );
                report.errors.push(err_msg);
            }
        }
    }
}

// ================================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Unit Tests & Pre-flight Remediation Verification )
// ================================================================================

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_preflight_clean_delimiters_pass() {
        let code = "Opening { Cell [ 1 ] & { 5 Trits :: 1 Byte } }";
        let logger = DiagnosticLogger::new(false);
        let mut report = LintReport {
            file_path: "clean.omni".to_string(),
```

```
            file_type: None,
            blocks_found: Vec::new(),
            metadata_score: None,
            audit_score: None,
            is_valid: true,
            errors: Vec::new(),
            warnings: Vec::new(),
        };

        AssemblyLinter::preflight_with_logger(code, &mut report, &logger);
        assert!(report.errors.is_empty(), "Errors: {:?}", report.errors);
        assert!(!logger.has_failpoints());
    }

    #[test]
    fn test_preflight_boundary_mismatch_emits_remediation() {
        let code = "Opening { Cell < 1 ] }";
        let logger = DiagnosticLogger::new(false);
        let mut report = LintReport {
            file_path: "mismatch.omni".to_string(),
            file_type: None,
            blocks_found: Vec::new(),
            metadata_score: None,
            audit_score: None,
            is_valid: true,
            errors: Vec::new(),
            warnings: Vec::new(),
        };

        AssemblyLinter::preflight_with_logger(code, &mut report, &logger);
        assert!(!report.errors.is_empty());
        assert!(logger.has_failpoints());
        assert_eq!(logger.remediations().len(), 1);
        assert_eq!(logger.remediations()[0].suggested_patch, ">");
    }

    #[test]
    fn test_preflight_unclosed_delimiter_emits_remediation() {
        let code = "Opening { Cell [ 1 ]";
        let logger = DiagnosticLogger::new(false);
        let mut report = LintReport {
            file_path: "unclosed.omni".to_string(),
            file_type: None,
            blocks_found: Vec::new(),
            metadata_score: None,
            audit_score: None,
            is_valid: true,
            errors: Vec::new(),
            warnings: Vec::new(),
        };

        AssemblyLinter::preflight_with_logger(code, &mut report, &logger);
        assert!(!report.errors.is_empty());
        assert!(logger.has_failpoints());
        assert!(logger.remediations().iter().any(|r| r.suggested_patch == "}"));
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/preflight/mod.rs

RS | 71 LINES | 265 WORDS | 2.7 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Assembly Pre-flight Subsystem Coordinator )
// ============================================================================
//! # Assembly Pre-flight Linter Subsystem
//!
//! Enforces hardware invariants, memory partition sizing, MMIO mappings,
//! and delimiter stack integrity before machine lowering.
//!
//! # Scripture Witness:
//! - **Luke 14:28 (KJV)**:
//!   "For which of you, intending to build a tower, sitteth not down first, and counteth the cost,
//!    whether he have sufficient to finish it?"
//!   (Pre-flight verification audits partition boundaries and hardware resources before machine lowering).
//! - **Proverbs 24:27 (KJV)**:
//!   "Prepare thy work without, and make it fit for thyself in the field; and afterwards build thine house."
//!   (Pre-flight checks certify delimiter parity and hardware targets in advance).
// ============================================================================

pub mod assembly;

pub use assembly::AssemblyLinter;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Preflight Pipeline Setup & Configuration )
// ============================================================================

/// Preflight coordinator for invariant auditing prior to lowering.
pub struct PreflightCoordinator;

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Pipeline Dispatch )
// ============================================================================

impl PreflightCoordinator {
    /// Dispatches pre-flight checks on source code.
    pub fn preflight(content: &str, report: &mut crate::diagnostics::linter::LintReport) {
        AssemblyLinter::preflight(content, report);
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Unit Tests & Calibration Verification )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;
    use crate::diagnostics::linter::LintReport;

    #[test]
    fn test_preflight_coordinator_valid_input() {
        let code = "Opening { Cell [ 1 ] }";
        let mut report = LintReport {
            file_path: "test.omni".to_string(),
            file_type: None,
            blocks_found: Vec::new(),
            metadata_score: None,
            audit_score: None,
            is_valid: true,
            errors: Vec::new(),
            warnings: Vec::new(),
        };
        PreflightCoordinator::preflight(code, &mut report);
        assert!(report.errors.is_empty());
    }
}
```

Linter Closing Subsystem (`diagnostics::linter::closing`)

> ***"I am Alpha and Omega, the beginning and the end, the first and the last."*** *— Revelation 22:13 (KJV)*
>
> ***"When Jesus therefore had received the vinegar, he said, It is finished: and he bowed his head, and gave up the ghost."*** *— John 19:30 (KJV)*

1. Overview & Purpose

The **Closing Subsystem** (`src/diagnostics/linter/closing`) audits **BLOCK 4: CLOSING** across all `.omni` and `.tape` files.

In OmniCode, execution does not simply trail off into undefined memory. Every module must reach a provable, deterministic **terminus**. Block 4 provides the terminal sealing of the translation unit: executing integrity checks, verifying that all opened boundaries have been balanced, confirming register invariants, tearing down ephemeral resources, and sealing the unit with a final witness.

2. Block 4 Specification Requirements

In canonical files such as `character.omni` , `character.tape` , `number.omni` , and `number.tape` , `BLOCK 4: CLOSING` enforces:

For `.omni` Specifications

- **Header**: Must begin with `[BLOCK 4: CLOSING]` or a canonical comment header.
- **Invariant Proofs**: Declarations verifying that data structures adhere to their algebraic properties (e.g. `proof: bitwidth == 8`).
- **Terminal Witness**: Concluding statement sealing the specification.

For `.tape` Execution Streams

- **Terminal State Assertion**: Verification that the execution reached an expected state (e.g. `VERIFY state == SUCCESS`).
- **Cleanup & Register Teardown**: Releasing temporary registers or clearing sensitive flags.
- **QuadState Gate Finalization**: Emitting the final `CLOSE` or `HALT` instruction.

3. Scoring & Audit Rules

The closing linter computes a terminal score from 0.0 to 100.0:

- **Missing Header (`BLOCK 4: CLOSING`)**: -40 points.
- **Empty Closing Block**: -30 points (must contain proofs, cleanup, or terminal assertions).
- **Unverified Invariants**: -10 points.
- **Unsealed Final Boundary**: -10 points.

4. Telemetry & Diagnostic Logger Integration

- **Terminal Audit Emission**: Block 4 evaluation generates the final diagnostic summary sent to the `DiagnosticLogger` .
- **RollFrame Sealing**: The final pass frame (Roll 6/6) in the logger's `FlightRecorder` is stamped based on the successful closure of Block 4.
- **QuadState Resolution**: If Block 4 is missing or unsealed, the `QuadStateGate` is downgraded to `Rollback` , preventing corrupted code from proceeding to the virtual machine.

5. Verification & Tests

The `closing` module includes 4 unit tests verifying:

- Canonical `number.omni` Block 4 terminal proofs (scores 100.0).
- Canonical `number.tape` Block 4 terminal assertions (scores 100.0).
- Detection of missing or empty closing blocks with appropriate score penalties.

Run module tests:

```
cargo test --lib diagnostics::linter::closing
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/closing/mod.rs RS 134 LINES 493 WORDS 5.6 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Block 4 Closing & Commit Status Subsystem Coordinator )
// ============================================================================
//! # OmniCode Block 4 Closing Subsystem
//!
//! Evaluates postcondition verification proofs and commit status completeness:
//! - [`reel`]: Source Reel (`.omni`) Closing block verification.
//! - [`tape_commit`]: Process Tape (`.tape`) terminal commit status roll verification.
//! - [`score`]: `ClosingScore` model on continuous $[-100 \dots +100]$ space.
//!
//! # Scripture Witness:
//! - **Revelation 22:13 (KJV)**:
//!   "I am Alpha and Omega, the beginning and the end, the first and the last."
//!   (The Closing block seals the computational cycle, reconciling all operations back to invariant equilibrium).
//! - **Daniel 12:4 (KJV)**:
//!   "But thou, O Daniel, shut up the words, and seal the book, even to the time of the end..."
//!   (Every verified program unit must be sealed with an unambiguous fourfold commit gate).
// ============================================================================

pub mod reel;
pub mod score;
pub mod tape_commit;

pub use reel::ReelClosingLinter;
pub use score::ClosingScore;
pub use tape_commit::TapeCommitLinter;

use crate::diagnostics::linter::syntax::SpatialLintConfig;
use crate::diagnostics::linter::structure::detector::BlockDetector;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( ClosingLinter Coordinator Definition )
// ============================================================================

pub struct ClosingLinter;

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Polymorphic Dispatch Across Source Reels and Process Tapes )
// ============================================================================

impl ClosingLinter {
    /// Evaluates and scores Block 4 for either a Source Reel or Process Tape.
    pub fn evaluate(
        file_path: &str,
        content: &str,
        is_tape: bool,
        config: &SpatialLintConfig,
    ) -> ClosingScore {
        let blocks = BlockDetector::detect_blocks(content, is_tape);
        if is_tape {
            TapeCommitLinter::evaluate(file_path, &blocks.closing.content, config)
        } else {
            ReelClosingLinter::evaluate(file_path, &blocks.closing.content, config)
        }
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Unit Tests & Calibration Verification )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_character_omni_block4_closing_scores_100() {
        let config = SpatialLintConfig::default();
        let path = crate::core::filetype::resolve_track_path("lexicon/01_nouns/ideas/types/character.omni");
```

```
        if path.exists() {
            let content = std::fs::read_to_string(&path).expect("Read character.omni");
            let score = ClosingLinter::evaluate("character.omni", &content, false, &config);
            assert_eq!(
                score.total, 100,
                "character.omni Block 4 Closing must score 100/100, got: {} ({:?})",
                score.total, score.breakdown
            );
            assert_eq!(score.rating, "A+ (Covenant Sealed & Verified)");
        }
    }

    #[test]
    fn test_character_tape_block4_commit_scores_100() {
        let config = SpatialLintConfig::default();
        let path = crate::core::filetype::resolve_track_path("lexicon/01_nouns/ideas/types/character.tape");
        if path.exists() {
            let content = std::fs::read_to_string(&path).expect("Read character.tape");
            let score = ClosingLinter::evaluate("character.tape", &content, true, &config);
            assert_eq!(
                score.total, 100,
                "character.tape Roll 6 Commit Status must score 100/100, got: {} ({:?})",
                score.total, score.breakdown
            );
            assert_eq!(score.rating, "A+ (Covenant Sealed & Verified)");
        }
    }

    #[test]
    fn test_number_omni_block4_closing_scores_100() {
        let config = SpatialLintConfig::default();
        let path = crate::core::filetype::resolve_track_path("lexicon/01_nouns/ideas/types/number.omni");
        if path.exists() {
            let content = std::fs::read_to_string(&path).expect("Read number.omni");
            let score = ClosingLinter::evaluate("number.omni", &content, false, &config);
            assert_eq!(
                score.total, 100,
                "number.omni Block 4 Closing must score 100/100, got: {} ({:?})",
                score.total, score.breakdown
            );
            assert_eq!(score.rating, "A+ (Covenant Sealed & Verified)");
        }
    }

    #[test]
    fn test_number_tape_block4_commit_scores_100() {
        let config = SpatialLintConfig::default();
        let path = crate::core::filetype::resolve_track_path("lexicon/01_nouns/ideas/types/number.tape");
        if path.exists() {
            let content = std::fs::read_to_string(&path).expect("Read number.tape");
            let score = ClosingLinter::evaluate("number.tape", &content, true, &config);
            assert_eq!(
                score.total, 100,
                "number.tape Roll 8 Commit Status must score 100/100, got: {} ({:?})",
                score.total, score.breakdown
            );
            assert_eq!(score.rating, "A+ (Covenant Sealed & Verified)");
        }
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/closing/reel.rs

RS | 113 LINES | 551 WORDS | 5.1 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Source Reel (.omni) Block 4 Closing Linter & Postcondition Verifier )
// ============================================================================
//! Source Reel (.omni) Block 4 Closing Linter
//!
//! Validates postcondition proofs, circuit breakers, and commit state machines:
//! - Operational roll verification proofs (`[ Verify ]: [ [ ROLL 1 ] = PROCEED :: ... ]`)
//! - Invariant validation & capacity bounds (`[ Validate ]: { Character` <= [ 1 Cell ] }`)
//! - Autonomous circuit breaker tripwiring (`[ Tripwire ]: ... --> [ ARMED ] | ... --> [ TRIPPED ]`)
//! - Diagnostic debug snapshot (`[ Debug ]: { ... }`)
//! - Episodic journal inscription (`[ Log ]: [ Journal ]: INSCRIBE { ... }`)
//! - Fourfold state machine (`+1:PROCEED`, `0:AWAIT`, `NULL:HALT`, `-1:ROLLBACK`)

use crate::diagnostics::linter::syntax::SpatialLintConfig;
use super::score::ClosingScore;

pub struct ReelClosingLinter;

impl ReelClosingLinter {
    /// Evaluates and scores Block 4 for `.omni` Source Reels.
    pub fn evaluate(
        _file_path: &str,
        slice: &str,
        config: &SpatialLintConfig,
    ) -> ClosingScore {
        let mut verify = 0i32;
        let mut validate = 0i32;
        let mut tripwire = 0i32;
        let mut fourfold = 0i32;
        let mut telemetry = 0i32;
        let mut breakdown = Vec::new();

        if slice.trim().is_empty() {
            breakdown.push("✗ Missing Block 4: Closing scope entirely (-50)".into());
            return ClosingScore::new(-20, -10, -10, -10, 0, breakdown);
        }

        // 1. Operational Verification Proofs (max 20 pts)
        let has_verify = slice.contains("Verify") || slice.contains("verify") || slice.contains("PROCEED");
        if has_verify {
            verify += 20;
            breakdown.push("✓ Operational verification proofs present (+20/20)".into());
        } else {
            verify -= 10;
            breakdown.push("✗ Missing operational verification proof stanza (-10/20)".into());
        }

        // 2. Invariant Validation & Bounds Parity (max 20 pts)
        let has_validate = slice.contains("Validate") || slice.contains("validate") || slice.contains("Enclosed") ||
slice.contains("Cell");
        if has_validate {
            validate += 20;
            breakdown.push("✓ Invariant bounds & substrate parity validated (+20/20)".into());
        } else {
            breakdown.push("⚠ No explicit validation bounds check (0/20)".into());
        }

        // 3. Autonomous Circuit Breaker / Tripwiring (max 20 pts)
        let has_tripwire = slice.contains("Tripwire") || slice.contains("ARMED") || slice.contains("TRIPPED") ||
slice.contains("circuit_breaker");
        if has_tripwire {
            tripwire += 20;
            breakdown.push("✓ Zero-tolerance tripwire circuit breaker armed (+20/20)".into());
        } else {
            breakdown.push("⚠ Execution operates without autonomous circuit breaker (0/20)".into());
        }

        // 4. Fourfold State Machine Completeness (max 25 pts)
        let upper = slice.to_uppercase();
        let states = ["PROCEED", "AWAIT", "HALT", "ROLLBACK"];
        let present_states = states.iter().filter(|&&s| upper.contains(s)).count();

```

```
        if present_states == 4 {
            fourfold += 25;
            breakdown.push("✓ Complete Balanced Ternary Fourfold State Machine (T4) affirmed (+25/25)".into());
        } else if present_states >= 2 {
            let pts = (present_states as i32) * 5;
            fourfold += pts;
            breakdown.push(format!("⚠ Partial commit state machine ({present_states}/4 states) (+{pts}/25)"));
        } else if upper.contains("COMMIT") || upper.contains("STATUS") {
            fourfold += 10;
            breakdown.push("✓ Basic commit status declared (+10/25)".into());
        } else {
            fourfold -= 15;
            breakdown.push("✗ Missing terminal commit status definition (-15/25)".into());
        }

        // 5. Telemetry & Journal Logging + Living Asides (max 15 pts)
        let has_telemetry = (slice.contains("Debug") || slice.contains("telemetry"))
            && (slice.contains("Log") || slice.contains("Journal"));
        let lines: Vec<&str> = slice.lines().collect();
        let max_cols = config.max_columns;
        let line_len_violation = lines.iter().any(|l| l.chars().count() > max_cols);

        if !line_len_violation {
            telemetry += 8;
            breakdown.push(format!("✓ All Block 4 lines <= {max_cols} columns (+8/8)"));
        } else {
            telemetry -= 8;
            breakdown.push(format!("✗ Block 4 lines exceed {max_cols}-column spatial ruler (-8/8)"));
        }

        if has_telemetry {
            telemetry += 7;
            breakdown.push("✓ Diagnostic debug snapshot & episodic logging verified (+7/7)".into());
        } else {
            telemetry += 4;
            breakdown.push("✓ Standard sealing gate verified (+4/7)".into());
        }

        ClosingScore::new(verify, validate, tripwire, fourfold, telemetry, breakdown)
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/closing/score.rs

RS | 71 LINES | 271 WORDS | 2.6 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Block 4 Closing Invariant Score Model & Rating Taxonomy )
// ============================================================================
//! OmniCode Block 4 Closing & Commit Status Score Model
//!
//! Evaluates postcondition verification proofs, tripwire circuit breakers,
//! and fourfold commit status completeness on a normalized $[-100 \dots +100]$ space:
//! 1. Operational Verification Proofs (max 20 pts)
//! 2. Invariant Validation & Substrate Parity (max 20 pts)
//! 3. Zero-Tolerance Tripwire Circuit Breaker (max 20 pts)
//! 4. Fourfold Commit State Machine Completeness (max 25 pts)
//! 5. Debug Telemetry, Journal Logging & Living Asides (max 15 pts)

#[derive(Debug, Clone, PartialEq, Eq)]
pub struct ClosingScore {
    pub verification_proofs_score: i32,
    pub invariant_validation_score: i32,
    pub tripwire_breaker_score: i32,
    pub fourfold_state_machine_score: i32,
    pub telemetry_asides_score: i32,
    pub total: i32,
    pub rating: &'static str,
    pub breakdown: Vec<String>,
}

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Rating Computation )
// ============================================================================

impl ClosingScore {
    pub fn compute_rating(total: i32) -> &'static str {
        match total {
            90..=100 => "A+ (Covenant Sealed & Verified)",
            75..=89 => "A (Compliant Closing)",
            50..=74 => "B (Adequate Verification)",
            20..=49 => "C (Unverified Commits)",
            0..=19 => "D (Missing Circuit Breakers)",
            _ => "F (Active Closing Hazard / Unsealed State)",
        }
    }

    pub fn new(
        verify: i32,
        validate: i32,
        tripwire: i32,
        fourfold: i32,
        telemetry: i32,
        breakdown: Vec<String>,
    ) -> Self {
        let total = (verify + validate + tripwire + fourfold + telemetry).clamp(-100, 100);
        Self {
            verification_proofs_score: verify,
            invariant_validation_score: validate,
            tripwire_breaker_score: tripwire,
            fourfold_state_machine_score: fourfold,
            telemetry_asides_score: telemetry,
            total,
            rating: Self::compute_rating(total),
            breakdown,
        }
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Status )
// ============================================================================
// ClosingScore sealed.
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/closing/tape_commit.rs

RS | 112 LINES | 499 WORDS | 4.6 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Process Tape (.tape) Terminal Commit Status Roll Linter )
// ============================================================================
//! Process Tape (.tape) Terminal Commit Status Roll Linter
//!
//! Validates the terminal sealing roll of a `.tape` process ribbon:
//! - Terminal roll identity (`[ ROLL N ] : [ Commit Status ] : [ ... ]`)
//! - Verification of preceding rolls achieving `+1:PROCEED`
//! - Tripwire circuit breaker arming
//! - Fourfold state machine completeness:
//!   `+1:PROCEED`, `0:AWAIT`, `NULL:HALT`, `-1:ROLLBACK`
//! - Attached Living Aside intent oracle

use crate::diagnostics::linter::syntax::SpatialLintConfig;
use super::score::ClosingScore;

pub struct TapeCommitLinter;

impl TapeCommitLinter {
    /// Evaluates the terminal commit status roll of a `.tape` file.
    pub fn evaluate(
        _file_path: &str,
        slice: &str,
        config: &SpatialLintConfig,
    ) -> ClosingScore {
        let mut verify = 0i32;
        let mut validate = 0i32;
        let mut tripwire = 0i32;
        let mut fourfold = 0i32;
        let mut telemetry = 0i32;
        let mut breakdown = Vec::new();

        if slice.trim().is_empty() {
            breakdown.push("✗ Missing terminal commit status roll (-50)".into());
            return ClosingScore::new(-20, -10, -10, -10, 0, breakdown);
        }

        // 1. Terminal Roll Identity & Verification (max 20 pts)
        let upper = slice.to_uppercase();
        let is_commit = upper.contains("COMMIT STATUS") || upper.contains("COMMIT_STATUS");
        if is_commit {
            verify += 20;
            breakdown.push("✓ Terminal Commit Status roll verified (+20/20)".into());
        } else {
            verify -= 10;
            breakdown.push("✗ Terminal roll is not a designated Commit Status roll (-10/20)".into());
        }

        // 2. Preceding Roll Affirmation (max 20 pts)
        let has_affirm = slice.contains("Verify") || slice.contains("PROCEED") || slice.contains("+1");
        if has_affirm {
            validate += 20;
            breakdown.push("✓ Affirmative execution condition verified (+20/20)".into());
        } else {
            breakdown.push("⚠ Implicit execution conditions (0/20)".into());
        }

        // 3. Tripwire Circuit Breaker Guard (max 20 pts)
        let has_tripwire = slice.contains("Tripwire") || slice.contains("ARMED") || slice.contains("TRIPPED");
        if has_tripwire {
            tripwire += 20;
            breakdown.push("✓ Zero-tolerance tripwire circuit breaker armed (+20/20)".into());
        } else {
            breakdown.push("⚠ Terminal commit operates without explicit tripwire check (0/20)".into());
        }

        // 4. Fourfold State Machine Completeness (max 25 pts)
        let states = ["PROCEED", "AWAIT", "HALT", "ROLLBACK"];
        let present_states = states.iter().filter(|&&s| upper.contains(s)).count();

        if present_states == 4 {
            fourfold += 25;
```

```
            breakdown.push("✓ Complete Balanced Ternary Fourfold State Machine (T4) affirmed (+25/25)".into());
        } else if present_states >= 2 {
            let pts = (present_states as i32) * 5;
            fourfold += pts;
            breakdown.push(format!("⚠ Partial commit state machine ({present_states}/4 states) (+{pts}/25)"));
        } else if is_commit {
            fourfold += 10;
            breakdown.push("✓ Basic commit status declared (+10/25)".into());
        } else {
            fourfold -= 15;
            breakdown.push("✗ Missing fourfold commit state machine (-15/25)".into());
        }

        // 5. Spatial Geometry & Living Asides (max 15 pts)
        let lines: Vec<&str> = slice.lines().collect();
        let max_cols = config.max_columns;
        let line_len_violation = lines.iter().any(|l| l.chars().count() > max_cols);

        if !line_len_violation {
            telemetry += 8;
            breakdown.push(format!("✓ All commit status lines <= {max_cols} columns (+8/8)"));
        } else {
            telemetry -= 8;
            breakdown.push(format!("✗ Commit status lines exceed {max_cols}-column spatial ruler (-8/8)"));
        }

        let has_aside = slice.contains('(') && slice.contains(')');
        if has_aside {
            telemetry += 7;
            breakdown.push("✓ Machine-readable Living Aside attached to terminal commit (+7/7)".into());
        } else {
            telemetry -= 5;
            breakdown.push("✗ Missing Living Aside on terminal commit (-5/7)".into());
        }

        ClosingScore::new(verify, validate, tripwire, fourfold, telemetry, breakdown)
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/syntax/README.md | MARKDOWN / DOC | 91 LINES | 555 WORDS | 4.5 KB

Linter Syntax Subsystem (`diagnostics::linter::syntax`)

> ***"Let all things be done decently and in order."*** *— 1 Corinthians 14:40 (KJV)*
>
> ***"Remove not the ancient landmark, which thy fathers have set."*** *— Proverbs 22:28 (KJV)*

1. Overview & Purpose

The **Syntax Subsystem** (`src/diagnostics/linter/syntax`) constitutes Pass 3 of the OmniCode linting pipeline. It verifies the grammatical, spatial, and typographical conventions of OmniCode source code.

OmniCode enforces a strict, mathematical harmony across space and grammar:

1. **Grammatical Order**: Operations follow deterministic Subject-Verb-Object (SVO) alignment.
2. **Spatial Boundaries**: Lines must respect the 120-column spatial boundary; tabs are forbidden in favor of 2 or 4-space indentations.
3. **Identifier Morphology**: Identifiers within brackets must follow snake_case, with explicit canonical exemptions for hardware bit rails (`Bit_0` .. `Bit_7`) and binary numeric literals (`0b`).

2. Component Architecture

```
syntax/
├── README.md        <-- This document
├── mod.rs           <-- Syntax coordinator & report aggregator
├── grammar.rs       <-- SVO verb validation, keyword rules, statement structure
└── spatial.rs       <-- 120-column limits, tab detection, bracket identifier linting
```

Module Responsibilities

- **`grammar.rs`** :
- Audits imperative and declarative verbs in `.tape` and `.omni` code.
- Recognizes canonical SVO verbs: `DEFINE` , `SET` , `EMIT` , `VERIFY` , `APPLY` , `PARTITION` , `CALCULATE` , and `CLOSE` .
- Flags unrecognizable or malformed instruction sequences.
- **`spatial.rs`** :
- Enforces the hard 120-character line length limit.
- Flags tab characters (`\t`), requiring pure space indentation.
- Validates bracketed identifiers: enforces `[snake_case]` conventions, while properly allowing canonical bit identifiers (`Bit_0` , `Bit_1` , etc.) and binary literals (`0b00101100`).

3. Grammar & Spatial Rules

Valid SVO Verbs in OmniCode

Verb	Category	Purpose	Canonical Example
`DEFINE`	Declarative	Type, schema, or constant declaration	`DEFINE Type Character`
`SET`	Mutative	Register or variable assignment	`SET Register_0 = 0x41`
`CALCULATE`	Transform	Arithmetic or logical transformation	`CALCULATE Result = A + B`
`PARTITION`	Structural	Bit slicing or memory segmentation	`PARTITION Register_0 INTO [Bit_0 .. Bit_7]`
`EMIT`	Telemetry	Output generation or event broadcast	`EMIT Event::CharProcessed`
`VERIFY`	Diagnostic	Invariant or contract assertion	`VERIFY Bit_7 == 0`
`APPLY`	Pipeline	Function application or transform pass	`APPLY Transform::Mask`

Verb	Category	Purpose	Canonical Example
`CLOSE`	Terminal	Block or session termination	`CLOSE Block`

Identifier Exemptions in Brackets

- Standard identifiers inside brackets must be lower snake_case: `[valid_identifier]`.
- **Bit Rails**: Hardware bit identifiers `Bit_0`, `Bit_1`, `Bit_2`, `Bit_3`, `Bit_4`, `Bit_5`, `Bit_6`, `Bit_7` (from `character.omni`) are explicitly exempted.
- **Numeric & Binary Literals**: Literals like `0b00101100`, `0x2A`, and numbers are recognized as values and exempted from identifier case rules.

---

4. Telemetry & Diagnostic Logger Integration

- Dispatches `SyntaxGrammarAnomaly` or `SyntaxSpatialAnomaly` failpoints to the `DiagnosticLogger`.
- Generates `FixSuggestion` instances for tabs (suggesting space replacement) and long lines (suggesting line break locations).
- Flags violations in episodic memory so automated generation scripts produce compliant code.

---

5. Verification & Tests

The `syntax` module includes 7 unit tests verifying:

- Valid SVO verbs and grammar structures.
- Spatial line length boundary checking (<= 120 chars).
- Bracket identifier case validation with `Bit_` and `0b` exemptions.
- Tab rejection and replacement suggestions.

Run module tests:

```
cargo test --lib diagnostics::linter::syntax
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/syntax/grammar.rs RS | 223 LINES | 904 WORDS | 9.3 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( OmniCode Lexical Syntax & Syntactic Grammar Linter )
// ================================================================================
//! # OmniCode Lexical Syntax & Grammar Linter
//!
//! Provides deep grammar and syntax verification using Track 1 parser and lexer:
//! 1. Full Lexical Tokenization Validation:
//!    - Detects unclosed strings, unclosed asides `( ... )`, illegal characters, and boundary leaks.
//! 2. Syntactic Grammar AST Validation:
//!    - Verifies SVO operational verbs (INSCRIBE, TRANSDUCE, PROJECT, SEQUENCE, PARTITION, CALCULATE).
//!    - Verifies directional vectors (`-->`, `<=>`, `::`, `&`, `|`).
//!    - Verifies state-prime synchronization (`Name``).
//!    - Validates hardware locators (`::@ 0x...`).
//!    - Validates delimiter balance across all 6 ranks of boundaries.
//!
//! # Scripture Witness:
//! - **Proverbs 8:8-9 (KJV)**:
//!   "All the words of my mouth are in righteousness; there is nothing froward or perverse in them.
//!    They are all plain to him that understandeth, and right to them that find knowledge."
//!   (True grammar contains no perversity or ambiguity; it is plain, pure, and mathematically sound).
//! - **Zephaniah 3:9 (KJV)**:
//!   "For then will I turn to the people a pure language, that they may all call upon the name of the LORD, to serve him with one consent."
//!   (A unified grammatical syntax enabling harmonious consent between humans, silicon, and AI agents).
// ================================================================================

use crate::diagnostics::debugger::OmniDebugger;
use crate::diagnostics::logger::{DiagnosticLogger, FailpointType};
use crate::diagnostics::linter::LintReport;
use crate::parser::lexer::{Lexer, TokenType};
use crate::parser::parser::Parser;

// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( GrammarLinter Definition )
// ================================================================================

pub struct GrammarLinter;

// ================================================================================
// BLOCK 3: BODY BLOCK
// ( Token Stream Hygiene & AST Grammar Verification )
// ================================================================================

impl GrammarLinter {
    /// Performs comprehensive lexical syntax and syntactic grammar analysis on source code.
    pub fn lint(
        file_path: &str,
        content: &str,
        logger: DiagnosticLogger,
        debugger: OmniDebugger,
        report: &mut LintReport,
    ) {
        // --------------------------------------------------------------------
        // 1. Lexical Tokenization Pass
        // --------------------------------------------------------------------
        let mut lexer = Lexer::new(content, logger.clone());
        let tokens = match lexer.tokenize() {
            Ok(toks) => toks,
            Err(e) => {
                let err_msg = format!("Syntax Grammar Error (Lexical): {e}");
                logger.failpoint(
                    FailpointType::SyntaxAnomaly,
                    "GRAMMAR",
                    format!("[{file_path}] {err_msg}"),
                );
                report.errors.push(err_msg);
                return;
            }
        };

        // --------------------------------------------------------------------
```

```
        // 2. Token Stream Grammatical Pattern Checks
        // ------------------------------------------------------------------
        let mut in_roll = false;
        let mut current_roll_line = 0;
        let mut roll_has_aside = false;

        for (idx, tok) in tokens.iter().enumerate() {
            // Check State Prime Grammatical Hygiene
            if let TokenType::Identifier(ref id) = tok.token_type {
                if id.ends_with('`') && id.len() > 1 {
                    // Valid state-prime notation
                }
            }

            // Track Rolls and verify each has an attached Living Aside
            if let TokenType::Identifier(ref id) = tok.token_type {
                if id == "ROLL" || id == "Roll" {
                    let is_roll_decl = idx + 2 < tokens.len() && matches!(tokens[idx + 2].token_type,
TokenType::InRelationTo);
                    if is_roll_decl {
                        if in_roll && !roll_has_aside && current_roll_line > 0 {
                            report.warnings.push(format!(
                                "Grammar Contract (line {current_roll_line}): Operational roll missing attached Living
Aside '( ... )' oracle."
                            ));
                        }
                        in_roll = true;
                        current_roll_line = tok.line;
                        roll_has_aside = false;
                    }
                }
            }

            if in_roll {
                if let TokenType::Aside(_) = tok.token_type {
                    roll_has_aside = true;
                }
            }

            // Check SVO Verb Grammar: Verbs must be followed by target containers
            // Operational verbs in active positions are ALL-CAPS: INSCRIBE, TRANSDUCE, PROJECT, SEQUENCE, PARTITION,
CALCULATE
            if let TokenType::Identifier(ref id) = tok.token_type {
                if matches!(id.as_str(), "INSCRIBE" | "TRANSDUCE" | "PROJECT" | "SEQUENCE" | "PARTITION" | "CALCULATE")
{
                    let is_chamber = idx > 0 && matches!(tokens[idx - 1].token_type, TokenType::LBracket);
                    if !is_chamber && idx + 1 < tokens.len() {
                        let next = &tokens[idx + 1];
                        if !next.token_type.is_open_boundary() && !matches!(&next.token_type, TokenType::Identifier(_))
{
                            report.warnings.push(format!(
                                "Grammar Error (line {}): SVO Verb '{}' must be followed by a target container or
operand, found '{:?}'.",
                                tok.line, id, next.token_type
                            ));
                        }
                    }
                }
            }
        }

        // Final roll aside check
        if in_roll && !roll_has_aside && current_roll_line > 0 {
            report.warnings.push(format!(
                "Grammar Contract (line {current_roll_line}): Operational roll missing attached Living Aside '( ... )'
oracle."
            ));
        }

        // ------------------------------------------------------------------
        // 3. Syntactic Parser AST Pass (for non-empty code)
        // ------------------------------------------------------------------
        // For .omni files, parse the entire program AST
        if file_path.ends_with(".omni") || content.contains("= [Module]:") || content.contains("Module <") {
            let mut parser = Parser::new(tokens, debugger.clone());
            match parser.parse_program() {
                Ok(_ast) => {
                    // AST successfully parsed and validated
```

```
                }
                Err(e) => {
                    let err_msg = format!("Syntactic Grammar Error: {e}");
                    logger.failpoint(
                        FailpointType::SyntaxAnomaly,
                        "GRAMMAR",
                        format!("[{file_path}] {err_msg}"),
                    );
                    report.errors.push(err_msg);
                }
            }
        }
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Unit Tests & Grammar AST Verification )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_grammar_linter_clean_code_passes() {
        let code = r#"
= [Module]: < Test > {
    [ Version ]: { "1.0.0" }
}
Opening { Cell [ 1 ] }
Body [
    [ ROLL 1 ] : [ Inscribe ] : { 'A' } --> [ Cell ] ( Step 1 )
]
Closing { [ Commit Status ]: [ +1:PROCEED ] }
"#;
        let logger = DiagnosticLogger::new(false);
        let debugger = OmniDebugger::new(logger.clone(), 5);
        let mut report = LintReport {
            file_path: "clean.omni".to_string(),
            file_type: None,
            blocks_found: Vec::new(),
            metadata_score: None,
            audit_score: None,
            is_valid: true,
            errors: Vec::new(),
            warnings: Vec::new(),
        };

        GrammarLinter::lint("clean.omni", code, logger.clone(), debugger, &mut report);
        assert!(report.errors.is_empty(), "Errors: {:?}", report.errors);
        assert!(!logger.has_failpoints());
    }

    #[test]
    fn test_grammar_linter_unclosed_string_fails() {
        let code = r#"
= [Module]: < Broken > {
    [ Version ]: { "1.0.0
}
"#;
        let logger = DiagnosticLogger::new(false);
        let debugger = OmniDebugger::new(logger.clone(), 5);
        let mut report = LintReport {
            file_path: "broken.omni".to_string(),
            file_type: None,
            blocks_found: Vec::new(),
            metadata_score: None,
            audit_score: None,
            is_valid: true,
            errors: Vec::new(),
            warnings: Vec::new(),
        };

        GrammarLinter::lint("broken.omni", code, logger.clone(), debugger, &mut report);
        assert!(!report.errors.is_empty());
        assert!(logger.has_failpoints());
```

```
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/syntax/mod.rs

RS | 70 LINES | 283 WORDS | 2.8 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Syntax, Spatial & Grammar Subsystem Coordinator )
// ============================================================================
//! # Syntax, Spatial Geometry & Grammar Linter Subsystem
//!
//! Enforces lexical syntax, grammatical AST correctness, and spatial geometry:
//! - [`spatial`]: 80-column IDE dual side-panel ruler, Living Asides, and bracket identifier naming hygiene.
//! - [`grammar`]: Lexer token stream hygiene, AST parser verification, SVO verbs, and state-prime notation.
//!
//! # Scripture Witness:
//! - **1 Corinthians 14:40 (KJV)**:
//!   "Let all things be done decently and in order."
//!   (Every line, token, delimiter, and aside must preserve rhythmic decency and grammatical order).
//! - **Isaiah 28:10 (KJV)**:
//!   "For precept must be upon precept, precept upon precept; line upon line, line upon line; here a little, and there a little..."
//!   (Columnar alignment and rule-by-rule grammatical parsing across the source tape).
// ============================================================================

pub mod grammar;
pub mod spatial;

pub use grammar::GrammarLinter;
pub use spatial::{SpatialLintConfig, SpatialLinter};

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Syntax Subsystem Initialization & Default Configuration )
// ============================================================================

/// Constructs the default spatial and grammar linting configuration.
pub fn default_syntax_config() -> SpatialLintConfig {
    SpatialLintConfig::default()
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Subsystem Pipeline Coordination )
// ============================================================================

/// Helper for running both spatial and grammar passes on source code.
pub fn run_syntax_passes(
    file_path: &str,
    content: &str,
    logger: crate::diagnostics::logger::DiagnosticLogger,
    debugger: crate::diagnostics::debugger::OmniDebugger,
    report: &mut crate::diagnostics::linter::LintReport,
    config: &SpatialLintConfig,
) {
    SpatialLinter::lint(content, report, config);
    GrammarLinter::lint(file_path, content, logger, debugger, report);
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Unit Tests & Calibration Verification )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_syntax_default_config() {
        let cfg = default_syntax_config();
        assert_eq!(cfg.max_columns, 80);
        assert!(!cfg.strict_column_limit);
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/syntax/spatial.rs

RS | 191 LINES | 706 WORDS | 7.9 KB

```rust
// ================================================================================
// BLOCK 1: METADATA BLOCK
// ( Spatial Geometry, Ruler Invariants & Syntax Hygiene Linter )
// ================================================================================
//! # OmniCode Spatial Geometry & Layout Linter
//!
//! Enforces physical spatial invariants across all OmniCode files:
//! 1. 80-Column Spatial Ruler: Guarantees legibility in IDE viewports with dual side panels.
//! 2. Delimiter & Bracket Hygiene: Disallows snake_case inside brackets (natural spaced phrases),
//!    exempting canonical bit rails (`Bit_0`..`Bit_7`) and hex/binary addresses.
//! 3. First-Class Intent Asides: Mandates Living Asides `( ... )` for AI cognitive prompts and test oracles.
//! 4. Multiline Compound Sentence Formatting: Flags run-on clauses that break columnar rhythm.
//!
//! # Scripture Witness:
//! - **Proverbs 22:28 (KJV)**:
//!   "Remove not the ancient landmark, which thy fathers have set."
//!   (The 80-column spatial boundary is the ancient landmark of readability and IDE viewport balance).
//! - **Isaiah 40:12 (KJV)**:
//!   "Who hath measured the waters in the hollow of his hand, and meted out heaven with the span..."
//!   (Precise geometric measurement ensures order across every horizontal and vertical axis).
// ================================================================================

use crate::diagnostics::linter::LintReport;

// ================================================================================
// BLOCK 2: OPENING BLOCK
// ( Configuration Structure & SpatialLinter Definition )
// ================================================================================

/// Configuration for spatial geometry and layout linting.
#[derive(Debug, Clone)]
pub struct SpatialLintConfig {
    /// Maximum allowed column width (default: 80).
    pub max_columns: usize,
    /// Whether column limit violations constitute fatal errors or advisory warnings.
    pub strict_column_limit: bool,
}

impl Default for SpatialLintConfig {
    fn default() -> Self {
        Self {
            max_columns: 80,
            strict_column_limit: false,
        }
    }
}

pub struct SpatialLinter;

// ================================================================================
// BLOCK 3: BODY BLOCK
// ( Spatial Ruler Enforcement & Layout Verification )
// ================================================================================

impl SpatialLinter {
    /// Lints source text against physical spatial geometry and syntax hygiene rules.
    pub fn lint(content: &str, report: &mut LintReport, config: &SpatialLintConfig) {
        let lines: Vec<&str> = content.lines().collect();

        // ------------------------------------------------------------------------
        // 1. 80-Column Spatial Ruler Verification
        // ------------------------------------------------------------------------
        for (idx, line) in lines.iter().enumerate() {
            let line_num = idx + 1;
            let col_count = line.chars().count();
            if col_count > config.max_columns {
                let msg = format!(
                    "Spatial Geometry (line {line_num}): Line exceeds {max} columns ({col_count} cols). \
                     OmniCode mandates <= {max} columns for dual side panel IDE viewports.",
                    max = config.max_columns
                );
                if config.strict_column_limit {
                    report.errors.push(msg);
```

```
                } else {
                    report.warnings.push(msg);
                }
            }
        }

        // ------------------------------------------------------------------
        // 2. First-Class Intent Asides Presence (Canon 1)
        // ------------------------------------------------------------------
        let total_lines = lines.len();
        let has_any_asides = content.contains('(') && content.contains(')');
        if total_lines > 15 && !has_any_asides {
            report.warnings.push(
                "Canon 1: Missing first-class intent Asides '(...)'. OmniCode mandates Asides \
                 for human-AI pronunciation, system prompt seeds, and pre-test oracles.".into(),
            );
        }

        // ------------------------------------------------------------------
        // 3. Snake_case Hygiene Inside Brackets
        // ------------------------------------------------------------------
        for (idx, line) in lines.iter().enumerate() {
            let line_num = idx + 1;
            let trimmed = line.trim();
            if trimmed.starts_with('#') {
                continue;
            }
            if let Some(start_bracket) = line.find('[') {
                if let Some(end_bracket) = line[start_bracket..].find(']') {
                    let bracket_content = &line[start_bracket + 1..start_bracket + end_bracket];
                    for word in bracket_content.split_whitespace() {
                        let is_exempt = word.starts_with("0x")
                            || word.starts_with("0b")
                            || word.starts_with("0X")
                            || word.starts_with("roll_")
                            || word.starts_with("Bit_")
                            || word.starts_with("bit_");
                        if word.contains('_') && !is_exempt {
                            report.warnings.push(format!(
                                "Syntax Hygiene (line {line_num}): Identifier '{word}' uses snake_case with underscores
inside brackets. \
                                 In OmniCode, use natural spaced phrases (e.g. '[ ... ]') without underscores."
                            ));
                            break;
                        }
                    }
                }
            }
        }
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Unit Tests & Spatial Invariant Calibration )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_spatial_linter_ruler_warning_and_strict() {
        let wide_line = format!("# {}\n", "A".repeat(85));

        // Advisory mode
        let mut report_adv = LintReport {
            file_path: "test.omni".to_string(),
            file_type: None,
            blocks_found: Vec::new(),
            metadata_score: None,
            audit_score: None,
            is_valid: true,
            errors: Vec::new(),
            warnings: Vec::new(),
        };
        let cfg_adv = SpatialLintConfig::default();
        SpatialLinter::lint(&wide_line, &mut report_adv, &cfg_adv);
```

```
        assert!(report_adv.errors.is_empty());
        assert!(!report_adv.warnings.is_empty());

        // Strict mode
        let mut report_strict = LintReport {
            file_path: "test.omni".to_string(),
            file_type: None,
            blocks_found: Vec::new(),
            metadata_score: None,
            audit_score: None,
            is_valid: true,
            errors: Vec::new(),
            warnings: Vec::new(),
        };
        let cfg_strict = SpatialLintConfig {
            max_columns: 80,
            strict_column_limit: true,
        };
        SpatialLinter::lint(&wide_line, &mut report_strict, &cfg_strict);
        assert!(!report_strict.errors.is_empty());
    }

    #[test]
    fn test_spatial_linter_bit_rail_exempt_from_snake_case_warning() {
        let code = "[ Bit_7 ]: \"0\"\n";
        let mut report = LintReport {
            file_path: "bit.omni".to_string(),
            file_type: None,
            blocks_found: Vec::new(),
            metadata_score: None,
            audit_score: None,
            is_valid: true,
            errors: Vec::new(),
            warnings: Vec::new(),
        };
        let cfg = SpatialLintConfig::default();
        SpatialLinter::lint(code, &mut report, &cfg);
        assert!(report.warnings.is_empty(), "Bit_7 must be exempt from snake_case warnings");
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/metadata/README.md | MARKDOWN / DOC | 72 LINES | 446 WORDS | 3.1 KB

Linter Metadata Subsystem (`diagnostics::linter::metadata`)

> ***"And he gave unto Moses, when he had made an end of communing with him upon mount Sinai, two tables of testimony, tables of stone, written with the finger of God."*** *— Exodus 31:18 (KJV)*
>
> ***"He telleth the number of the stars; he calleth them all by their names."*** *— Psalm 147:4 (KJV)*

1. Overview & Purpose

The **Metadata Subsystem** (`src/diagnostics/linter/metadata`) audits **BLOCK 1: METADATA** of all OmniCode files (`.omni` and `.tape`).

In the OmniCode philosophy, code without explicit identity, provable authorship, and foundational ethical/spiritual grounding is untrusted and unprovable. Block 1 establishes the provenance, semantic ontology, versioning epoch, and sacred witness of the code unit before any execution or type definition is evaluated.

2. Block 1 Specification Requirements

A canonical `BLOCK 1: METADATA` must include the following attributes, demonstrated in `character.omni` and `number.omni` :

Field Name	Description	Required	Example
Header	Block identification tag	Yes	`[BLOCK 1: METADATA]` or `// BLOCK 1: METADATA`
Title / Module	Formal name of the specification	Yes	`Title: Canonical Character Specification`
Author / Signer	Responsible creator or authority	Yes	`Author: OmniCode Framework Authors`
Epoch / Version	Temporal epoch and semantic version	Yes	`Epoch: 2026.1 / Version: 1.0.0`
License / Law	Legal or open framework governance	Yes	`License: MIT / Apache-2.0`
Witness (KJV)	Foundational Scripture citation	Yes	`Witness: Proverbs 11:1 (KJV)`

3. Scoring & Audit Rules

The metadata linter computes a score between 0.0 and 100.0:

- **Missing Header**: -40 points.
- **Missing Title / Module**: -15 points.
- **Missing Author**: -15 points.
- **Missing Epoch / Version**: -10 points.
- **Missing License**: -10 points.
- **Missing Scripture Witness**: -10 points.

A file with all required fields scores `100.0/100.0` and satisfies the `QuadStateGate::Proceed` criterion for Block 1.

4. Telemetry & Diagnostic Logger Integration

When metadata elements are missing or malformed:

- A `MetadataAnomaly` failpoint is emitted to the `DiagnosticLogger` .
- A remediation suggestion is created specifying the missing fields with canonical skeleton text.
- An episodic `CognitiveNote` records the violation so future generation passes automatically include the required metadata headers.

5. Verification & Tests

The `metadata` module includes 4 unit tests verifying:

- Canonical `number.omni` Block 1 metadata (scores 100.0).
- Canonical `number.tape` Block 1 metadata (scores 100.0).
- Detection of missing title, author, or witness fields with precise score degradation.

Run module tests:

```
cargo test --lib diagnostics::linter::metadata
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/metadata/head.rs RS 247 LINES 953 WORDS 10.3 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Process Tape (.tape) Block 1 Tape Head & Cardinal Coordinates Linter )
// ============================================================================
//! Process Tape (.tape) Block 1 Tape Head Linter
//!
//! Evaluates and scores the Tape Head manifest for `.tape` Process Ribbons.
//! While `.omni` files represent declarative Source Reels, `.tape` files represent
//! sequential execution ribbons. Both embody the exact same ontological covenant
//! and foundation, but with process ribbon mechanics:
//! 1. Formal Tape Head Declaration (`= [Tape]: < ... > & [ ... ] {`).
//! 2. Defined Operational Space & Partitions (`WHERE { ... Cell } = [ 1 ]`).
//! 3. Covenant Witness Grounding (`< Scripture { ... } >` anchor).
//! 4. The Six Cardinal Coordinates (`WHO`, `WHAT`, `WHEN`, `WHERE`, `WHY`, `HOW`)
//!    with operational verbs (`--> INSCRIBE`) and causal directionals.
//! 5. Spatial Geometry (<= 80 columns) and bracket naming hygiene.

use crate::diagnostics::linter::syntax::SpatialLintConfig;
use super::score::MetadataScore;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( TapeHeadLinter Definition & Slice Extractor )
// ============================================================================

pub struct TapeHeadLinter;

impl TapeHeadLinter {
    /// Extracts the text slice corresponding exclusively to the Tape Head.
    pub fn extract_slice(content: &str) -> (usize, usize, String) {
        let lines: Vec<&str> = content.lines().collect();
        if lines.is_empty() {
            return (0, 0, String::new());
        }

        let mut start_idx = None;
        let mut end_idx = None;

        for (idx, line) in lines.iter().enumerate() {
            let trimmed = line.trim();
            if start_idx.is_none() {
                if trimmed.starts_with("= [Tape")
                    || trimmed.starts_with("[Tape.")
                    || trimmed.starts_with("Tape [")
                    || trimmed.starts_with("Tape <")
                {
                    start_idx = Some(idx);
                }
            } else {
                let upper = trimmed.to_uppercase();
                if upper.starts_with("[ ROLL")
                    || upper.starts_with("ROLL_")
                    || trimmed.starts_with("# DECLARED ROLLS")
                {
                    end_idx = Some(idx);
                    break;
                }
            }
        }

        let start = start_idx.unwrap_or(0);
        let end = end_idx.unwrap_or(lines.len());
        (start + 1, end, lines[start..end].join("\n"))
    }
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Evaluation Engine for Process Tape Head Invariants )
// ============================================================================

impl TapeHeadLinter {
    /// Evaluates and scores Block 1 for a `.tape` Process Ribbon.
```

```
    pub fn evaluate(
        _file_path: &str,
        slice: &str,
        config: &SpatialLintConfig,
    ) -> MetadataScore {
        let mut identity = 0u32;
        let mut provenance = 0u32;
        let mut witness = 0u32;
        let mut ontology = 0u32;
        let mut spatial = 0u32;
        let mut breakdown = Vec::new();

        // ---------------------------------------------------------------------
        // 1. Identity & Formal Tape Declaration (max 20 pts)
        // ---------------------------------------------------------------------
        let has_tape_decl = slice.contains("= [Tape]:")
            || slice.contains("[Tape.")
            || slice.contains("Tape [")
            || slice.contains("Tape <");
        if has_tape_decl {
            identity += 15;
            breakdown.push("✓ Formal Process Tape Head declared (+15/15)".into());
        } else {
            breakdown.push("✗ Missing formal tape head declaration (0/15)".into());
        }

        let has_tape_context = (slice.contains('<') && slice.contains('>'))
            && (slice.contains('[') && slice.contains(']'));
        if has_tape_context {
            identity += 5;
            breakdown.push("✓ Dual-domain tape contextualization affirmed (+5/5)".into());
        } else {
            breakdown.push("✗ Missing dual domain context in tape declaration (0/5)".into());
        }

        // ---------------------------------------------------------------------
        // 2. Operational Environment & Topology (max 25 pts)
        // ---------------------------------------------------------------------
        let has_defined_space = slice.contains("WHERE")
            && (slice.contains("Cell") || slice.contains("Partition") || slice.contains('['));
        if has_defined_space {
            provenance += 10;
            breakdown.push("✓ Defined space / cell partition bounds declared (+10/10)".into());
        } else {
            breakdown.push("✗ Missing defined space or cell bounds in tape head (0/10)".into());
        }

        let has_domain_context = slice.contains("Lexicon") || slice.contains("Kernel") || slice.contains("Domain");
        if has_domain_context {
            provenance += 10;
            breakdown.push("✓ Execution domain context declared (+10/10)".into());
        } else {
            breakdown.push("✗ Missing execution domain context (0/10)".into());
        }

        let has_comments = slice.contains("TOPOLOGY")
            || slice.contains("PROCESS TAPE")
            || slice.lines().any(|l| l.trim().starts_with('#'));
        if has_comments {
            provenance += 5;
            breakdown.push("✓ Architectural topology commentary inscribed (+5/5)".into());
        } else {
            breakdown.push("✗ Missing architectural topology header commentary (0/5)".into());
        }

        // ---------------------------------------------------------------------
        // 3. Covenant & Witness Invariant Grounding (max 20 pts)
        // ---------------------------------------------------------------------
        let has_scripture = slice.contains("< Scripture") || slice.contains("Exodus");
        if has_scripture {
            witness += 15;
            breakdown.push("✓ Invariant Scripture / Covenant witness anchored (+15/15)".into());
        } else {
            breakdown.push("✗ Missing scripture/covenant witness in tape head (0/15)".into());
        }

        let has_witness_aside = slice.contains("Invariant Witness")
            || (has_scripture && slice.contains('(') && slice.contains(')'));
```

```
        if has_witness_aside {
            witness += 5;
            breakdown.push("✓ Machine-readable Witness Aside oracle attached (+5/5)".into());
        } else {
            breakdown.push("✗ Missing witness Aside oracle (0/5)".into());
        }

        // ------------------------------------------------------------------
        // 4. Six Cardinal Coordinates & Operational Verbs (max 20 pts)
        // ------------------------------------------------------------------
        let upper = slice.to_uppercase();
        let cardinal_clauses = ["WHO", "WHAT", "WHEN", "WHERE", "WHY", "HOW"];
        let present_clauses = cardinal_clauses.iter().filter(|&&c| upper.contains(c)).count();

        if present_clauses == 6 {
            ontology += 15;
            breakdown.push(
                "✓ All Six Cardinal Coordinates (WHO, WHAT, WHEN, WHERE, WHY, HOW) present (+15/15)".into(),
            );
        } else {
            let pts = (present_clauses as u32) * 2;
            ontology += pts;
            breakdown.push(format!(
                "⚠ Cardinal coordinates incomplete ({present_clauses}/6) (+{pts}/15)"
            ));
        }

        let has_op_verbs = upper.contains("INSCRIBE") || upper.contains("PROCEED") || upper.contains("-->");
        if has_op_verbs {
            ontology += 5;
            breakdown.push("✓ Operational SVO verb and directional vectors wired (+5/5)".into());
        } else {
            breakdown.push("✗ Missing operational verbs or directional vectors in cardinal clauses (0/5)".into());
        }

        // ------------------------------------------------------------------
        // 5. Spatial Geometry & Syntax Hygiene (max 15 pts)
        // ------------------------------------------------------------------
        let lines: Vec<&str> = slice.lines().collect();
        let max_cols = config.max_columns;
        let line_len_violation = lines.iter().any(|l| l.chars().count() > max_cols);

        if !line_len_violation {
            spatial += 10;
            breakdown.push(format!("✓ All Tape Head lines <= {max_cols} columns (+10/10)"));
        } else {
            breakdown.push(format!("✗ Tape Head lines exceed {max_cols}-column spatial ruler (0/10)"));
        }

        let snake_violation = lines.iter().any(|line| {
            let trimmed = line.trim();
            if trimmed.starts_with('#') {
                return false;
            }
            if let Some(start) = line.find('[') {
                if let Some(end) = line[start..].find(']') {
                    let inside = &line[start + 1..start + end];
                    for w in inside.split_whitespace() {
                        if w.contains('_') && !w.starts_with("0x") && !w.starts_with("roll_") {
                            return true;
                        }
                    }
                }
            }
            false
        });

        if !snake_violation {
            spatial += 5;
            breakdown.push("✓ Natural spaced identifier hygiene observed (+5/5)".into());
        } else {
            breakdown.push("✗ snake_case identifiers found inside brackets (0/5)".into());
        }

        MetadataScore::new(identity, provenance, witness, ontology, spatial, breakdown)
    }
}

```

```
// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Unit Tests for Tape Head Block 1 Linting )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_tape_head_extraction_and_empty_check() {
        let (start, end, slice) = TapeHeadLinter::extract_slice("");
        assert_eq!(start, 0);
        assert_eq!(end, 0);
        assert!(slice.is_empty());
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/metadata/mod.rs

RS | 168 LINES | 617 WORDS | 7.6 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Block 1 Metadata & Tape Head Invariant Subsystem Coordinator )
// ============================================================================
//! # OmniCode Block 1 Metadata & Tape Head Subsystem
//!
//! Organizes Block 1 verification into dedicated, domain-specific modules:
//! - [`score`]: `MetadataScore` invariant model, point totals, and rating tiers.
//! - [`reel`]: Source Reel (`.omni`) Module Manifest & Invariant Type inspection.
//! - [`head`]: Process Tape (`.tape`) Tape Head & Cardinal Coordinates inspection.
//!
//! While `.omni` and `.tape` have different syntaxes and structures, both
//! embody the same proper ontological understanding: foundational identity,
//! provenance, covenant witness grounding, invariants/clauses, and spatial hygiene.
//!
//! # Scripture Witness:
//! - **Exodus 31:18 (KJV)**:
//!   "And he gave unto Moses, when he had made an end of communing with him upon mount Sinai,
//!    two tables of testimony, tables of stone, written with the finger of God."
//!   (The immutable inscription of metadata provenance and divine covenant witnesses).
//! - **Psalm 147:4 (KJV)**:
//!   "He telleth the number of the stars; he calleth them all by their names."
//!   (Every module, type, and tape possesses unique identity and cardinal coordinates).
// ============================================================================

pub mod head;
pub mod reel;
pub mod score;

pub use head::TapeHeadLinter;
pub use reel::ReelMetadataLinter;
pub use score::MetadataScore;

use crate::diagnostics::linter::syntax::SpatialLintConfig;
use crate::diagnostics::linter::LintReport;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( MetadataLinter Coordinator Definition )
// ============================================================================

pub struct MetadataLinter;

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Polymorphic Dispatch Across Source Reels and Process Tapes )
// ============================================================================

impl MetadataLinter {
    /// Evaluates and scores Block 1 for either a Source Reel or Process Tape,
    /// selecting the dedicated linter while preserving unified understanding.
    pub fn evaluate(
        file_path: &str,
        content: &str,
        is_tape: bool,
        config: &SpatialLintConfig,
        _report: &mut LintReport,
    ) -> MetadataScore {
        let blocks = crate::diagnostics::linter::structure::detector::BlockDetector::detect_blocks(content, is_tape);
        if is_tape {
            TapeHeadLinter::evaluate(file_path, &blocks.metadata.content, config)
        } else {
            ReelMetadataLinter::evaluate(file_path, &blocks.metadata.content, config)
        }
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Unit Tests & Calibration Verification )
// ============================================================================

#[cfg(test)]
```

```rust
mod tests {

    #[test]
    fn test_character_omni_block1_scores_100() {
        let logger = crate::diagnostics::logger::DiagnosticLogger::new(false);
        let linter = crate::diagnostics::linter::OmniLinter::new(logger);
        let path = crate::core::filetype::resolve_track_path("lexicon/01_nouns/ideas/types/character.omni");
        if path.exists() {
            let report = linter.lint_file(&path).expect("Lint character.omni");
            assert!(report.is_valid, "character.omni must be valid");
            let score = report.metadata_score.expect("Metadata score must be present");
            assert_eq!(
                score.total, 100,
                "character.omni Block 1 must score 100/100, got: {} ({:?})",
                score.total, score.breakdown
            );
            assert_eq!(score.rating, "A+ (Exemplary Invariant)");
            assert_eq!(score.identity_score, 20);
            assert_eq!(score.provenance_score, 25);
            assert_eq!(score.witness_score, 20);
            assert_eq!(score.ontology_score, 20);
            assert_eq!(score.spatial_hygiene_score, 15);
        }
    }

    #[test]
    fn test_character_tape_block1_scores_100() {
        let logger = crate::diagnostics::logger::DiagnosticLogger::new(false);
        let linter = crate::diagnostics::linter::OmniLinter::new(logger);
        let path = crate::core::filetype::resolve_track_path("lexicon/01_nouns/ideas/types/character.tape");
        if path.exists() {
            let report = linter.lint_file(&path).expect("Lint character.tape");
            assert!(report.is_valid, "character.tape must be valid");
            let score = report.metadata_score.expect("Metadata score must be present");
            assert_eq!(
                score.total, 100,
                "character.tape Block 1 must score 100/100, got: {} ({:?})",
                score.total, score.breakdown
            );
            assert_eq!(score.rating, "A+ (Exemplary Invariant)");
            assert_eq!(score.identity_score, 20);
            assert_eq!(score.provenance_score, 25);
            assert_eq!(score.witness_score, 20);
            assert_eq!(score.ontology_score, 20);
            assert_eq!(score.spatial_hygiene_score, 15);
        }
    }

    #[test]
    fn test_number_omni_block1_scores_100() {
        let logger = crate::diagnostics::logger::DiagnosticLogger::new(false);
        let linter = crate::diagnostics::linter::OmniLinter::new(logger);
        let path = crate::core::filetype::resolve_track_path("lexicon/01_nouns/ideas/types/number.omni");
        if path.exists() {
            let report = linter.lint_file(&path).expect("Lint number.omni");
            assert!(report.is_valid, "number.omni must be valid: {:?}", report.errors);
            let score = report.metadata_score.expect("Metadata score must be present");
            assert_eq!(
                score.total, 100,
                "number.omni Block 1 must score 100/100, got: {} ({:?})",
                score.total, score.breakdown
            );
            assert_eq!(score.rating, "A+ (Exemplary Invariant)");
            assert_eq!(score.identity_score, 20);
            assert_eq!(score.provenance_score, 25);
            assert_eq!(score.witness_score, 20);
            assert_eq!(score.ontology_score, 20);
            assert_eq!(score.spatial_hygiene_score, 15);
        }
    }

    #[test]
    fn test_number_tape_block1_scores_100() {
        let logger = crate::diagnostics::logger::DiagnosticLogger::new(false);
        let linter = crate::diagnostics::linter::OmniLinter::new(logger);
        let path = crate::core::filetype::resolve_track_path("lexicon/01_nouns/ideas/types/number.tape");
        if path.exists() {
            let report = linter.lint_file(&path).expect("Lint number.tape");
```

```
            assert!(report.is_valid, "number.tape must be valid: {:?}", report.errors);
            let score = report.metadata_score.expect("Metadata score must be present");
            assert_eq!(
                score.total, 100,
                "number.tape Block 1 must score 100/100, got: {} ({:?})",
                score.total, score.breakdown
            );
            assert_eq!(score.rating, "A+ (Exemplary Invariant)");
            assert_eq!(score.identity_score, 20);
            assert_eq!(score.provenance_score, 25);
            assert_eq!(score.witness_score, 20);
            assert_eq!(score.ontology_score, 20);
            assert_eq!(score.spatial_hygiene_score, 15);
        }
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/metadata/reel.rs

RS | 250 LINES | 995 WORDS | 10.6 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Source Reel (.omni) Block 1 Metadata & Invariant Type Linter )
// ============================================================================
//! Source Reel (.omni) Block 1 Metadata Linter
//!
//! Evaluates and scores the ontological foundation of `.omni` files:
//! 1. Formal Module Identity (`= [Module]: < ... > & [ ... ]` with intent Aside).
//! 2. Invariant Provenance (`Version`, `Author`, `Created`, `Updated`, `System`).
//! 3. Covenant Witness Grounding (`< Scripture { ... } >` or `[ Witness ]:`).
//! 4. Ontological Invariants & Freestanding Type Definitions (`< Type > = ...`
//!    across the 9 dimensions: Relation, Domain, Substrate, Inscribe, Transduce,
//!    Exemplar, Compare, Sequence, Bound).
//! 5. Spatial Geometry (<= 80 columns) and bracket identifier naming hygiene.

use crate::diagnostics::linter::syntax::SpatialLintConfig;
use super::score::MetadataScore;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( ReelMetadataLinter Definition & Slice Extractor )
// ============================================================================

pub struct ReelMetadataLinter;

impl ReelMetadataLinter {
    /// Extracts the text slice corresponding to Block 1 in a `.omni` file.
    pub fn extract_slice(content: &str) -> (usize, usize, String) {
        let lines: Vec<&str> = content.lines().collect();
        if lines.is_empty() {
            return (0, 0, String::new());
        }

        let mut start_idx = None;
        let mut end_idx = None;

        for (idx, line) in lines.iter().enumerate() {
            let upper = line.to_uppercase();
            if start_idx.is_none() {
                if upper.contains("BLOCK 1") || upper.contains("METADATA BLOCK") {
                    start_idx = Some(idx);
                }
            } else if upper.contains("BLOCK 2") || upper.contains("OPENING BLOCK") {
                end_idx = Some(idx);
                break;
            }
        }

        let start = start_idx.unwrap_or(0);
        let end = end_idx.unwrap_or(lines.len());
        (start + 1, end, lines[start..end].join("\n"))
    }
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Evaluation Engine for Source Reel Metadata Invariants )
// ============================================================================

impl ReelMetadataLinter {
    /// Evaluates and scores Block 1 for a `.omni` Source Reel.
    pub fn evaluate(
        _file_path: &str,
        slice: &str,
        config: &SpatialLintConfig,
    ) -> MetadataScore {
        let mut identity = 0u32;
        let mut provenance = 0u32;
        let mut witness = 0u32;
        let mut ontology = 0u32;
        let mut spatial = 0u32;
        let mut breakdown = Vec::new();

```

```
        // ------------------------------------------------------------------
        // 1. Identity & Formal Declaration (max 20 pts)
        // ------------------------------------------------------------------
        let has_module_decl = slice.contains("= [Module]:")
            || slice.contains("Module <")
            || slice.contains("Module [")
            || slice.contains("Scroll <");
        if has_module_decl {
            identity += 15;
            breakdown.push("✓ Formal Module Identity declared (+15/15)".into());
        } else {
            breakdown.push("✗ Missing formal module identity declaration (0/15)".into());
        }

        let has_identity_aside = slice.lines().any(|l| {
            (l.contains("= [Module]:") || l.contains("Module <")) && l.contains('(')
                || (slice.contains("= [Module]:") && slice.contains("( This is the Module"))
        }) || (has_module_decl && slice.contains('(') && slice.contains(')'));

        if has_identity_aside {
            identity += 5;
            breakdown.push("✓ Module intent Aside '( ... )' attached (+5/5)".into());
        } else {
            breakdown.push("✗ Missing intent Aside on module identity (0/5)".into());
        }

        // ------------------------------------------------------------------
        // 2. Provenance Attributes & Environment (max 25 pts)
        // ------------------------------------------------------------------
        let has_version = slice.contains("[ Version ]") || slice.contains("version:");
        if has_version {
            provenance += 8;
            breakdown.push("✓ Invariant Version attribute specified (+8/8)".into());
        } else {
            breakdown.push("✗ Missing version attribute in metadata context (0/8)".into());
        }

        let has_author = slice.contains("[ Author ]") || slice.contains("author:");
        if has_author {
            provenance += 6;
            breakdown.push("✓ Author / Organization provenance recorded (+6/6)".into());
        } else {
            breakdown.push("✗ Missing author provenance specification (0/6)".into());
        }

        let has_timestamps = slice.contains("[ Created ]") || slice.contains("[ Updated ]");
        if has_timestamps {
            provenance += 6;
            breakdown.push("✓ Created / Updated temporal anchors inscribed (+6/6)".into());
        } else {
            breakdown.push("✗ Missing temporal created/updated timestamps (0/6)".into());
        }

        let has_system = slice.contains("[ System ]") || slice.contains("system:");
        if has_system {
            provenance += 5;
            breakdown.push("✓ Target system / runtime context declared (+5/5)".into());
        } else {
            breakdown.push("✗ Missing target system context declaration (0/5)".into());
        }

        // ------------------------------------------------------------------
        // 3. Covenant & Witness Grounding (max 20 pts)
        // ------------------------------------------------------------------
        let has_scripture = slice.contains("< Scripture") || slice.contains("[ Witness ]");
        if has_scripture {
            witness += 15;
            breakdown.push("✓ Invariant Covenant / Scripture witness anchored (+15/15)".into());
        } else {
            breakdown.push("✗ Missing scripture/covenant witness anchor (0/15)".into());
        }

        let has_witness_aside = slice.contains("Invariant Witness")
            || (has_scripture && slice.contains('(') && slice.contains("Witness"));
        if has_witness_aside {
            witness += 5;
            breakdown.push("✓ Machine-readable Witness Aside oracle attached (+5/5)".into());
        } else if has_scripture {
```

```
            witness += 3;
            breakdown.push("⚠ Living Aside on witness partially detected (+3/5)".into());
        } else {
            breakdown.push("✗ Missing witness Aside oracle (0/5)".into());
        }

        // ------------------------------------------------------------------
        // 4. Ontological Dimensions & Invariant Types (max 20 pts)
        // ------------------------------------------------------------------
        let has_freestanding_type = slice.contains("< Character > =")
            || (slice.contains("< ") && slice.contains(" > =") && slice.contains('&'));
        let has_nature_semantics = slice.contains("[ Nature ]") || slice.contains("[ Semantics ]");

        if has_freestanding_type {
            let dimensions = [
                "Relation", "Domain", "Substrate", "Inscribe",
                "Transduce", "Exemplar", "Compare", "Sequence", "Bound",
            ];
            let matched_dims = dimensions.iter().filter(|&&d| slice.contains(d)).count();
            if matched_dims >= 7 {
                ontology += 20;
                breakdown.push(format!(
                    "✓ Freestanding Invariant Type defined across {matched_dims}/9 dimensions (+20/20)"
                ));
            } else {
                let pts = (matched_dims as u32) * 2;
                ontology += pts;
                breakdown.push(format!(
                    "⚠ Freestanding Type defined with partial dimensions ({matched_dims}/9) (+{pts}/20)"
                ));
            }
        } else if has_nature_semantics {
            ontology += 20;
            breakdown.push("✓ Nature and Semantics ontology bounds defined (+20/20)".into());
        } else {
            breakdown.push("✗ Missing freestanding invariant type or nature/semantics bounds (0/20)".into());
        }

        // ------------------------------------------------------------------
        // 5. Spatial Geometry & Syntax Hygiene within Block 1 (max 15 pts)
        // ------------------------------------------------------------------
        let lines: Vec<&str> = slice.lines().collect();
        let max_cols = config.max_columns;
        let line_len_violation = lines.iter().any(|l| l.chars().count() > max_cols);

        if !line_len_violation {
            spatial += 10;
            breakdown.push(format!("✓ All Block 1 lines <= {max_cols} columns (+10/10)"));
        } else {
            breakdown.push(format!("✗ Block 1 lines exceed {max_cols}-column spatial ruler (0/10)"));
        }

        let snake_violation = lines.iter().any(|line| {
            let trimmed = line.trim();
            if trimmed.starts_with('#') {
                return false;
            }
            if let Some(start) = line.find('[') {
                if let Some(end) = line[start..].find(']') {
                    let inside = &line[start + 1..start + end];
                    for w in inside.split_whitespace() {
                        if w.contains('_') && !w.starts_with("0x") && !w.starts_with("roll_") {
                            return true;
                        }
                    }
                }
            }
            false
        });

        if !snake_violation {
            spatial += 5;
            breakdown.push("✓ Natural spaced identifier hygiene observed (+5/5)".into());
        } else {
            breakdown.push("✗ snake_case identifiers found inside brackets (0/5)".into());
        }

        MetadataScore::new(identity, provenance, witness, ontology, spatial, breakdown)
```

```
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Unit Tests for Source Reel Block 1 Linting )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_reel_metadata_extraction_and_empty_check() {
        let (start, end, slice) = ReelMetadataLinter::extract_slice("");
        assert_eq!(start, 0);
        assert_eq!(end, 0);
        assert!(slice.is_empty());
    }
}
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/metadata/score.rs RS | 104 LINES | 452 WORDS | 4.2 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Block 1 Metadata & Tape Head Score Model & Rating Taxonomy )
// ============================================================================
//! OmniCode Metadata & Tape Head Scoring Taxonomy
//!
//! Provides the unified scoring model for Block 1 across both `.omni` Source Reels
//! and `.tape` Process Ribbons. While the concrete syntax of reels and tapes differs,
//! both express the exact same ontological foundation and covenant understanding:
//! 1. Identity & Formal Declaration (max 20 pts)
//! 2. Provenance & Operational Environment (max 25 pts)
//! 3. Covenant & Scripture Invariant Grounding (max 20 pts)
//! 4. Ontological Invariants / Cardinal Coordinates (max 20 pts)
//! 5. Spatial Geometry & Naming Hygiene (max 15 pts)

/// Comprehensive rubric breakdown and total score for Block 1 (0..=100).
#[derive(Debug, Clone, PartialEq, Eq)]
pub struct MetadataScore {
    /// Score for formal identity and module/tape declaration (max 20).
    pub identity_score: u32,
    /// Score for provenance, author, version, and environment (max 25).
    pub provenance_score: u32,
    /// Score for scripture/covenant invariant witness grounding (max 20).
    pub witness_score: u32,
    /// Score for ontological dimensions or 6 cardinal clauses (max 20).
    pub ontology_score: u32,
    /// Score for spatial geometry and delimiter naming hygiene (max 15).
    pub spatial_hygiene_score: u32,
    /// Total aggregate score (0..=100).
    pub total: u32,
    /// Letter grade and descriptive rating.
    pub rating: &'static str,
    /// Granular audit notes explaining points awarded or deducted.
    pub breakdown: Vec<String>,
}

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Rating Computation & Rubric Evaluation )
// ============================================================================

impl MetadataScore {
    /// Computes the letter grade and descriptive status from the total score.
    pub fn compute_rating(total: u32) -> &'static str {
        match total {
            100 => "A+ (Exemplary Invariant)",
            90..=99 => "A (Compliant)",
            80..=89 => "B (Adequate Foundation)",
            70..=79 => "C (Needs Provenance / Witness Grounding)",
            50..=69 => "D (Deficient Metadata)",
            _ => "F (Unanchored / Incomplete)",
        }
    }
}

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Subsystem Summary & Aggregation Methods )
// ============================================================================

impl MetadataScore {
    /// Creates a new MetadataScore from component scores and breakdown notes.
    pub fn new(
        identity: u32,
        provenance: u32,
        witness: u32,
        ontology: u32,
        spatial: u32,
        breakdown: Vec<String>,
    ) -> Self {
        let total = identity + provenance + witness + ontology + spatial;
        Self {
            identity_score: identity,
```

```
            provenance_score: provenance,
            witness_score: witness,
            ontology_score: ontology,
            spatial_hygiene_score: spatial,
            total,
            rating: Self::compute_rating(total),
            breakdown,
        }
    }
}

// ============================================================================
// BLOCK 4: CLOSING BLOCK
// ( Unit Tests for MetadataScore Rating Taxonomy )
// ============================================================================

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_metadata_score_rating_tiers() {
        assert_eq!(MetadataScore::compute_rating(100), "A+ (Exemplary Invariant)");
        assert_eq!(MetadataScore::compute_rating(95), "A (Compliant)");
        assert_eq!(MetadataScore::compute_rating(85), "B (Adequate Foundation)");
        assert_eq!(MetadataScore::compute_rating(75), "C (Needs Provenance / Witness Grounding)");
        assert_eq!(MetadataScore::compute_rating(60), "D (Deficient Metadata)");
        assert_eq!(MetadataScore::compute_rating(30), "F (Unanchored / Incomplete)");
    }
}
```

Linter Scaffold Subsystem (`diagnostics::linter::scaffold`)

> ***"The LORD by wisdom hath founded the earth; by understanding hath he established the heavens."*** *— Proverbs 3:19 (KJV)*
>
> ***"For every house is builded by some man; but he that built all things is God."*** *— Hebrews 3:4 (KJV)*

1. Overview & Purpose

The **Scaffold Subsystem** (`src/diagnostics/linter/scaffold`) constitutes Pass 5 of the OmniCode linting pipeline.

While Passes 1 through 4 diagnose and score existing files, the Scaffold Subsystem provides **generative scaffolding**. It produces 100% compliant, canonical 4-Block boilerplate for new `.omni` specifications and `.tape` execution streams.

Every template generated by this subsystem is guaranteed to pass the full `OmniLinter` audit with a perfect **100.0/100.0 score** and achieve a unanimous `QuadStateGate::Proceed` verdict.

2. API Surface & Generators

The module provides two primary generator functions:

1. `scaffold_omni`

```
pub fn scaffold_omni(options: &ScaffoldOptions) -> String
```

Generates a complete `.omni` specification template containing:

- `[BLOCK 1: METADATA]` : Title, Author, Epoch, License, and KJV Scripture Witness.
- `[BLOCK 2: OPENING]` : Semantic namespace declaration and imported prerequisites.
- `[BLOCK 3: BODY]` : Type definitions, record schemas, and state invariants.
- `[BLOCK 4: CLOSING]` : Terminal verification proofs and integrity witness.

2. `scaffold_tape`

```
pub fn scaffold_tape(options: &ScaffoldOptions) -> String
```

Generates a complete `.tape` execution stream template containing:

- `[BLOCK 1: METADATA]` : Stream Title, Author, Epoch, Target Tape, and KJV Scripture Witness.
- `[BLOCK 2: OPENING]` : Specification bindings, accumulator seeding, register zeroing.
- `[BLOCK 3: BODY]` : Ordered execution rolls with SVO verbs (`SET` , `CALCULATE` , `VERIFY`).
- `[BLOCK 4: CLOSING]` : Terminal state assertions, cleanup, and closure instructions.

3. Configuration with `ScaffoldOptions`

```
use omnicode_host::diagnostics::linter::scaffold::{scaffold_omni, ScaffoldOptions};

let options = ScaffoldOptions {
    name: "Vector3D".to_string(),
    author: "OmniCode Core Architecture Team".to_string(),
    witness: "Proverbs 11:1 (KJV)".to_string(),
    description: "3-Dimensional Euclidean Vector Specification".to_string(),
    epoch: "2026.1".to_string(),
    license: "MIT".to_string(),
};
```

```
let omni_source = scaffold_omni(&options);
```

4. Telemetry & Diagnostic Logger Integration

- **Restorator Scaffolding Injection**: When the `Restorator` diagnoses a completely corrupted or missing block, it calls the `scaffold` subsystem to synthesize the missing block skeleton.
- **AI Cortex Template Grounding**: When `AICortex` generates new modules, it seeds its context with scaffolding generated by this subsystem, preventing block order violations or missing witness headers.

5. Verification & Tests

The `scaffold` module includes 4 unit tests verifying:

- Generated `.omni` scaffolds pass the full linter with `100.0/100.0` and `QuadStateGate::Proceed`.
- Generated `.tape` scaffolds pass the full linter with `100.0/100.0` and `QuadStateGate::Proceed`.
- Custom authors, descriptions, and Scripture witnesses are preserved accurately in the generated output.

Run module tests:

```
cargo test --lib diagnostics::linter::scaffold
```

<TRACK 1> : {Host Engine & Virtual Machine}/src/diagnostics/linter/scaffold/mod.rs

RS | 266 LINES | 1335 WORDS | 10.3 KB

```rust
// ============================================================================
// BLOCK 1: METADATA BLOCK
// ( Canonical Code Scaffolding Templates for .omni, .tape, and .scroll )
// ============================================================================
//! # Canonical Code Scaffolding Engine
//!
//! Generates compliant, 80-column bounded skeleton templates for all OmniCode file formats:
//! - `.omni`: 4-Block Source Reel
//! - `.tape`: Sequential Process Ribbon
//! - `.scroll`: Cognitive Episodic Scroll
//!
//! # Scripture Witness:
//! - **Proverbs 3:19 (KJV)**:
//!   "The LORD by wisdom hath founded the earth; by understanding hath he established the heavens."
//!   (Scaffolding establishes the architectural foundation and understanding for all created files).
//! - **Hebrews 8:5 (KJV)**:
//!   "...See, saith he, that thou make all things according to the pattern shewed to thee in the mount."
//!   (Every generated template strictly follows the divine 4-block pattern).
// ============================================================================

use crate::core::filetype::OmniFileType;

// ============================================================================
// BLOCK 2: OPENING BLOCK
// ( Scaffolder Structure Definition )
// ============================================================================

pub struct LinterScaffolder;

// ============================================================================
// BLOCK 3: BODY BLOCK
// ( Template Generation Engine for Scrolls, Reels, and Tapes )
// ============================================================================

impl LinterScaffolder {
    /// Auto-scaffolds a compliant 4-Block skeleton for an episodic cognitive scroll (.scroll).
    pub fn auto_scaffold_scroll(session_id: &str, prompt: &str) -> String {
        OmniFileType::EpisodicScroll.scaffold_template(session_id, prompt)
    }

    /// Auto-scaffolds a compliant 4-Block skeleton for a given module name (.omni),
    /// strictly bounded to the 80-column ruler with 3-tier boundary comments.
    pub fn auto_scaffold_omni(module_name: &str, intent_aside: &str) -> String {
        format!(
r#"# ============================================================================
# BLOCK 1: METADATA BLOCK
# ( Invariant Type: < Idea {{ Type }} : {module_name} > )
# ============================================================================
= [Module]: < Lexicon {{ {module_name} }} > & [ Noun {{ {module_name} }} ]
    ( {intent_aside} )
{{
    [ Version ]:   {{ "1.0.0" }}
        ( Version 1.0.0 ),

    [ Author ]:    {{ "CreativeWorkzStudio LLC / ProfessorSeanEX" }}
        ( System Architect ),

    [ System ]:    {{ OmniCode {{ Core }} }}
        ( Core Subsystem ),

    [ Witness ]:   < Scripture {{ Exodus 31:18 KJV }} >
        {{ "Written with the finger of God." }}
        ( Invariant Scripture Anchor ),

    [ Nature ]:    < InvariantType > & {{ BaseBlock }}
        ( Nature is Invariant Type ),

    [ Semantics ]: {{ StandardLibrary :: Specification }}
        ( Semantics specification )
}}

< {module_name} > = < Idea {{ Type }} > & [ Noun {{ {module_name} }} ]
    ( Invariant Definition )
```

```
{{
    [ Relation ]: [ {module_name} ] : [ Unit ] :: [ Trit ] : [ Word ]
        ( Ontological Relation ),

    [ Substrate ]: [ 1 Byte {{ 8 Bits }} ] & [ 1 Cell {{ 5 Trits }} ]
        ( Dual Substrate ),

    [ Bound ]: {{ {module_name} }} = [ Enclosed ]: TRUE
        ( Invariant Single Cell Boundary )
}}

# ================================================================================
# BLOCK 2: OPENING BLOCK
# ================================================================================
Opening {{
    [ Entity ]: < {module_name} > & [ Cell ]
        ( Entity allocated in cell ),

    [ Cell ]: [ 1 ] & {{ 5 Trits :: 1 Byte }}
        ( One cell container ),

    [ Preconditions ]: [ (+1) plus (-1) <=> 0 ] & [ Status = +1:PROCEED ]
        ( Equilibrium verified )
}}

# ================================================================================
# BLOCK 3: BODY BLOCK
# ================================================================================
Body [
    [ ROLL 1 ] : [ Inscribe ] :
        [ Input ] --> INSCRIBE [ Cell ] = [ Entity` ]
        ( Inscribe input into Cell yielding Entity-prime ),

    [ ROLL 2 ] : [ Compare ] :
        {{ Entity` = Input }} --> [ Status` = +1:PROCEED ]
        | {{ Entity` != Input }} --> [ Status` = -1:ROLLBACK ]
        ( Evaluate identity and verify bounds )
]

# ================================================================================
# BLOCK 4: CLOSING BLOCK
# ================================================================================
Closing
    ( Sealing Gate: Validate, tripwire, debug, log, and affirm execution )
{{
    [ Verify ]: [
        [ ROLL 1 ] = PROCEED ::
        [ ROLL 2 ] = PROCEED
    ]
    ( Verify that all rolls achieved affirmative PROCEED ),

    [ Validate ]:
        {{ Entity` <= [ 1 Cell ] }} & {{ [ 8 Bits ] <=> [ 5 Trits ] }}
        ( Validate single cell bounds and substrate parity ),

    [ Tripwire ]:
        {{ Status` = +1:PROCEED }} --> [ ARMED ]
        | {{ Status` != +1:PROCEED }} --> [ TRIPPED ]
            --> [ Commit Status = -1:ROLLBACK ]
        ( Circuit breaker vetoes on anomaly ),

    [ Debug ]:
        {{ [ Entity` ] :: [ Cell` ] :: [ Status` ] }}
        ( Expose diagnostic telemetry snapshot ),

    [ Log ]: [ Journal ]:
        INSCRIBE {{ [ {module_name} Verified ] :: [ Entity` ] }}
        ( Inscribe event record into system journal ),

    [ Commit Status ]: [
        {{ [ Verify ] = PROCEED }} & {{ [ Tripwire ] = ARMED }}
            --> [ +1:PROCEED ] |
        {{ [ Status` ] = PENDING }}
            --> [ 0:AWAIT ] :: REPEAT |
        {{ [ Status` ] = NULL }}
            --> [ HALT:NULL ] |
        {{ [ Tripwire ] = TRIPPED }} | {{ [ Verify ] != PROCEED }}
            --> [ -1:ROLLBACK ]
```

```
    ]
    ( Fourfold state machine: Proceed, Await, Halt, Rollback )
}}
"#
        )
    }

    /// Auto-scaffolds a compliant `.tape` sequential process ribbon skeleton,
    /// strictly bounded to 80 columns with Tape Head and Cardinal Clauses.
    pub fn auto_scaffold_tape(tape_name: &str, intent_aside: &str) -> String {
        format!(
r#"# ================================================================================
# PROCESS TAPE: {upper} INVARIANT TYPE EXECUTION RIBBON
# ================================================================================
# ARCHITECTURAL TOPOLOGY:
#   Executes atomic lifecycle for Invariant Type < {tape_name} >.
#   - Defined Space: [ 1 Cell ] {{ 5 Trits :: 1 Byte }}
#   - Execution Domain: Lexicon {{ Types of Ideas }}
#   - Witness: Scripture Exodus 31:18 KJV
# ================================================================================

= [Tape]: < Lexicon {{ Types of Ideas }} > & [ Process {{ {tape_name} }} ] {{
    WHO   {{ OmniCode Lexicon }} ::
    WHAT  {{ {tape_name} }} --> DISPATCH {{ Process }} ::
    WHEN  {{ Input Event }} ::
    WHERE {{ {tape_name} Cell }} = [ 1 ] ::
    WHY   ( {intent_aside} ) ::
    HOW   [ +1 : PROCEED ] ::
    < Scripture {{ Exodus 31:18 KJV }} > ::
    :: {{ "Written with the finger of God." }} ::
    :: ( Invariant Witness Exodus 31:18: Written with finger of God )
}}

# ================================================================================
# DECLARED ROLLS: ATOMIC INVARIANT EXECUTION
# ================================================================================

# --------------------------------------------------------------------------------
# ROLL 1: INITIAL STATE ALLOCATION
# --------------------------------------------------------------------------------
[ ROLL 1 ] : [ Inscribe ] :
    [ Input ] --> INSCRIBE [ Cell ] = [ {tape_name}` ]
    ( Inscribe initial input into cell container )

# --------------------------------------------------------------------------------
# ROLL 2: INVARIANT VERIFICATION
# --------------------------------------------------------------------------------
[ ROLL 2 ] : [ Compare ] :
    {{ {tape_name}` = Input }} ! GUARD --> [ Status` = +1:PROCEED ]
    | {{ {tape_name}` != Input }} --> [ Status` = -1:ROLLBACK ]
    ( Verify invariant bounds under guard )

# --------------------------------------------------------------------------------
# ROLL 3: COMMIT STATUS & FOURFOLD STATE MACHINE
# --------------------------------------------------------------------------------
[ ROLL 3 ] : [ Commit Status ] : [
    {{ [ Verify ] = PROCEED }} & {{ [ Tripwire ] = ARMED }}
        --> [ +1:PROCEED ] |
    {{ [ Status` ] = PENDING }}
        --> [ 0:AWAIT ] :: REPEAT |
    {{ [ Status` ] = NULL }}
        --> [ HALT:NULL ] |
    {{ [ Tripwire ] = TRIPPED }} | {{ [ Verify ] != PROCEED }}
        --> [ -1:ROLLBACK ]
]
( Fourfold commit status: Proceed, Await, Halt, Rollback )
"#,
            upper = tape_name.to_uppercase(),
            tape_name = tape_name,
            intent_aside = intent_aside
        )
    }
}

// ================================================================================
// BLOCK 4: CLOSING BLOCK
// ( Subsystem Unit Tests & Scaffolding Certification )
// ================================================================================
```

```
#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_auto_scaffold_omni_four_blocks() {
        let code = LinterScaffolder::auto_scaffold_omni("Vector", "Dynamic vector space");
        assert!(code.contains("BLOCK 1: METADATA BLOCK"));
        assert!(code.contains("BLOCK 2: OPENING BLOCK"));
        assert!(code.contains("BLOCK 3: BODY BLOCK"));
        assert!(code.contains("BLOCK 4: CLOSING BLOCK"));
        assert!(code.contains("Vector"));
    }

    #[test]
    fn test_auto_scaffold_tape_ribbon() {
        let code = LinterScaffolder::auto_scaffold_tape("Echo", "Echo input to stdout");
        assert!(code.contains("= [Tape]:"));
        assert!(code.contains("WHO"));
        assert!(code.contains("WHAT"));
        assert!(code.contains("WHEN"));
        assert!(code.contains("WHERE"));
        assert!(code.contains("WHY"));
        assert!(code.contains("HOW"));
        assert!(code.contains("[ ROLL 1 ]"));
        assert!(code.contains("Echo"));
    }

    #[test]
    fn test_auto_scaffold_scroll() {
        let code = LinterScaffolder::auto_scaffold_scroll("sess_99", "Calculate Fibonacci");
        assert!(code.contains("Scroll < EpistemicTrace.sess_99 >"));
        assert!(code.contains("Calculate Fibonacci"));
    }
}
```